

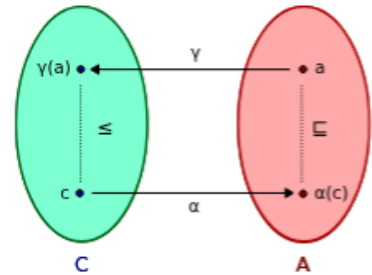
PROGRAM ANALYSIS

Galois

Abstract Interpretation = a technique to formally reason on approximations

Ingredients of Abstract Interpretation

- A concrete domain C
- An abstract domain A
- A concretization function γ that relates the abstract domain to the concrete one
- An abstraction function α that connects the concrete domain to the abstract one
- Together they form a **Galois Connection**
- In this lecture we first revisit elements of order theory (C and A are ordered) and formalize the concept of Galois Connection and Galois Insertion



Poset = partial order on a set X (reflexive, anti-symmetric, transitive)

validation = program P satisfying a specification S as the set inclusion $P \subseteq S$

soundness = outputs a result that is coarser than the actual behavior

$a \sqcup b$ = lub of a and b

$a \sqcap b$ = glb of a and b

$\sqcup A$ = lub of A

$\sqcap A$ = glb of A

$(\varphi(X), \subseteq) \implies \text{lub} = \cup, \text{glb} = \cap$

chain = in a poset is a subset that is totally ordered $\forall c, d \in C : (c \sqsubseteq d) \vee (d \sqsubseteq c)$

Una **ascending chain** (catena ascendente) in un poset $(X, \sqsubseteq)$ è una sequenza numerabile di elementi $\{x_i\}_{i \in \mathbb{N}}$ tale che ogni elemento precede il successivo nell'ordine parziale.

A poset $(X, \sqsubseteq)$ has finite height $n \in \mathbb{N}$ if the cardinality of the longest chain is $n + 1$

A poset $(X, \sqsubseteq)$ satisfies the Ascending Chain Condition (ACC) when each ascending chain converges (i.e., there are no infinite ascending chains)

ACC abstract domains guarantee that the analysis always terminates!

CPO = a poset such that every chain has a lub. ($\sqcup \emptyset = \perp$)

Pointed CPOs = CPOs with bottom ($\perp$)

Lattice $(X, \sqsubseteq, \sqcup, \sqcap)$ = poset such that $\forall a, b \in X: a \sqcup b$ and $a \sqcap b$ exists

Complete Lattice $(X, \sqsubseteq, \sqcup, \sqcap, \perp, \top)$ = poset such: $\forall A \subseteq X: \sqcup A$ exists $\sqcap A$ exists

$\perp = \text{lub}(\emptyset) = \text{glb}(X)$

$\top = \text{lub}(X) = \text{glb}(\emptyset)$

interval lattice = $(\{[a, b] \mid a, b \in \mathbb{Z}, a \leq b\} \cup \perp, \sqsubseteq, \sqcup, \sqcap) \rightarrow [a, b] \sqcup [a', b'] = [\min(a, a'), \max(b, b')]$
(not all lubs exist)

A poset $(X, \sqsubseteq)$ is a **CPO** if every (possibly empty) chain has a lub (must have $\perp$)

A poset $(X, \sqsubseteq)$ is a **lattice** if every pair of elements has both lub and glb

A poset $(X, \sqsubseteq)$ is a **complete lattice** if every (possibly empty) subset has both lub and glb

$\perp = \text{lub}(\emptyset) = \text{glb}(X)$ and $\top = \text{lub}(X) = \text{glb}(\emptyset)$

lub and glb are commutative and associative

Each finite lattice is a complete lattice

Each complete lattice is a CPO (and a lattice)

Definition 2.5 (Complete lattice). A complete lattice $(X, \sqsubseteq, \sqcup, \sqcap, \perp, \top)$ is a poset such that:

1. $\forall A \subseteq X: \sqcup A$ exists;

2. $\forall A \subseteq X: \sqcap A$ exists;

prove that: 1 holds iff 2 holds

Ecco la dimostrazione del fatto che la condizione (1) implica la condizione (2):

1. Sia $B \subseteq X$ un sottoinsieme arbitrario. Vogliamo dimostrare che $\sqcap B$ esiste.
2. Definiamo l'insieme dei limiti inferiori: Sia $L = \{x \in X \mid \forall b \in B, x \sqsubseteq b\}$ l'insieme di tutti gli elementi di X che sono minoranti di B .
3. Appliciamo l'ipotesi (1): Per ipotesi, ogni sottoinsieme di X ha un lub. Quindi esiste $g = \sqcup L$.
4. Verifichiamo che g sia un minorante: Poiché ogni elemento $b \in B$ è, per definizione di L , un maggiorante dell'intero insieme L , e poiché g è il minimo dei maggioranti di L , allora deve valere $g \sqsubseteq b$ per ogni $b \in B$. Quindi g è un minorante di B .
5. Verifichiamo che g sia il massimo dei minoranti: Ogni altro minorante x di B appartiene per definizione a L . Dato che $g = \sqcup L$, allora $x \sqsubseteq g$ per ogni $x \in L$. Questo rende g il massimo dei minoranti, ovvero $g = \sqcap B$.

L'implicazione inversa ($2 \implies 1$) si ottiene in modo identico applicando il **Principio di Dualità**: se ogni sottoinsieme ha un glb, allora $\sqcup A$ può essere definito come il glb dell'insieme dei suoi maggioranti.

lcm = least common multiple
gcd = greatest common divisor

monotonicity = respect the order
continuity = preserve limits
extensivity = $\forall a \in A: a \sqsubseteq f(a)$
reductive = $\forall a \in A: f(a) \sqsubseteq a$

Extensivity is essentially useful when combined with monotonicity to bootstrap iteration: starting from an arbitrary x , we have $x \sqsubseteq f(x)$ by extensivity; then, applying monotonicity $f(x) \sqsubseteq f^2(x)$, etc., so the sequence $\{f^i(x) | i \in \mathbb{N}\}$ is increasing.

Abstract Domain = $(A, \sqsubseteq)$ expresses some properties of the concrete values (The order $\sqsubseteq$ on the abstract domain reflects the precision)

Abstract Interpretation = $(C, A, \gamma: A \rightarrow C, \alpha: C \rightarrow A)$

Concretization $(\gamma: A \rightarrow C)$ = a monotone function that maps abstract a into the greatest concrete that it approximates.

$a \in A$ is a sound abstraction of $c \in C \iff c \leq \gamma(a)$

Given a specification S , then if:

- the specification S can be exactly represented in the abstract as $S^\sharp$, i.e., $S = \gamma(S^\sharp)$, and
- the result $P^\sharp$ of a static analysis is a sound abstraction of the concrete semantics P , i.e., $P \subseteq \gamma(P^\sharp)$ and
- $P^\sharp \sqsubseteq S^\sharp$ i.e., the analyzer proves in the abstract that the program satisfies its specification,

then, the program indeed satisfies its specification in the concrete, i.e., $P \subseteq S$. Note that, not only the computation of $P^\sharp$, but also the check $P^\sharp \sqsubseteq S^\sharp$, are done entirely in the abstract world. Hence, effective abstract algorithms lead to sound and effective static analyses.

67

Abstraction $(\alpha: C \rightarrow A)$ = a monotone function that maps c concrete into the most precise abstract element that approximates it.

Galois connections

Definition 2.12 (Galois connection). Given two posets $(C, \leq)$ and $(A, \sqsubseteq)$, the pair $(\alpha : C \rightarrow A, \gamma : A \rightarrow C)$ is a Galois connection if:

$$\forall a \in A, c \in C. \quad c \leq \gamma(a) \iff \alpha(c) \sqsubseteq a \quad (2.5)$$

which is denoted as $(C, \leq) \xleftrightarrow[\alpha]{\gamma} (A, \sqsubseteq)$.

α and γ are said to be adjoint functions, where the abstraction α is the upper adjoint and the concretization γ is the lower adjoint. ■

Definition 2.13. A Galois connection $(C, \leq) \xleftrightarrow[\alpha]{\gamma} (A, \sqsubseteq)$ is a Galois embedding if $\alpha \circ \gamma = id$

Galois embeddings are also called **Galois insertions**

GIs from GCs

How to “repair” a GC into a GI by eliminating the “useless” abstract values?

We take the image of C as a “reduced” abstract domain $A^{\text{red}} = \alpha(C)$ and restrict the original abstraction and concretization maps to operate on A^{red} :

$$\alpha^{\text{red}} : C \rightarrow A^{\text{red}} \quad \text{with} \quad \alpha^{\text{red}}(c) = \alpha(c)$$

$$\gamma^{\text{red}} : A^{\text{red}} \rightarrow C \quad \text{with} \quad \gamma^{\text{red}}(a) = \gamma(c)$$

We get a GI that has the same “expressive power” of the original GC

Alternative definition of GC

Definition 2.12 (Galois connection). *Given two posets $(C, \leq)$ and $(A, \sqsubseteq)$, the pair $(\alpha : C \rightarrow A, \gamma : A \rightarrow C)$ is a Galois connection if:*

$$\forall a \in A, c \in C, c \leq \gamma(a) \iff \alpha(c) \sqsubseteq a \quad (2.5)$$

The fundamental property of Galois connections (2.5) provides, in a very compact form, a strong connection between the concrete and the abstract world. The following theorem provides a less compact, but equivalent view:

Theorem 2.3 (Alternate characterization of Galois connections). *We have a Galois connection $(C, \leq) \xleftrightarrow[\alpha]{\gamma} (A, \sqsubseteq)$ if and only if the function pair (α, γ) satisfies all the following properties:*

1. γ is monotonic;
2. α is monotonic;
3. $\gamma \circ \alpha$ is extensive, i.e., $\forall c \in C: c \sqsubseteq \gamma(\alpha(c))$;
4. $\alpha \circ \gamma$ is reductive, i.e., $\forall a \in A: \alpha(\gamma(a)) \leq a$. ■

87

Properties of GC

Theorem 2.4 (Galois connection properties). *Given a Galois connection $(C, \leq) \xleftrightarrow[\alpha]{\gamma} (A, \sqsubseteq)$, we have:*

1. $\gamma \circ \alpha \circ \gamma = \gamma$ and $\alpha \circ \gamma \circ \alpha = \alpha$; **round trips**
2. $\alpha \circ \gamma$ and $\gamma \circ \alpha$ are idempotent; **idempotency**
3. $\forall c \in C: \alpha(c) = \sqcap \{ a \mid c \leq \gamma(a) \}$; **γ induces α**
4. $\forall a \in A: \gamma(a) = \vee \{ c \mid \alpha(c) \sqsubseteq a \}$; **α induces γ**
5. α maps concrete lubs to abstract lubs:
 $\forall X \subseteq C$: if $\vee X$ exists, then $\alpha(\vee X) = \sqcup \{ \alpha(x) \mid x \in X \}$; **α preserves lubs**
6. γ maps abstract glbs to concrete glbs:
 $\forall X \subseteq A$: if $\sqcap X$ exists, then $\gamma(\sqcap X) = \wedge \{ \gamma(x) \mid x \in X \}$. **γ preserves glbs** ■

idempotency = $f(f(x)) = f(x)$

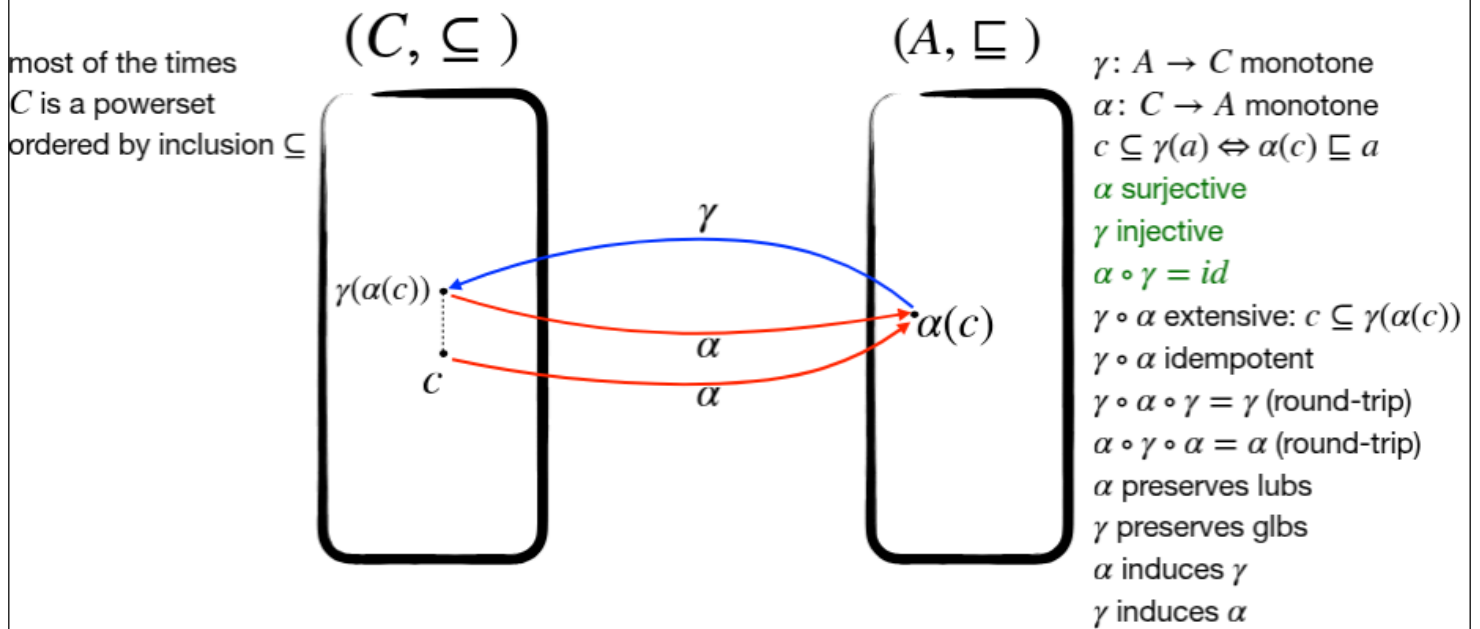
Definition 2.13. A Galois connection $(C, \leq) \xleftrightarrow[\alpha]{\gamma} (A, \sqsubseteq)$ is a Galois embedding if any of the following, equivalent properties hold:

1. α is surjective: $\forall a \in A: \exists c \in C: \alpha(c) = a$;
2. γ is injective: $\forall a, a' \in A: \gamma(a) = \gamma(a') \implies a = a'$;
3. $\alpha \circ \gamma = id$.

We denote a Galois embedding as $(C, \leq) \xleftrightarrow[\alpha]{\gamma} (A, \sqsubseteq)$. ■

Properties 1 and 2 state that every abstract element is useful: there is no redundancy as every abstract element abstracts a different concrete element. Property 3 allows us to view the abstract domain A as isomorphic to a subset of the concrete domain C . This is depicted in Fig. 2.8.(b).

Focus on Galois insertions



$A: C \rightarrow C$

$A(c) = \gamma(\alpha(c)) \rightarrow$ closure operator (monotone, extensive, idempotent)

$A(c)$ = set of expressible elements

$F: C \rightarrow C$ = concrete operator

$F^\#: A \rightarrow A$ = corresponding operator in the abstract domain

$F^\#$ is a **sound abstraction** of F if $\forall a \in A: F(\gamma(a)) \subseteq \gamma(F^\#(a))$

$F^\#$ is a **sound abstraction** of F if $\forall a \in A: \alpha(F(\gamma(a))) \sqsubseteq F^\#(a)$

$F^\#$ is a **sound abstraction** of F if $\forall a \in A: \alpha(F(c)) \subseteq F^\#(\alpha(c))$

Sound approximation

$$\alpha \circ F \circ \gamma \sqsubseteq F^\# \iff \alpha \circ F \sqsubseteq F^\# \circ \alpha$$

$$\alpha \circ F \circ \gamma \sqsubseteq F^\#$$

(function composition) $\iff$

$$\alpha \circ F \circ \gamma \circ \alpha \sqsubseteq F^\# \circ \alpha$$

($\gamma \circ \alpha$ extensive) $\implies$

$$\alpha \circ F \sqsubseteq F^\# \circ \alpha$$

$$\alpha \circ F \sqsubseteq F^\# \circ \alpha$$

(function composition) $\implies$

$$\alpha \circ F \circ \gamma \sqsubseteq F^\# \circ \alpha \circ \gamma$$

($\alpha \circ \gamma$ reductive) $\implies$

$$\alpha \circ F \circ \gamma \sqsubseteq F^\#$$

any sound approximation $F^\#$ is less precise than $F^A \triangleq \alpha \circ F \circ \gamma$

$$F^A \triangleq \alpha \circ F \circ \gamma$$

Given a Galois connection and a concrete operator F the best abstraction (bca) of F is given by F^A

Sound operator composition

Theorem. Let $F_1 : C \rightarrow C$ and $F_2 : C \rightarrow C$ and be concrete (monotone) operators and let $F_1^\# : A \rightarrow A$ and $F_2^\# : A \rightarrow A$ some corresponding sound abstractions. Then, $F_2^\# \circ F_1^\#$ is a sound abstraction of $F_2 \circ F_1$.

Proof. $(F_2 \circ F_1) \circ \gamma \subseteq F_2 \circ \gamma \circ F_1^\# \subseteq \gamma \circ (F_2^\# \circ F_1^\#)$

$$F_1 \circ \gamma \subseteq \gamma \circ F_1^\#$$

$$F_2 \circ \gamma \subseteq \gamma \circ F_2^\#$$

BCA composition?

Remark. Let $F_1 : C \rightarrow C$ and $F_2 : C \rightarrow C$ and be concrete (monotone) operators and let $F_1^A : A \rightarrow A$ and $F_2^A : A \rightarrow A$ their corresponding BCAs.

Then, $F_2^A \circ F_1^A$ is a sound abstraction of $F_2 \circ F_1$ but not necessarily its BCA.

$$F_2^A \circ F_1^A = \alpha \circ F_2 \circ \gamma \circ \alpha \circ F_1 \circ \gamma \quad \text{vs.} \quad \alpha \circ (F_2 \circ F_1) \circ \gamma$$

The composition $F_2^A \circ F_1^A$ performs an additional trip $\gamma \circ \alpha$ through the abstract world that represents a potential source of precision loss

Thus, in general: $(F_2 \circ F_1)^A \subseteq F_2^A \circ F_1^A \subseteq F_2^\# \circ F_1^\#$

the composition of two GCs is a GC

Piranha domain:

$$(N^\infty, \leq), \gamma(\top) = \infty, \gamma(\perp) = 3, \alpha(n) = \perp \text{ (if } n \leq 3), \alpha(n) = \top \text{ (otherwise)}$$

Pirandello domain:

$$(N^\infty, \leq), \gamma(\top) = \infty, \gamma(1) = 1, \gamma(\perp) = 0, \alpha(0) = \perp, \alpha(1) = 1, \alpha(n) = \top \text{ (otherwise)}$$

Constant domain:

$$(\wp(\mathbb{V}), \subseteq), \gamma(\top) = \mathbb{V}, \gamma(k_n) = k_n, \gamma(\perp) = \emptyset$$

Constant set domain:

$$(\wp(\mathbb{V}), \subseteq), \gamma(\top) = \mathbb{V}, \gamma(X^\#) = X^\# \text{ (if } X^\# \subseteq \wp_k(K))$$

BCA for product in $A_{|k|}$:

$$X^\# \times^\# Y^\# \triangleq \alpha(\gamma(X^\#) \times \gamma(Y^\#)).$$

Case 1: Empty set. If $X^\# = \emptyset$ or $Y^\# = \emptyset$, the result is $\emptyset$.

Case 2: Zero element. If $X^\# = 0$ or $Y^\# = 0$ (and neither is $\emptyset$), the result is 0 . This holds even if the other argument is $\top^\#$, as the concrete product of any set with zero is 0 .

Case 3: Unknown information ($\top^\#$). If $X^\# = \top^\#$ or $Y^\# = \top^\#$ (and neither is $\emptyset$ or 0), the result is $\top^\#$.

Case 4: Bounded sets. If both arguments are finite sets $X^\#, Y^\# \subseteq \mathcal{P}_{\leq k}(\mathbb{I})$, we first compute the concrete set of products $S = \{x \cdot y \mid x \in X^\#, y \in Y^\#\}$:

- If $|S| \leq k$, the result is S .
- If $|S| > k$, the result is $\top^\#$.

This definition ensures that the abstract operation mimics the concrete behavior with no loss of precision beyond what is forced by the domain's size constraint k

adjoint abstraction map $\alpha: \wp(\mathbb{Z}) \rightarrow A_{|2|}$:

In the **bounded set domain** $A_{|2|}$, which tracks sets containing at most two integers, the **adjoint abstraction map** $\alpha: \wp(\mathbb{Z}) \rightarrow A_{|2|}$ maps a concrete set S to the most precise representation available in the abstract domain.

It is defined as follows:

$$\alpha(S) \triangleq \{S \text{ if } |S| \leq 2 \top \text{ if } |S| > 2$$

Key Characteristics

- **Precision:** If the concrete set has two or fewer elements (including the empty set $\emptyset$ or singletons), α is **exact**, meaning $\gamma(\alpha(S)) = S$.
- **Over-approximation:** If the set contains more than two elements, the domain lacks a specific representation for it, so it must be abstracted to $\top$ (representing the entire set $\mathbb{Z}$), which is the smallest sound abstraction available.
- **Galois Insertion:** Because this mapping ensures every abstract element corresponds to a unique concrete set (no redundancy), it forms a **Galois insertion** where α is surjective and γ is injective.

Interval domain:

$$(\{[a, b] \mid a \in \mathbb{Z} \cup \{+\infty\}, a \leq b\} \cup \{\perp\}, \subseteq, \sqcup, \cap, \perp, [-\infty, +\infty])$$

$$\gamma(\perp) = \emptyset, \gamma([a, b]) = \{x \in \mathbb{Z} \mid a \leq x \leq b\}$$

$$[a, b] \sqsubseteq [c, d] \iff (c \leq a) \wedge (b \leq d)$$

$$[a, b] \sqcup^\# [c, d] \triangleq [\min(a, c), \max(b, d)]$$

$$[a, b] \cap^\# [c, d] \triangleq [\max(a, c), \min(b, d)] \text{ if } \max(a, c) \leq \min(b, d)$$

$$[a, b] \cap^\# [c, d] \triangleq \perp \text{ otherwise}$$

$$[a, b] -^\# [c, d] = [a - d, b - c]$$

$$[a, b] +^\# [c, d] = [a + c, b + d]$$

Widening

Definition 2.17. A binary operator $\nabla : A \times A \rightarrow A$ is a widening operator in an abstract domain $(A, \sqsubseteq)$ if:

1. it computes upper bounds: $\forall x, y \in A. x \sqsubseteq x \nabla y$ and $y \sqsubseteq x \nabla y$
2. and it enforces convergence: for any sequence $(y^i)_{i \in \mathbb{N}}$ in A , the sequence $(x^i)_{i \in \mathbb{N}}$ computed as $x^0 \stackrel{\text{def}}{=} y^0, x^{i+1} \stackrel{\text{def}}{=} x^i \nabla y^{i+1}$ stabilizes in finite time. $\exists k \geq 0: x^{k+1} = x^k$. ■

Theorem 2.9. If f is a monotonic operator in a complete concrete lattice and g is a sound abstraction of f , then the following iteration:

$$\begin{array}{lcl} x^0 & \stackrel{\text{def}}{=} & \perp \\ x^{i+1} & \stackrel{\text{def}}{=} & x^i \nabla g(x^i) \end{array} \quad (2.6)$$

converges in finite time, and its limit x is a sound abstraction of the least fixpoint $\text{lfp } f$: $\text{lfp } f \leq \gamma(x)$. ■

67

Some widening

naive widening

$$x \nabla y \stackrel{\text{def}}{=} \begin{cases} x & \text{if } y \sqsubseteq x \\ \top & \text{otherwise} \end{cases}$$

The standard interval widening consists in replacing any unstable upper bound with $+\infty$, and any unstable lower bound with $-\infty$:

standard widening

$$[a, b] \nabla_b [c, d] \stackrel{\text{def}}{=} \left[\begin{cases} a & \text{if } a \leq c \\ -\infty & \text{otherwise} \end{cases}, \begin{cases} b & \text{if } b \geq d \\ +\infty & \text{otherwise} \end{cases} \right] \quad (4.7)$$

widening with threshold

$$[a, b] \nabla_b^0 [c, d] \stackrel{\text{def}}{=} \left[\begin{cases} a & \text{if } a \leq c \\ 0 & \text{if } 0 \leq c < a \\ -\infty & \text{otherwise} \end{cases}, \begin{cases} b & \text{if } b \geq d \\ 0 & \text{if } 0 \geq d > b \\ +\infty & \text{otherwise} \end{cases} \right] \quad (4.12)$$

68

domain	invariants	memory cost	time cost	Sect.
affine equalities	$\sum_i \alpha_i V_i = \beta$	$\mathcal{O}(\mathbb{V} ^2)$	$\mathcal{O}(\mathbb{V} ^3)$	5.2
zones	$V_j - V_i \leq c$	$\mathcal{O}(\mathbb{V} ^2)$	$\mathcal{O}(\mathbb{V} ^3)$	5.4
octagons	$\pm V_j \pm V_i \leq c$	$\mathcal{O}(\mathbb{V} ^2)$	$\mathcal{O}(\mathbb{V} ^3)$	5.4.5
template	$\sum_i \alpha_i V_i \geq \beta$	$\mathcal{O}(m)$	polynomial	5.5
polyhedra	$\sum_i \alpha_i V_i \geq \beta$	exponential	exponential	5.3

affine inequalities

80

Convex Polyhedra domain:

Convex Polyhedra domain

sets of numerical constraints of the form

$$c_1x + c_2y \leq c$$

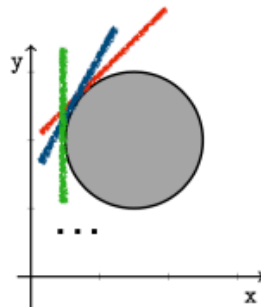
(at most two variables per constraint, with coefficients)

is a **relational** domain, can be used
when **intervals are not expressive enough**

does not admit a best abstraction

No best approximation

Computing the best approximation is not always possible



In practice we can use abstractions
as precise as possible, but maybe not the best
(still best abstractions can exist for some sets of states)

Octagons domain:

Octagons domain

sets of numerical constraints of the form

$$\pm x \pm y \leq c$$

(at most two variables per constraint, with **unit** coefficients)

is a **relational** domain, can be used
when **intervals are not expressive enough**
and **polyhedra are more costly**
(allow trading precision for efficiency)

Congruence domain:

abstract elements of the form $a\mathbb{Z} + b$

(with $a \in \mathbb{N}$, $b \in \mathbb{Z}$, γ is not injective)

sometimes is assumed $0 \leq b < a$ (when $a \neq 0$)

(then γ becomes injective)

- the greatest element is $1\mathbb{Z} + 0$
- singletons $\{c\}$ are represented as $0\mathbb{Z} + c$
- the bottom element is explicitly added

$$\gamma(\perp) = \emptyset$$

$$\gamma(a\mathbb{Z} + b) = \{ak + b \mid k \in \mathbb{Z}\}$$

$$\alpha(X) = \sqcup_{x \in X}^{\#} 0\mathbb{Z} + x$$

Although the domain has an infinite height, there is no infinite strictly increasing chain!

$$-^{\#}(a\mathbb{Z} + b) = (a\mathbb{Z} + (-b))$$

$$(a\mathbb{Z} + b) +^{\#} (c\mathbb{Z} + d) = (\gcd(a, c)\mathbb{Z} + (b + d))$$

$$(a\mathbb{Z} + b) -^{\#} (c\mathbb{Z} + d) = (\gcd(a, c)\mathbb{Z} + (b - d))$$

$$(a\mathbb{Z} + b) \times^{\#} (c\mathbb{Z} + d) = (\gcd(ac, ad, bc)\mathbb{Z} + bd)$$

Meet

$$(a\mathbb{Z} + b) \sqcap^{\#} (a'\mathbb{Z} + b') = \begin{cases} \perp & \text{if } b \not\equiv b' \pmod{\gcd(a, a')} \\ a''\mathbb{Z} + b'' & \text{otherwise} \end{cases}$$

Precise!

greatest common divisor

least common multiple

where $a'' = \text{lcm}(a, a')$ and b'' satisfies $\begin{cases} b'' \equiv b \pmod{a} \\ b'' \equiv b' \pmod{a'} \end{cases}$

e.g. $4\mathbb{Z} + 1 \sqcap^{\#} 6\mathbb{Z} + 3$

$\gcd(4, 6) = 2 \quad 1 \equiv 3 \pmod{2} \quad \text{Compatible!}$

$a'' = \text{lcm}(4, 6) = 12 \quad \begin{matrix} b'' \equiv 1 \pmod{4} \\ b'' \equiv 3 \pmod{6} \end{matrix} \rightarrow b'' = 9 \quad 12\mathbb{Z} + 9$

$$(a\mathbb{Z} + b) \sqcup^{\#} (a'\mathbb{Z} + b') = d\mathbb{Z} + r$$

where $d = \gcd(a, a', b - b')$ and $r \equiv b \pmod{d}$

e.g. $4\mathbb{Z} + 1 \sqcup^{\#} 4\mathbb{Z} + 3$

$d = \gcd(4, 4, 2), r = 1 \pmod{2} \quad 2\mathbb{Z} + 1$

Product domains

concrete domain = powerset of integers

abstract domain = intervals abstraction x congruence abstraction

Cartesian product

Given the (complete) lattices $(L_i, \sqsubseteq_i, \sqcup_i, \sqcap_i, \perp_i, \top_i)$ for $i = 1, 2$

We derive their **cartesian product** lattice $(L, \sqsubseteq, \sqcup, \sqcap, \perp, \top)$ by taking:

- $L \triangleq L_1 \times L_2$;
- $(a_1, a_2) \sqsubseteq (b_1, b_2)$ iff $a_1 \sqsubseteq_1 b_1$ and $a_2 \sqsubseteq_2 b_2$;
- $(a_1, a_2) \sqcup (b_1, b_2) \triangleq (a_1 \sqcup_1 b_1, a_2 \sqcup_2 b_2)$;
- $(a_1, a_2) \sqcap (b_1, b_2) \triangleq (a_1 \sqcap_1 b_1, a_2 \sqcap_2 b_2)$;
- $\perp \triangleq (\perp_1, \perp_2)$;
- $\top \triangleq (\top_1, \top_2)$.

$$(C, \leq) \xrightleftharpoons[\alpha_1]{\gamma_1} (A_1, \sqsubseteq_1) \quad (C, \leq) \xrightleftharpoons[\alpha_2]{\gamma_2} (A_2, \sqsubseteq_2)$$

the product of two GCs is a GC

$$(C, \leq) \xrightleftharpoons[\alpha]{\gamma} (A_1 \times A_2, \sqsubseteq)$$

$$\gamma(a_1, a_2) = \gamma_1(a_1) \wedge \gamma_2(a_2)$$

Reduced product domain

$$(C, \leq) \xrightleftharpoons[\alpha_1]{\gamma_1} (A_1, \sqsubseteq_1) \quad (C, \leq) \xrightleftharpoons[\alpha_2]{\gamma_2} (A_2, \sqsubseteq_2)$$

the reduced product of two GCs is a GC

$$(C, \leq) \xrightleftharpoons[\alpha]{\gamma} ((A_1 \times A_2)_{\equiv}, \sqsubseteq)$$

take the equivalence classes

$$(a_1, a_2) \equiv (b_1, b_2) \Leftrightarrow \gamma(a_1, a_2) = \gamma(b_1, b_2)$$

$$\gamma([a_1, a_2]_{\equiv}) = \gamma_1(a_1) \cap \gamma_2(a_2)$$

Point-wise lifting

Given the (complete) lattice $(L_1, \sqsubseteq_1, \sqcup_1, \sqcap_1, \perp_1, \top_1)$ and a set X

We derive its **pointwise lifting** lattice $(L, \sqsubseteq, \sqcup, \sqcap, \perp, \top)$ by taking:

- $L \triangleq X \rightarrow L_1$; $x, y \in X, l \in L_1 \setminus \{ \perp_{L_1} \}$ we can have $\{x \mapsto \perp_{L_1}, y \mapsto l\}$
- $f \sqsubseteq g$ iff $\forall x \in X : f(x) \sqsubseteq_1 g(x)$;
- $(f \sqcup g) \triangleq \lambda x. f(x) \sqcup_1 g(x)$;
- $(f \sqcap g) \triangleq \lambda x. f(x) \sqcap_1 g(x)$;
- $\perp \triangleq \lambda x. \perp_1$;
- $\top \triangleq \lambda x. \top_1$.

Given the (complete) lattice $(L_1, \sqsubseteq_1, \sqcup_1, \sqcap_1, \perp_1, \top_1)$ and a set X

We derive its **smashed pointwise lifting** lattice $(L, \sqsubseteq, \sqcup, \sqcap, \perp, \top)$ by taking:

- $L = X \dot{\rightarrow} L_1 \triangleq (X \rightarrow (L_1 \setminus \{ \perp_1 \})) \uplus \{ \lambda x. \perp_1 \}$;
- $f \sqsubseteq g$ iff $\forall x \in X : f(x) \sqsubseteq_1 g(x)$;
- $(f \sqcup g) \triangleq \lambda x. f(x) \sqcup_1 g(x)$;
- $(f \sqcap g) \triangleq \lambda x. f(x) \sqcap_1 g(x)$;
- $\perp \triangleq \lambda x. \perp_1$;
- $\top \triangleq \lambda x. \top_1$.

$x, y \in X$, we can only have $\{x \mapsto \perp_{L_1}, y \mapsto \perp_{L_1}\}$

Smashed point-wise lifting domain

$$(C, \leq) \xrightleftharpoons[\alpha_1]{\gamma_1} (A_1, \sqsubseteq_1)$$

$$(X \rightarrow C, \leq) \xrightleftharpoons[\alpha]{\gamma} (X \dot{\rightarrow} A_1, \sqsubseteq)$$

smashed

$$\gamma(f) = \lambda x. \gamma_1(f(x))$$

Abstract stores

let $\Sigma \triangleq \{ \sigma : X \rightarrow \mathbb{Z} \}$ be the set of stores

concrete domain = powerset of store $\wp(\Sigma)$

$$(\wp(\mathbb{Z}), \subseteq) \xrightleftharpoons[\alpha_1]{\gamma_1} (A_1, \sqsubseteq_1)$$

first: (smashed) point-wise abstraction (cartesian abstraction)
from $\wp(\Sigma)$ to $\{ \rho : X \rightarrow \wp(\mathbb{Z}) \}$ (looses relational info!)

second: lifting the abstraction A_1

from $\{ \rho : X \rightarrow \wp(\mathbb{Z}) \}$ to $\{ \rho^\# : X \rightarrow A_1 \}$

First: Cartesian abstraction

from $\wp(X \rightarrow \mathbb{Z})$ to $X \rightarrow \wp(\mathbb{Z})$ (loses all relational info!)

$$\gamma(\rho) = \{ \sigma : X \rightarrow \mathbb{Z} \mid \forall x \in X. \sigma(x) \in \rho(x) \}$$

e.g., suppose $\rho = [x \mapsto \{0,5\}, y \mapsto \{0,3\}]$

$$\gamma(\rho) = \{ [x \mapsto 0, y \mapsto 0], [x \mapsto 0, y \mapsto 3], [x \mapsto 5, y \mapsto 0], [x \mapsto 5, y \mapsto 3] \}$$

$$\alpha(\{[x \mapsto 0, y \mapsto 0], [x \mapsto 5, y \mapsto 3]\}) = \rho$$

Second: lifting abstraction

$$(\wp(\mathbb{Z}), \subseteq) \xrightleftharpoons[\alpha_1]{\gamma_1} (A_1, \sqsubseteq_1)$$

from $X \rightarrow \wp(\mathbb{Z})$ to $\{\rho^\# : X \rightarrow A_1\}$

$$\gamma(\rho^\#) = \{ \rho : X \rightarrow \wp(\mathbb{Z}) \mid \forall x \in X. \rho(x) = \gamma_1(\rho^\#(x)) \}$$

e.g., suppose $\rho^\# = [x \mapsto [0,5], y \mapsto [0,3]]$

$$\gamma(\rho^\#) = [x \mapsto \{0,1,2,\dots,5\}, y \mapsto \{0,1,2,3\}]$$

$$\alpha([x \mapsto \{0,5\}, y \mapsto \{0,3\}]) = \rho^\#$$

Exercises:

Classification of the 7-element structure

Exercise: Consider the covering relations: $a \lessdot c, a \lessdot e, b \lessdot d, b \lessdot f, c \lessdot g, d \lessdot g, e \lessdot g, f \lessdot g$. Classify the structure.

1. **Poset? Yes.** The transitive closure of these relations satisfies reflexivity, antisymmetry, and transitivity.
2. **CPO? No.** A Complete Partial Order must be "pointed," meaning it must have a least element ($\perp$). This structure has two distinct minimal elements (a and b) and no unique bottom.

3. **Lattice? No.** In a lattice, every pair of elements must have a unique least upper bound (lub) and greatest lower bound (glb). Here, the pair a, b has no lower bounds at all, so their glb does not exist.
 4. **Complete Lattice? No.** A complete lattice requires that *every* subset has a lub and a glb, which implies the existence of a bottom element ($\perp = \text{lub}(\emptyset)$).
-

Sound Abstractions in Intervals

Exercise: Which intervals offer a sound abstraction for the following sets?

1. **The finite set $0, 5, 9$:** The best abstraction is the smallest interval containing all elements, which is $[0, 9]$.
 2. **The set of all powers of 2 $1, 2, 4, \dots$:** The lower bound is 1 and it has no upper bound, so the best abstraction is $[1, \infty]$.
 3. **The set of all even numbers $\dots, -2, 0, 2, \dots$:** Since it includes both arbitrarily large positive and negative numbers, the best abstraction is $[-\infty, \infty]$ (also denoted as $\top$).
-

Complete Lattice: Lub vs Glb Existence

Exercise: Prove that in a poset $(X, \sqsubseteq)$, the property (1) "every subset has a lub" holds if and only if property (2) "every subset has a glb" holds.

1. **Assume (1) holds.** Let $B \subseteq X$ be an arbitrary subset. We wish to show $\sqcap B$ exists.
 2. Define $L = \{x \in X \mid \forall b \in B, x \sqsubseteq b\}$, the set of all lower bounds of B .
 3. By hypothesis (1), the subset L has a least upper bound $g = \sqcup L$.
 4. Since every $b \in B$ is an upper bound for L , and g is the *least* upper bound, then $g \sqsubseteq b$ for all $b \in B$. Thus, g is a lower bound of B .
 5. By definition of L , any other lower bound x of B belongs to L . Since g is the lub of L , $x \sqsubseteq g$. Thus, g is the greatest lower bound of B ($g = \sqcap B$).
 6. The converse (2) $\implies$ (1) follows by the Duality Principle.
-

Ex. 1: Extensivity of $\gamma \circ \alpha$

Exercise: Prove $c \leq \gamma(\alpha(c))$ for all $c \in C$ given the Galois Connection (GC) property $c \leq \gamma(a) \iff \alpha(c) \sqsubseteq a$.

1. In the property $c \leq \gamma(a) \iff \alpha(c) \sqsubseteq a$, let $a = \alpha(c)$.
 2. The right side becomes $\alpha(c) \sqsubseteq \alpha(c)$, which is true by reflexivity of the poset $(A, \sqsubseteq)$.
 3. Because the equivalence holds, the left side $c \leq \gamma(\alpha(c))$ must also be true.
-

Ex. 2: Reductivity of $\alpha \circ \gamma$

Exercise: Prove $\alpha(\gamma(a)) \sqsubseteq a$ for all $a \in A$ given the GC property.

1. In the property $c \leq \gamma(a) \iff \alpha(c) \sqsubseteq a$, let $c = \gamma(a)$.
 2. The left side becomes $\gamma(a) \leq \gamma(a)$, which is true by reflexivity of the poset $(C, \leq)$.
 3. Because the equivalence holds, the right side $\alpha(\gamma(a)) \sqsubseteq a$ must also be true.
-

Ex. 3: Monotonicity of α

Exercise: Prove that if $c_1 \leq c_2$, then $\alpha(c_1) \sqsubseteq \alpha(c_2)$.

1. From Ex. 1, we know $c_2 \leq \gamma(\alpha(c_2))$.
 2. If $c_1 \leq c_2$, then by transitivity of $\leq$, $c_1 \leq \gamma(\alpha(c_2))$.
 3. By the GC property $c \leq \gamma(a) \iff \alpha(c) \sqsubseteq a$, where $a = \alpha(c_2)$, we conclude $\alpha(c_1) \sqsubseteq \alpha(c_2)$.
-

Ex. 4: Monotonicity of γ

Exercise: Prove that if $a_1 \sqsubseteq a_2$, then $\gamma(a_1) \leq \gamma(a_2)$.

1. From Ex. 2, we know $\alpha(\gamma(a_1)) \sqsubseteq a_1$.
 2. If $a_1 \sqsubseteq a_2$, then by transitivity of $\sqsubseteq$, $\alpha(\gamma(a_1)) \sqsubseteq a_2$.
 3. By the GC property $c \leq \gamma(a) \iff \alpha(c) \sqsubseteq a$, where $c = \gamma(a_1)$, we conclude $\gamma(a_1) \leq \gamma(a_2)$.
-

Ex. 5: Equivalent Definition

Exercise: Prove that a pair of maps (α, γ) satisfies the GC property iff α and γ are monotone, $\gamma \circ \alpha$ is extensive, and $\alpha \circ \gamma$ is reductive.

1. ($\implies$) proven in Ex. 1, 2, 3, and 4.
 2. ($\impliedby$) Assume monotonicity, extensivity, and reductivity.
 3. If $\alpha(c) \sqsubseteq a$, then by monotonicity of γ , $\gamma(\alpha(c)) \leq \gamma(a)$. By extensivity, $c \leq \gamma(\alpha(c))$, so by transitivity $c \leq \gamma(a)$.
 4. If $c \leq \gamma(a)$, then by monotonicity of α , $\alpha(c) \sqsubseteq \alpha(\gamma(a))$. By reductivity, $\alpha(\gamma(a)) \sqsubseteq a$, so by transitivity $\alpha(c) \sqsubseteq a$.
-

Ex. 6: Round-trips

Exercise: Prove $\gamma \circ \alpha \circ \gamma = \gamma$ and $\alpha \circ \gamma \circ \alpha = \alpha$.

1. **For γ :** By extensivity (Ex. 1), $\gamma(a) \leq \gamma(\alpha(\gamma(a)))$. By reductivity (Ex. 2) and monotonicity of γ , $\gamma(\alpha(\gamma(a))) \leq \gamma(a)$. By antisymmetry, $\gamma(\alpha(\gamma(a))) = \gamma(a)$.

2. **For α :** By reductivity, $\alpha(\gamma(\alpha(c))) \sqsubseteq \alpha(c)$. By extensivity and monotonicity of α , $\alpha(c) \sqsubseteq \alpha(\gamma(\alpha(c)))$. By antisymmetry, $\alpha(\gamma(\alpha(c))) = \alpha(c)$.
-

Ex. 7: Idempotency

Exercise: Prove that $\gamma \circ \alpha$ and $\alpha \circ \gamma$ are idempotent.

1. For $\gamma \circ \alpha$: $(\gamma \circ \alpha) \circ (\gamma \circ \alpha) = \gamma \circ (\alpha \circ \gamma \circ \alpha) = \gamma \circ \alpha$ (using Ex. 6).
 2. For $\alpha \circ \gamma$: $(\alpha \circ \gamma) \circ (\alpha \circ \gamma) = \alpha \circ (\gamma \circ \alpha \circ \gamma) = \alpha \circ \gamma$ (using Ex. 6).
-

Ex. 8: γ induces α

Exercise: Prove $\alpha(c) = \sqcap a \mid c \leq \gamma(a)$.

1. By the GC property, $c \leq \gamma(a) \iff \alpha(c) \sqsubseteq a$.
 2. The set $a \mid c \leq \gamma(a)$ is therefore equal to the set $a \mid \alpha(c) \sqsubseteq a$, which is the set of all upper bounds of $\alpha(c)$.
 3. The greatest lower bound (glb) of the set of all upper bounds of an element is the element itself. Thus, $\sqcap a \mid \alpha(c) \sqsubseteq a = \alpha(c)$.
-

Ex. 9: α induces γ

Exercise: Prove $\gamma(a) = \sqcup c \mid \alpha(c) \sqsubseteq a$.

1. By the GC property, $\alpha(c) \sqsubseteq a \iff c \leq \gamma(a)$.
 2. The set $c \mid \alpha(c) \sqsubseteq a$ is therefore equal to the set $c \mid c \leq \gamma(a)$, which is the set of all lower bounds of $\gamma(a)$.
 3. The least upper bound (lub) of the set of all lower bounds of an element is the element itself. Thus, $\sqcup c \mid c \leq \gamma(a) = \gamma(a)$.
-

Ex. 10: α preserves lub

Exercise: Prove $\alpha(\sqcup X) = \sqcup \alpha(x) \mid x \in X$ whenever the concrete lub $\sqcup X$ exists.

1. By monotonicity (Ex. 3), for any $x \in X$, $x \leq \sqcup X \implies \alpha(x) \sqsubseteq \alpha(\sqcup X)$. Thus, $\alpha(\sqcup X)$ is an upper bound of $\alpha(x) \mid x \in X$.
2. Let a be any other upper bound such that $\forall x \in X, \alpha(x) \sqsubseteq a$.
3. By the GC property, this is equivalent to $\forall x \in X, x \leq \gamma(a)$.
4. This means $\gamma(a)$ is an upper bound of X . Since $\sqcup X$ is the **least** upper bound, $\sqcup X \leq \gamma(a)$.
5. Applying the GC property again, $\alpha(\sqcup X) \sqsubseteq a$. Thus, $\alpha(\sqcup X)$ is the least upper bound.

Ex. 11: γ preserves glbs

Exercise: Prove $\gamma(\sqcap Y) = \bigwedge \gamma(y) \mid y \in Y$ whenever the abstract glb $\sqcap Y$ exists.

1. By monotonicity (Ex. 4), for any $y \in Y$, $\sqcap Y \sqsubseteq y \implies \gamma(\sqcap Y) \leq \gamma(y)$. Thus, $\gamma(\sqcap Y)$ is a lower bound of $\gamma(y) \mid y \in Y$.
 2. Let c be any other lower bound such that $\forall y \in Y, c \leq \gamma(y)$.
 3. By the GC property, this is equivalent to $\forall y \in Y, \alpha(c) \sqsubseteq y$.
 4. This means $\alpha(c)$ is a lower bound of Y . Since $\sqcap Y$ is the **greatest** lower bound, $\alpha(c) \sqsubseteq \sqcap Y$.
 5. Applying the GC property again, $c \leq \gamma(\sqcap Y)$. Thus, $\gamma(\sqcap Y)$ is the greatest lower bound.
-

Ex. 12: Surjectivity/Injectivity and Galois Insertion

Exercise: Prove that in a Galois Connection, α is surjective iff γ is injective iff $\alpha \circ \gamma = id_A$.

1. $\alpha \circ \gamma = id_A \implies \gamma$ is injective: If $\gamma(a_1) = \gamma(a_2)$, then $\alpha(\gamma(a_1)) = \alpha(\gamma(a_2))$, which means $id_A(a_1) = id_A(a_2) \implies a_1 = a_2$.
 2. $\alpha \circ \gamma = id_A \implies \alpha$ is surjective: For any $a \in A$, let $c = \gamma(a)$. Then $\alpha(c) = \alpha(\gamma(a)) = a$.
 3. γ is injective $\implies \alpha \circ \gamma = id_A$: From Ex. 6, $\gamma(\alpha(\gamma(a))) = \gamma(a)$. If γ is injective, then $\alpha(\gamma(a)) = a$.
 4. α is surjective $\implies \alpha \circ \gamma = id_A$: For any $a \in A$, there exists c such that $\alpha(c) = a$. Then $\alpha(\gamma(a)) = \alpha(\gamma(\alpha(c))) = \alpha(c) = a$ (using Ex. 6).
-

Summary of key properties used

- **Definition 2.1:** Poset (reflexive, antisymmetric, transitive).
- **Definition 2.3:** CPO (pointed poset where chains have lubs).
- **Definition 2.4/2.5:** (Complete) Lattice (existence of lubs and glbs).
- **Definition 2.12:** Galois Connection ($c \leq \gamma(a) \iff \alpha(c) \sqsubseteq a$).
- **Definition 2.13:** Galois Embedding/Insertion (α is surjective, γ is injective, $\alpha \circ \gamma = id$).
- **Theorem 2.3:** Alternate characterization of GC (monotonicity, extensivity, reductivity).
- **Theorem 2.4:** Properties of GC (round-trips, idempotency, induction of adjoints, preservation of limits).

Does this comprehensive walkthrough of the Galois insertion exercises clarify the relationships between the concrete and abstract worlds, or would you like to see a practical application in [Sign Analysis](#)?

Abstract Analysis

Abstract interpreter

$$\llbracket e \rrbracket \triangleq \dots$$

$$\llbracket e \rrbracket^\# \triangleq \llbracket e \rrbracket^A \text{ (when possible)}$$

$$\llbracket c_1; c_2 \rrbracket \triangleq \llbracket c_2 \rrbracket \circ \llbracket c_1 \rrbracket$$

$$\llbracket c_1; c_2 \rrbracket^\# \triangleq \llbracket c_2 \rrbracket^\# \circ \llbracket c_1 \rrbracket^\#$$

$$\llbracket c_1 + c_2 \rrbracket \triangleq \llbracket c_1 \rrbracket \cup \llbracket c_2 \rrbracket$$

$$\llbracket c_1 + c_2 \rrbracket^\# \triangleq \llbracket c_1 \rrbracket^\# \sqcup \llbracket c_2 \rrbracket^\#$$

$$\llbracket c^\star \rrbracket \triangleq \bigcup_{k=0}^{\infty} \llbracket c \rrbracket^k$$

$$\llbracket c^\star \rrbracket^\# \triangleq \bigsqcup_{k=0}^{\infty} (\llbracket c \rrbracket^\#)^k$$

or equivalently

$$\llbracket c^\star \rrbracket \triangleq \lambda S. \text{ lfp } \lambda X. S \cup \llbracket c \rrbracket X$$

or, not equivalently,

$$\llbracket c^\star \rrbracket^\# \triangleq \lambda S^\#. \text{ lfp } \lambda X^\#. S^\# \sqcup \llbracket c \rrbracket^\# X^\#$$

or using widening (precision loss)

$$\llbracket c^\star \rrbracket^\# \triangleq \lambda S^\#. \text{ lfp } \lambda X^\#. S^\# \nabla \llbracket c \rrbracket^\# X^\#$$

$F^\#$ is correct on an abstract domain A whenever it returns a correct approximation of the result of the concrete computation

$$F^\# \sqsupseteq \alpha \circ F \circ \gamma = F^A$$

$F^\#$ is complete on an abstract domain A whenever it returns the best abstraction of the result of the concrete computation

$$F^\# \circ \alpha = \alpha \circ F$$

$$F^\# \text{ is complete} \implies F^\# = F^A$$

$$\alpha \circ F = F^\# \circ \alpha \implies F^A = \alpha F \gamma = F^\# \circ \alpha \circ \gamma = F^\#$$

if F monotone and $F^\#$ correct then $\text{lfp}(F^\#)$ is a correct over-approx of $\text{lfp}(F)$

Concrete Equivalence $\approx$:

$$c_1 \approx c_2$$



They compute the same function

$$\forall S \in \wp(\Sigma). \llbracket c_1 \rrbracket S = \llbracket c_2 \rrbracket S$$

Abstract Equivalence $\sim$:

$$c_1 \sim c_2$$



They compute the same abstract function

$$\forall a : X \rightarrow A. \llbracket c_1 \rrbracket^\# a = \llbracket c_2 \rrbracket^\# a$$

If F monotone and F^A is complete then $\alpha(\text{lfp}(F)) = \text{lfp}(F^A)$

Three staged approach: 1

Selection of the semantics and properties of interest

define a concrete domain $(C, \subseteq)$

define program behaviour $\llbracket \cdot \rrbracket$

fix the goal of the analysis

describe properties to be verified

Three staged approach: 2

Choice of the abstraction

fix the abstract domain $(A, \sqsubseteq)$

must express all properties of interest

(must express useful invariants)

define abstract interpreter $\llbracket \cdot \rrbracket^\#$

We need the ability to

- compare abstract elements for inclusion
- compute (possibly best) abstract representations
- compute the union/widening of abstract elements
- apply sound transformations to abstract elements

$$(A, \sqsubseteq) \ a \sqsubseteq b \implies \gamma(a) \subseteq \gamma(b)$$

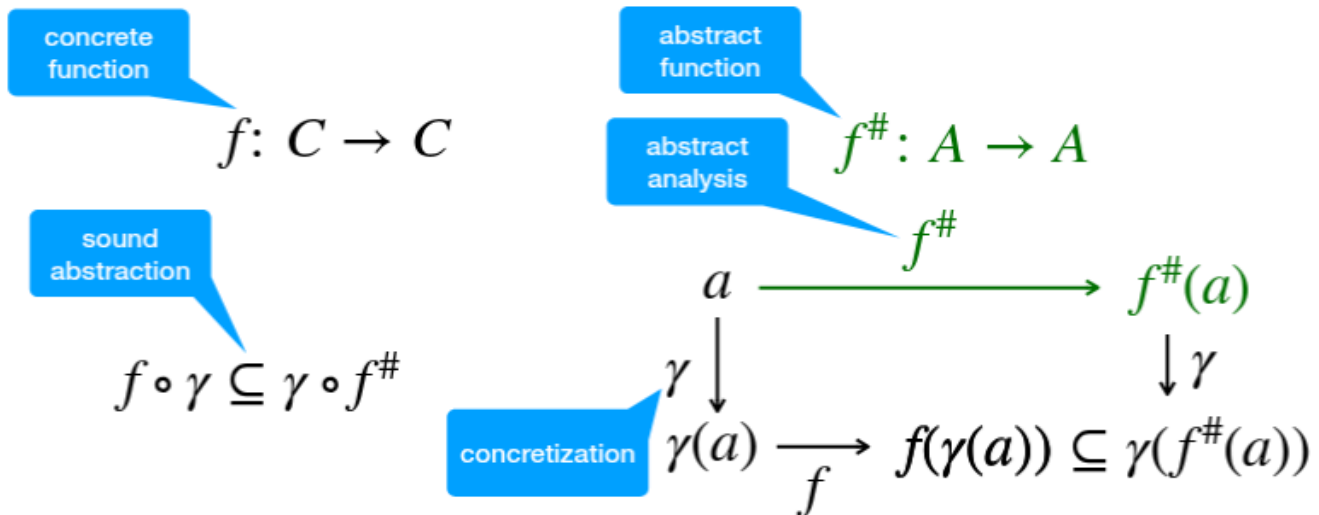
$$c \subseteq \gamma(a) \implies \gamma(\alpha(c)) \subseteq \gamma(a)$$

$$a_1, a_2 \subseteq a_1 \sqcup a_2 \text{ (union)}$$

$$a_1 \sqcup a_2 \sqsubseteq a_1 \nabla a_2 \text{ (widening)}$$

Abstract domain

We need the ability to
apply sound transformations to abstract elements



Three staged approach: 3

Derivation of the analysis algorithm

follow the choices made in first two phases
the analysis closely follows the semantics

Exercises:

Classification Exercise (Slides 294–296)

Problem: Classify whether each abstract property belongs to a **relational** or **non-relational** abstract domain and justify why.

- **(a) Signs: Non-relational.** This domain abstracts the set of possible values for each variable independently from others.
- **(b) Parity: Non-relational.** Like signs, parity tracks a property (even or odd) of a single variable's value without considering its relationship to other variables.
- **(c) Intervals around 0 ($[-l, r]$): Non-relational.** This is a variation of the interval domain that constrains the range of a single variable.
- **(d) Equalities ($c_1 \cdot x = c_2 \cdot y$): Relational.** These properties capture numerical constraints and dependencies between two or more variables.

- **(e) Inequalities ($c_1 \cdot x \neq c_2 \cdot y$): Relational.** Similar to equalities, these define a specific logical relationship between variables x and y .
-

Completeness 1 (Slide 312)

Problem: Given $P \triangleq -7,5$ and $c \triangleq (x < 0)?; x := -x$, compute the abstract semantics $[[c]]^\#$ on the **Sign** domain for $\alpha_{\text{Sign}}(P)$ and compare it with $\alpha_{\text{Sign}}([c]P)$.

1. Abstraction of the input:

- $\alpha_{\text{Sign}}(-7,5) = \mathbb{Z}$. Since the set P contains both a negative and a positive number and the Sign domain does not have a "non-zero" element, the most precise abstraction is **Top** ($\mathbb{Z}$).

2. Abstract Semantics $[[c]]^\# \mathbb{Z}$:

- Apply the guard $(x < 0)?^\#$: In the Sign domain, filtering Top with "negative" results in $\mathbb{Z}_{<0}$.
- Apply the assignment $x := -x^\#$: Negating a negative sign results in a positive sign. Thus, $-\#(\mathbb{Z}_{<0}) = \mathbb{Z}_{>0}$.
- **Result:** $[[c]]^\# \alpha_{\text{Sign}}(P) = \mathbb{Z}_{>0}$.

3. Concrete Semantics and its Abstraction:

- $[[c]]-7,5$: The guard $(x < 0)?$ filters the set to -7 . The assignment $x := -x$ transforms it to 7 .
- $\alpha_{\text{Sign}}(7) = \mathbb{Z}_{>0}$.

Final Answer: The abstract transformer is **complete** for this example because $[[c]]^\# \alpha(P) = \alpha([c]P) = \mathbb{Z}_{>0}$.

Completeness 2 (Slide 313)

Problem: Determine if the Best Correct Approximation (BCA) of $f(x) = x$ if $x \leq 10$ (and a different branch for $x > 10$) is complete on the **Interval** domain.

Solution: A function f is complete on the Interval domain if its abstract counterpart matches the abstraction of the concrete result: $\alpha(f(S)) = f^A(\alpha(S))$.

- If the "otherwise" branch makes f **non-monotone** (e.g., $f(11) = 0$), completeness fails.
- **Counterexample:** Let $S = 10,11$. Then $\alpha(S) = .$
- $\alpha(f(S)) = \alpha(10,0) = .$
- $f^A(\alpha(S)) = \alpha(f()) = \alpha(10,0) = .$ (In this specific case it might seem complete).
- However, generally, piecewise functions that "break" an interval into non-contiguous sets (like the absolute value $|x|$) are **not complete** on intervals because the interval hull α introduces spurious values that were not in the concrete image.

Final Answer: Generally, the BCA of such a piecewise function is **not complete** because the abstraction of the image of an interval often includes values that the concrete function would never reach.

Completeness 3 (Slide 313)

Problem: Given the domain $A = \top, \perp, < 0, \geq 0$: (a) define γ and α , and (b) determine if it admits a complete abstract multiplication.

(a) Formal Definitions:

- **Concretization (γ):**
 - $\gamma(\perp) = \emptyset$
 - $\gamma(< 0) = \{n \in \mathbb{Z} \mid n < 0\}$
 - $\gamma(\geq 0) = \{n \in \mathbb{Z} \mid n \geq 0\}$
 - $\gamma(\top) = \mathbb{Z}$
- **Abstraction (α):**
 - $\alpha(S) = \perp$ if $S = \emptyset$
 - $\alpha(S) = (< 0)$ if $S \neq \emptyset \wedge S \subseteq \{n \mid n < 0\}$
 - $\alpha(S) = (\geq 0)$ if $S \neq \emptyset \wedge S \subseteq \{n \mid n \geq 0\}$
 - $\alpha(S) = \top$ otherwise.

(b) Multiplication Completeness: To be complete,

$\alpha(n \times m) = \alpha(n) \times^{\#} \alpha(m)$ must hold.

- **Counterexample:** Let $n = -1$ and $m = 0$.
- **Concrete:** $n \times m = 0$. $\alpha(0) = (\geq 0)$.
- **Abstract:** $\alpha(-1) = (< 0)$ and $\alpha(0) = (\geq 0)$.
- $\alpha((< 0) \times^{\#} (\geq 0)) = \alpha(\{x \cdot y \mid x < 0, y \geq 0\})$.
- The set of products is $\{n \in \mathbb{Z} \mid n \leq 0\}$.
- In this domain, the set ≤ 0 contains both negative and non-negative (zero) values. Since there is no "non-positive" element, it must be abstracted to $\top$.
- **Result:** $\alpha(0) \neq \top$.

Final Answer: The domain **does not admit a complete multiplication operator** because it lacks the precision to represent the "non-positive" result ≤ 0 specifically.

Does this clarify the relationship between domain expressiveness and completeness, or should we look at the **Reduced Product** construction next?

Local Completeness Logic

$$A(.) = \gamma(\alpha(.))$$

$$\forall P . A(\llbracket c \rrbracket P) = \llbracket c \rrbracket^{\#} A(P)$$

$$\subseteq \quad \subseteq$$

$$A(\llbracket c \rrbracket A(P))$$

$\llbracket \cdot \rrbracket$ is the semantics

$\llbracket \cdot \rrbracket^A$ is the bca

$\llbracket \cdot \rrbracket^{\#}$ is the abstract semantics

$\llbracket \cdot \rrbracket_A^{\#}$ is also the abstract semantics

Completeness equation: $\forall P . A(\llbracket c \rrbracket P) = A(\llbracket c \rrbracket A(P))$

If the bca $\llbracket c \rrbracket^A$ is not complete, then $\llbracket c \rrbracket^{\#}$ is not complete

Assume $A(Spec) = Spec$

Take a post $Q \in C$

If $Q \subseteq Spec$

then $A(Q) \subseteq A(Spec) = Spec$

If $Q \not\subseteq Spec$

then $Q \subseteq A(Q) \not\subseteq Spec$

$$Q \subseteq Spec \Leftrightarrow A(Q) \subseteq Spec$$

Q

$Spec$

Q



The role of completeness

correctness of AI
if $\llbracket c \rrbracket_A^{\#} A(P) \subseteq Spec$
then $\llbracket c \rrbracket P \subseteq Spec$

$$\llbracket c \rrbracket P \subseteq A(\llbracket c \rrbracket P) \subseteq \llbracket c \rrbracket_A^{\#} A(P) \subseteq Spec$$

expressible specification
 $Spec = A(Spec)$

if completeness holds

if $\llbracket c \rrbracket_A^{\#} A(P) \not\subseteq Spec$
then $\llbracket c \rrbracket P \not\subseteq Spec$

$$A(\llbracket c \rrbracket P) = \llbracket c \rrbracket_A^{\#} A(P) \not\subseteq Spec$$

$$\Leftrightarrow \llbracket c \rrbracket P \not\subseteq Spec$$



$$\llbracket c \rrbracket P \subseteq Spec \Leftrightarrow \llbracket c \rrbracket_A^{\#} A(P) \subseteq Spec$$

Completeness equation: $\forall P. A(\llbracket e \rrbracket P) = A(\llbracket e \rrbracket A(P))$

if a test b is complete in A , then b and $\neg b$ are expressible in A

Proof:

Dimostrazione (per contrapposizione)

1. **Assunzione:** Supponiamo che b non sia esprimibile in A . Poiché per ogni proprietà $p \subseteq A(p)$, se b non è esprimibile deve essere $b \subset A(b)$ 2.
2. **Implicazione:** Questo significa che l'approssimazione astratta di b "esce" dai confini di b , quindi l'intersezione tra la sua astrazione e la sua negazione non è vuota: $A(b) \cap \neg b \neq \emptyset$ 2.
3. **Verifica della completezza:** Verifichiamo se l'equazione di completezza $A(p \cap \neg b) = A(A(p) \cap \neg b)$ tiene per il test $\neg b$? usando come input $p = b$:
 - **Parte destra (RHS):** $A(A(b) \cap \neg b)$. Poiché abbiamo stabilito al punto 2 che questa intersezione non è vuota, e dato che il dominio è stretto, il risultato deve essere $\neq \emptyset$ 2.
 - **Parte sinistra (LHS):** $A(b \cap \neg b) = A(\emptyset) = \emptyset$ 2.
4. **Conclusione:** Poiché $\emptyset \neq \text{RHS}$, l'equazione di completezza non è soddisfatta. Pertanto, se b non è esprimibile, il test $\neg b$? non può essere completo. Lo stesso ragionamento vale simmetricamente per $\neg b$ 2.

Let b and $\neg b$ be expressible in A .

The test b is complete in $A \iff$ the join of any two abstract points below b and $\neg b$ is expressible.

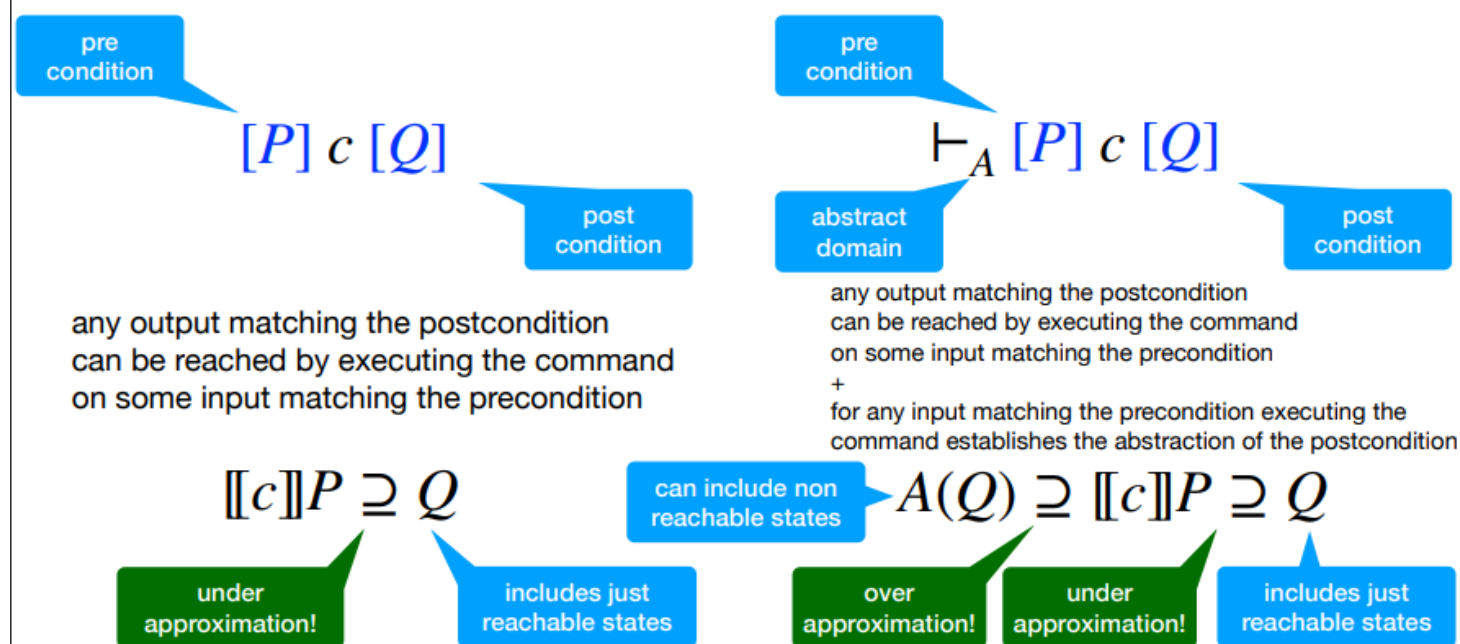
Local completeness equation: $A(\llbracket c \rrbracket P) = A(\llbracket c \rrbracket A(P))$

We say that e is locally complete in A for input P and write

$$\mathbb{C}_P^A(e) = A(\llbracket c \rrbracket P) = \llbracket c \rrbracket^\# A(P)$$

O'Hearn's triples

LCL triples



Convexity

Lemma. [convexity]

If $\mathbb{C}_P^A(e)$ and $P \Rightarrow R \Rightarrow A(P)$ then $\mathbb{C}_R^A(e)$

Proof.

Assume $A(\llbracket e \rrbracket P) = A(\llbracket e \rrbracket A(P))$

we want to prove $A(\llbracket e \rrbracket R) = A(\llbracket e \rrbracket A(R))$

$$A(\llbracket e \rrbracket P) \leq A(\llbracket e \rrbracket R) \leq A(\llbracket e \rrbracket A(R)) = A(\llbracket e \rrbracket A(P)) = A(\llbracket e \rrbracket P)$$

The rules of LCL

$$\begin{array}{c} \frac{\mathbb{C}_P^A(e)}{\vdash_A [P] e \llbracket e \rrbracket P} \text{ (transfer)} \quad \frac{P' \leq P \leq A(P') \quad \vdash_A [P'] r [Q'] \quad Q \leq Q' \leq A(Q)}{\vdash_A [P] r [Q]} \text{ (relax)} \\ \\ \frac{\vdash_A [P] r_1 [R] \quad \vdash_A [R] r_2 [Q]}{\vdash_A [P] r_1; r_2 [Q]} \text{ (seq)} \quad \frac{\vdash_A [P] r_1 [Q_1] \quad \vdash_A [P] r_2 [Q_2]}{\vdash_A [P] r_1 \oplus r_2 [Q_1 \vee Q_2]} \text{ (join)} \\ \\ \frac{\vdash_A [P] r [R] \quad \vdash_A [P \vee R] r^* [Q]}{\vdash_A [P] r^* [Q]} \text{ (rec)} \quad \frac{\vdash_A [P] r [Q] \quad Q \leq A(P)}{\vdash_A [P] r^* [P \vee Q]} \text{ (iterate)} \\ \\ \text{Auxiliary rules} \\ \frac{\vdash_A [P] r [Q] \quad Q \leq P}{\vdash_A [P] r^* [P]} \text{ (invariant)} \quad \frac{\forall n \in \mathbb{N}. \vdash_A [P_n] r [P_{n+1}]}{\vdash_A [P_0] r^* [\bigvee_{i \in \mathbb{N}} P_i]} \text{ (limit)} \\ \\ \frac{\vdash_A [P] r [Q] \quad A(P) = A(Q)}{\vdash_A [P] r^* [Q]} \text{ (abs-fix)} \end{array}$$

A LCL triple $\vdash_A [P]c[Q]$ is valid if $Q \subseteq \llbracket c \rrbracket P \subseteq A(Q)$

if $\vdash_A [P]c[Q]$ then $Q \subseteq \llbracket c \rrbracket P \subseteq A(Q) = \llbracket c \rrbracket_A^\# A(P)$

Proof:

By induction on the derivation

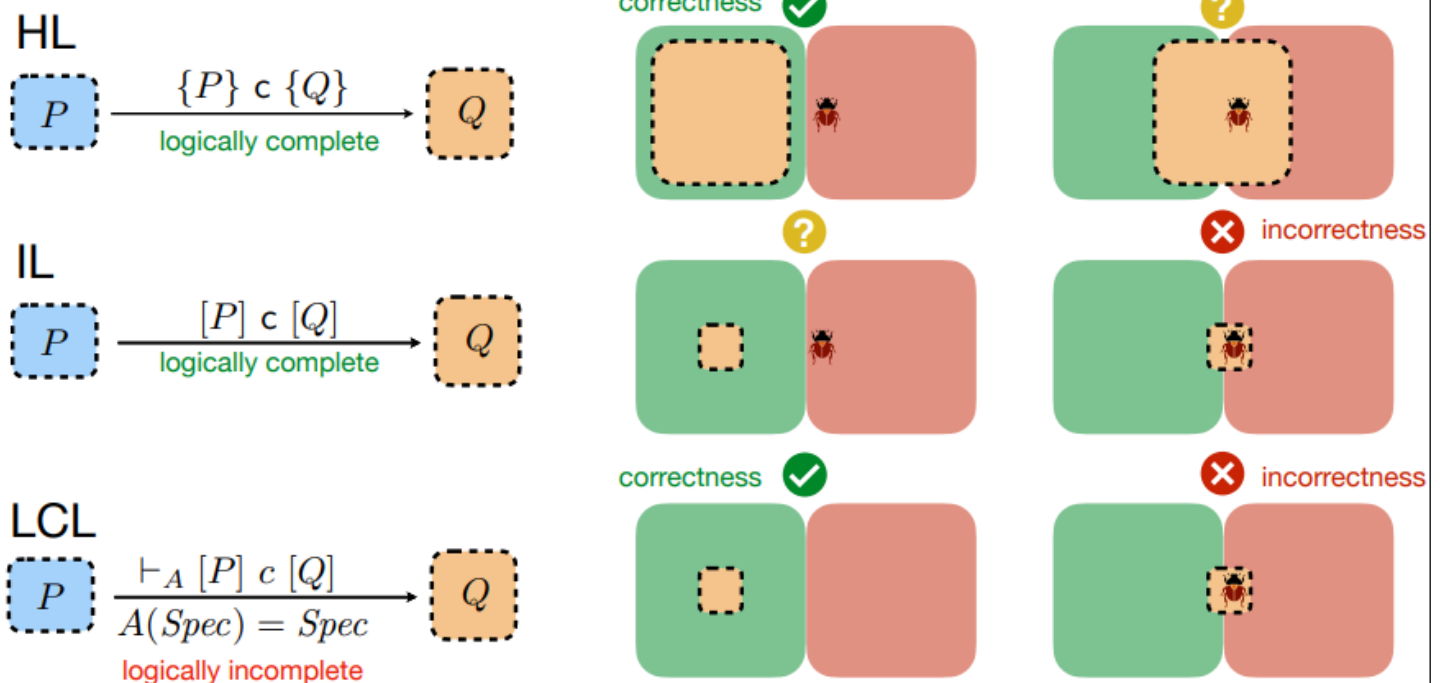
if A is complete for any atomic command in c , then any valid triple $\vdash_A [P]c[Q]$ can be derived

Proof:

1. derive $\vdash_A [P]c[\llbracket c \rrbracket P]$
2. use [relax] with $Q \subseteq \llbracket c \rrbracket P$

For any Turing complete language and any non-trivial abstraction A , there are valid triples that cannot be proved

Over vs Under



Soundness

Are these “mixed” HL+LCL inference rules valid ?

$$\frac{\vdash_A [P] c [Q]}{\{P\} c \{A(Q)\}} \qquad \frac{\vdash_A [P] c [Q]}{\{A(P)\} c \{A(Q)\}}$$

If $\vdash_A [P] c [Q]$ then $\llbracket c \rrbracket P \subseteq A(Q)$, hence $\{P\} c \{A(Q)\}$ is valid

If $\vdash_A [P] c [Q]$ then $\llbracket c \rrbracket A(P) \subseteq \llbracket c \rrbracket^\# A(P) = A(Q)$, hence $\{A(P)\} c \{A(Q)\}$ is valid

HL

$$\{P\} c \{Q\} \iff \llbracket c \rrbracket P \subseteq Q$$

- P = precondition
- Q = postcondition

Whenever c is executed in a state satisfying P if the execution of c terminates, it terminates in a state satisfying Q

We talk about partial correctness because the previous semantic condition does not guarantee that the execution of c on states satisfying P terminate

Q is an over-approximation of the states that can be reached by states in P

[Floyd] forward oriented

[Hoare] backward oriented

Soundness:

Th. Any derivable HL triple is valid

Proof. By induction on the derivation tree

The variant expresses a general condition that is true for every execution, but is still strong enough to guarantee the desired postcondition when the loop terminates

[cons] forward/backward strengthening P and weakening Q

Total correctness = partial correctness + termination

$$wlp(c, Q) \triangleq \{\sigma \in \Sigma \mid [[c]]\{\sigma\} \subseteq Q\}$$

$$P \implies wlp(c, Q) \iff [[c]]P \subseteq Q$$

$$\text{if } b \text{ then } c_1 \text{ else } c_2 \triangleq (b?; c_1) + (\neg b?; c_2)$$

$$\text{while } b \text{ do } c \triangleq (b?; c)^*; \neg b?$$

For correctness reasoning

You **get to forget** information as you go along a path, but you **must remember** all the paths.

$$\{P\} \ c \ \{Q\}$$

$$\text{validity: } \llbracket c \rrbracket P \subseteq Q$$

$$\forall \sigma \in P. \forall \delta \in \llbracket c \rrbracket \sigma. \delta \in Q$$

can prove the absence of bugs
(any execution of c from P is correct)

Exercises:

1

La dimostrazione della correttezza (soundness) della regola di congiunzione *conj* in Hoare Logic si basa sulla **monotonia** della semantica collettiva $\llbracket c \rrbracket$.

$$\frac{P_1 c Q_1 \quad P_2 c Q_2}{P_1 \wedge P_2 c Q_1 \wedge Q_2}$$

Dimostrazione

1. **Assunzione:** Ipotizziamo che le premesse siano valide. Per definizione di validità in HL, ciò significa che:
 - $\llbracket c \rrbracket P_1 \subseteq Q_1$
 - $\llbracket c \rrbracket P_2 \subseteq Q_2$
2. **Proprietà di Monotonia:** La semantica di un comando $\llbracket c \rrbracket$ è una funzione monotona (se $A \subseteq B \implies \llbracket c \rrbracket A \subseteq \llbracket c \rrbracket B$).
3. **Applicazione alla congiunzione:** Consideriamo l'input $P_1 \cap P_2$ (che corrisponde logicamente a $P_1 \wedge P_2$):
 - Poiché $(P_1 \cap P_2) \subseteq P_1$, per monotonia abbiamo $\llbracket c \rrbracket (P_1 \cap P_2) \subseteq \llbracket c \rrbracket P_1 \subseteq Q_1$.
 - Poiché $(P_1 \cap P_2) \subseteq P_2$, per monotonia abbiamo $\llbracket c \rrbracket (P_1 \cap P_2) \subseteq \llbracket c \rrbracket P_2 \subseteq Q_2$.
4. **Conclusione:** Dato che il risultato $\llbracket c \rrbracket (P_1 \cap P_2)$ è contenuto sia in Q_1 che in Q_2 , deve essere contenuto nella loro intersezione:

$$\llbracket c \rrbracket (P_1 \cap P_2) \subseteq Q_1 \cap Q_2$$

Questo dimostra che la tripla $P_1 \wedge P_2 c Q_1 \wedge Q_2$ è semanticamente valida, rendendo la regola **sound**.

2

Show that the following rule for assignment is not sound

$$\frac{}{\{P\}x := a\{P[a/x]\}}$$

Questa regola non è corretta (**unsound**) perché tenta di applicare la sostituzione sintattica nel "verso sbagliato" rispetto alla semantica dell'assegnamento.

Ecco un **controesempio** per dimostrarne l'invalidità:

1. **Istanza:** Consideriamo $P \triangleq (y = x)$ e il comando $x := 0$.
2. **Applicazione della regola:** La regola produrrebbe la tripla $\{y = x\}x := 0\{y = 0\}$, poiché sostituendo 0 al posto di x in P si ottiene $y = 0$.
3. **Verifica semantica:**
 - Partiamo da uno stato iniziale $\sigma = [x \mapsto 1, y \mapsto 1]$. Questo stato soddisfa la pre-condizione ($1 = 1$).
 - Eseguiamo il comando $x := 0$. Il nuovo stato sarà $\delta = [x \mapsto 0, y \mapsto 1]$.
 - Verifichiamo la post-condizione nello stato finale: $y = 1$, quindi l'asserzione $y = 0$ è **falsa**.
4. **Conclusione:** Poiché esiste un'esecuzione che termina in uno stato che non soddisfa la post-condizione, la tripla non è valida e la regola è **unsound**.

Le regole corrette (sound) sono quelle di **Hoare** (orientata all'indietro: $Q[a/x]x := aQ$) o di **Floyd** (orientata in avanti con quantificatori).

IL

$$[P]c[Q] \iff [[c]]P \supseteq Q$$

stronger $P \subseteq Q$ weaker

stronger $P \Rightarrow Q$ weaker

stronger $P \wedge Q \Rightarrow P \vee Q$ weaker

Hoare's triples

O'Hearn's triples

pre
condition

$$\{P\} c \{Q\}$$

post
condition

for any input matching the precondition
executing the command establishes the postcondition

$$\llbracket c \rrbracket P \subseteq Q$$

over
approximation!

can include non
reachable states

pre
condition

$$[P] c [Q]$$

post
condition

any output matching the postcondition
can be reached by executing the command
on some input matching the precondition

$$\llbracket c \rrbracket P \supseteq Q$$

under
approximation!

includes just
reachable states

$$\{P\} c \{Q\}$$

$$\llbracket c \rrbracket P \subseteq Q \quad \equiv \quad \forall \sigma \in P. \forall \delta \in \llbracket c \rrbracket \sigma. \delta \in Q$$

Any reachable output satisfies the postcondition

$$[P] c [Q]$$

$$\llbracket c \rrbracket P \supseteq Q \quad \equiv \quad \forall \delta \in Q. \exists \sigma \in P. \delta \in \llbracket c \rrbracket \sigma$$

Any output in the postcondition is reachable

Some dualities

$$\begin{array}{c} \text{[disj]} \frac{[P_1] \text{ c } [Q_1] \quad [P_2] \text{ c } [Q_2]}{[P_1 \vee P_2] \text{ c } [Q_1 \vee Q_2]} \begin{array}{l} \Rightarrow \text{ by rule [disj]} \\ \Leftarrow \text{ by rule [cons]} \end{array} \end{array}$$

$$[P] \text{ c } [Q_1] \wedge [P] \text{ c } [Q_2] \Leftrightarrow [P] \text{ c } [Q_1 \vee Q_2]$$

$$\{P\} c \{Q_1\} \wedge \{P\} c \{Q_2\} \iff \{P\} c \{Q_1 \wedge Q_2\}$$

$$\text{\{conj\}} \frac{\{P_1\} c \{Q_1\} \quad \{P_2\} c \{Q_2\}}{\{P_1 \wedge P_2\} c \{Q_1 \wedge Q_2\}} \begin{array}{l} \Rightarrow \text{by rule \{conj\}} \\ \Leftarrow \text{by rule \{cons\}} \end{array}$$

dropping conjuncts by {cons} rule

$$\frac{\{P\} c \{Q \wedge R\}}{\{P\} c \{Q\}}$$

$$Q \wedge R \Rightarrow Q$$

dropping disjuncts by [cons] rule

$$\frac{[P] \text{ c } [Q \vee R]}{[P] \text{ c } [Q]}$$

$$Q \vee R \Leftarrow Q$$

$[y = 0]$ if $even(x)$ then $y := 42$ $[y = 42 \wedge even(x)]$

Just seen



$$\{y = 0\} \text{ if } \textit{even}(x) \text{ then } y := 42 \{y = 42 \wedge \textit{even}(x)\}$$

For instance, $\sigma' = [x \mapsto 1, y \mapsto 0] \models (y = 0)$



$$\{y = 0 \wedge \text{even}(x)\} \text{ if } \text{even}(x) \text{ then } y := 42 \{y = 42\}$$

Note that also the following triple is valid, but we can drop the part $\text{even}(x)$:
 $\{y = 0 \wedge \text{even}(x)\} \text{ if } \text{even}(x) \text{ then } y := 42 \{y = 42 \wedge \text{even}(x)\}$



$[y = 0 \wedge \text{even}(x)]$ if $\text{even}(x)$ then $y := 42$ $[y = 42]$

$[y = 42]$ is too large and not sufficiently precise: we have forgotten that x is even. Also $\sigma' = [x \mapsto 1, y \mapsto 0] \models (y = 42)$



For incorrectness reasoning

You **must remember** information as you go along a path, but you **get to forget** some of the paths

How do we unroll a finite number of times?

$$\begin{array}{c}
 \frac{\frac{\frac{[P_0] \text{ r}^* [P_0]}{[P_0] \text{ r}^*; \text{r} [P_1]} \text{ [iter0]} \quad \frac{\dots}{[P_0] \text{ r} [P_1]} \text{ [seq]}}{[P_0] \text{ r}^* [P_1]} \text{ [unroll]} \quad \frac{\dots}{[P_1] \text{ r} [P_2]} \text{ [seq]}}{[P_0] \text{ r}^*; \text{r} [P_2]} \text{ [unroll]} \\
 \frac{}{[P_0] \text{ r}^* [P_2]}
 \end{array}$$

Key Idea: loop unrolling proves a loop triple by evaluating a finite number of iterations, without requiring invariants. It allows us to discover some post-conditions for a loop

Termination: reachability does not imply that a loop must terminate on all executions: only that enough paths terminate to cover all the states in the result assertion

Soundness: it is enough to show the result is reachable via some execution path, rather than reasoning about all possible loop behaviors

$$\text{while } b \text{ do } c \triangleq (b?; c)^*; \neg b?$$

$$\frac{\forall n \in \mathbb{N}. [P_n \wedge b] \text{ c } [P_{n+1}]}{\forall n \in \mathbb{N}. [P_n] b?; \text{ c } [P_{n+1}]}$$

$$\frac{\forall n \in \mathbb{N}. [P_n] b?; \text{ c } [P_{n+1}]}{[P_0] (b?; c)^* [\exists k. P_k]}$$

$$\frac{[P_0] (b?; c)^* [\exists k. P_k]}{[P_0] \text{ while } b \text{ do } c [\exists k. P_k \wedge \neg b]}$$

$$\frac{[P_0] \text{ while } b \text{ do } c [P_0 \wedge \neg b] \quad \frac{[P_0 \wedge b] \text{ c } [P_1]}{[P_0] \text{ while } b \text{ do } c [(P_0 \vee P_1) \wedge \neg b]}}{[P_0] \text{ while } b \text{ do } c [(P_0 \vee P_1) \wedge \neg b]}$$

Principle of agreement

Theorem.

If $\llbracket P' \rrbracket c \llbracket Q' \rrbracket$
 $\wedge P' \Rightarrow P$
 $\wedge \{P\} c \{Q\}$
 then $Q' \Rightarrow Q$

Proof.

Q'
 $\subseteq \llbracket c \rrbracket P' \quad // \text{ by IL}$
 $\subseteq \llbracket c \rrbracket P \quad // P' \Rightarrow P$
 $\subseteq Q \quad // \text{ by HL}$

Principle of denial

$\llbracket P' \rrbracket c \llbracket Q' \rrbracket \wedge P' \subseteq P \wedge \neg (Q' \subseteq Q) \Rightarrow \neg \{P\} c \{Q\}$

$\llbracket P \rrbracket c[\epsilon : Q]$

- ok: normal (successful) exec
- er: erroneous exec

Semantics: compositions

$\llbracket c_1 + c_2 \rrbracket \epsilon \triangleq \llbracket c_1 \rrbracket \epsilon \cup \llbracket c_2 \rrbracket \epsilon$

$\llbracket c^* \rrbracket \epsilon \triangleq \bigcup_{k \in \mathbb{N}} \llbracket c^k \rrbracket \epsilon \quad \text{where } c^k \triangleq \underbrace{c; \dots; c}_{k \text{ times}}$

$\llbracket c_1; c_2 \rrbracket \text{ok} \triangleq \llbracket c_2 \rrbracket \text{ok} \circ \llbracket c_1 \rrbracket \text{ok}$

$\llbracket c_1; c_2 \rrbracket \text{er} \triangleq \llbracket c_1 \rrbracket \text{er} \cup (\llbracket c_2 \rrbracket \text{er} \circ \llbracket c_1 \rrbracket \text{ok})$

$S, T \subseteq \Sigma \times \Sigma$

$T \circ S \triangleq \{(\sigma_1, \sigma_2) \mid \exists \sigma. (\sigma_1, \sigma) \in S \wedge (\sigma, \sigma_2) \in T\} \subseteq \Sigma \times \Sigma$

$\llbracket c \rrbracket \text{ok} \subseteq \Sigma \times \Sigma$

$\llbracket c \rrbracket \text{er} \subseteq \Sigma \times \Sigma$

$\llbracket c \rrbracket \epsilon \subseteq \Sigma \times \Sigma$

any of
ok, er

c_1 fails

c_1 ok, but
 c_2 fails

c_1 ok,
but ...

$\llbracket \text{frame} \rrbracket$ assigned variables in c are disjoint from free variables in R

HL: Given c and Q , there is a (weakest)
 precondition P such that $\{P\} c \{Q\}$
 proof: take $P = wlp(c, Q)$

IL: Given c and Q , a (weakest) precondition P
 such that $[P] c [Q]$ may not exist
 proof: take Q with some unreachable state
 e.g. $[?] x := 42 [true]$

Minimal set of rules

$$\begin{array}{c}
 \frac{}{[P] e [[e]]P} \text{[atom]} \qquad \frac{[P] c_1 [R] \quad [R] c_2 [Q]}{[P] c_1; c_2 [Q]} \text{[seq]} \\
 \\
 \frac{\forall i \in \{1,2\} \quad [P] c_i [Q_i]}{[P] c_1 + c_2 [Q_1 \cup Q_2]} \text{[choice]} \qquad \frac{\forall n \geq 0. [P_n] c [P_{n+1}]}{[P_0] c^\star [\exists k. P_k]} \text{[iter]} \\
 \\
 \frac{P' \Rightarrow P \quad [P'] c [Q] \quad Q \Rightarrow Q'}{[P] c [Q]} \text{[cons]}
 \end{array}$$

Auxiliary rules

$$\frac{[P_1] \text{ c } [Q_1] \quad [P_2] \text{ c } [Q_2]}{[P_1 \vee P_2] \text{ c } [Q_1 \vee Q_2]} \text{ [disj]}$$

$$\frac{}{[P] \text{ c}^\star [P]} \text{ [iter0]}$$

$$\frac{[P] \text{ c}^\star; \text{ c } [Q]}{[P] \text{ c}^\star [Q]} \text{ [unroll]}$$

$$\frac{[P] \text{ c } [Q]}{[P \wedge R] \text{ c } [Q \wedge R]} \text{ [frame]}$$

assigned variables in c
are disjoint from
free variables in R

$$\frac{P' \Rightarrow P \quad [P'] \text{ c } [Q]}{[P] \text{ c } [Q]} \text{ [weak]}$$

$$\frac{[P] \text{ c } [Q] \quad Q \Rightarrow Q'}{[P] \text{ c } [Q]} \text{ [stren]}$$

Soundness:

Th. Any derivable IL triple is valid

Proof. By induction on the derivation tree

(Relative) Completeness:

Th. Any valid IL triple can be derived, provided we have an oracle to decide implications

Proof sketch. (Assuming an oracle to decide implications.) Solution: we go semantic! P and Q are sets of states, implications is subset inclusion, no issues with finiteness.

Proof sketch ctd.

First, we show that $[P] \text{ c } \llbracket c \rrbracket P$ is derivable for any c and P .
(We do so by *induction* on the structure of c .)

Since any valid triple $[P] \text{ c } [Q]$ satisfies $\llbracket c \rrbracket P \supseteq Q$,
we derive it using the rule of consequence

$$\frac{\frac{}{[P] \text{ c } \llbracket c \rrbracket P} \quad \llbracket c \rrbracket P \supseteq Q}{[P] \text{ c } [Q]}$$

Proof sketch ctd. ($[P] \text{ c } \llbracket c \rrbracket P$ is derivable)

Roughly, by structural induction on the command c .

Atomic commands: **[atom]**

Choice and sequence: **by inductive hyp. + [choice] / [seq]**

- $[P] c^* [\text{ok} : \llbracket c^* \rrbracket \text{ok } P]$

There exists a finite number of iterations of c that, starting from a state in P , reaches a state satisfying $\llbracket c^* \rrbracket \text{ok } P$ (shortly written $\llbracket c^* \rrbracket P$)

- inductive hyp. $[P] c \llbracket [c] P \rrbracket$

$$\frac{\forall n \geq 0. [P_n] c [P_{n+1}]}{[P_0] c^* [\exists k. P_k]} \text{[iter]}$$

- $P_n \triangleq \llbracket c^n \rrbracket P$ and $P_0 \triangleq P$ we instantiate [iter] as follows $\frac{\forall n \geq 0. \llbracket [c^n] P \rrbracket c \llbracket [c^{n+1}] P \rrbracket}{[P] c^* [\exists k. \llbracket c^k \rrbracket P]}$

by applying the inductive hypothesis we know that

$$\llbracket [c^n] P \rrbracket c \llbracket [c] (\llbracket c^n \rrbracket P) \rrbracket \text{ but } \llbracket [c] (\llbracket c^n \rrbracket P) \rrbracket \equiv \llbracket [c^n; c] P \rrbracket \equiv \llbracket [c^{n+1}] P \rrbracket$$

by applying applying [iter] we can conclude $[P] c^* [\exists k. \llbracket c^k \rrbracket P] \equiv \llbracket [c^*] P \rrbracket$

13

- $[P] c^* [\text{er} : Q_{\text{er}}]$

There exists a finite number of iterations of c that, starting from a state in P , reaches a state satisfying $[\text{er} : Q_{\text{er}}]$

By inductive hyp. $\forall R, [R] c [\text{er} : Q_{\text{er}}]$ and by previous result

we know that $[P] c^* [\exists k. \llbracket c^k \rrbracket P]$. By instantiating $R = \exists k. \llbracket c^k \rrbracket P$ we have

$$\frac{[P] c^* [\exists k. \llbracket c^k \rrbracket P] \quad [\exists k. \llbracket c^k \rrbracket P] c [\text{er} : Q_{\text{er}}]}{[P] c^*; c [\text{er} : Q_{\text{er}}]} \text{ by } \frac{[P] c^*; c [Q]}{[P] c^* [Q]} \text{[unroll]}$$

we conclude $[P] c^* [\text{er} : Q_{\text{er}}]$

$$[P] c [Q]$$

$$\text{validity: } \llbracket [c] P \rrbracket \supseteq Q$$

$$\forall \delta \in Q. \exists \sigma \in P. \delta \in \llbracket [c] \sigma \rrbracket$$

can prove the presence of bugs

(any error in Q is reachable executing c)

NC

$\llbracket [c] \rrbracket^{op} \triangleq \{\sigma \mid \delta \in \llbracket [c] \sigma \rrbracket\} \rightarrow \text{weakest possible precondition (wpp)}$

$$\sigma \in [[c]]^{op} \delta \iff \delta \in [[c]] \sigma$$

$$[[c]]^{op} Q = \bigcup_{\delta \in Q} [[c]]^{op} \delta$$

specification = What my code should do and should not do

Sufficient Precondition: if it holds, the code is correct

Sufficient Precondition: if it does not hold... we don't know

Necessary Precondition: if it **does not** hold, the code is not correct

Necessary Precondition: if it **does not** hold, something bad can happen

The idea of NC is to prevent the invocation of the function with arguments that will inevitably lead to some error

$$(P)c(Q) \iff [[c]]^{op} Q \subseteq P$$

Given Q the set of good final states (the specification), the NC triple means that any states σ that admits at least one non-erroneous execution of c is in P

Program traces: $T = G \cup B \cup I$

G = good traces, terminating in a state in Q

B = bad traces, terminating outside Q in an assertion violation

assertions:

Language-induced: division by zero, null pointers, buffer overrun ...

User-supplied annotations: assertions, preconditions, postconditions, object invariants

I = infinite traces, non-termination

Notation: $G(\sigma)$ are the good traces starting from σ ,....

$$\sigma \in wlp(c, Q) \implies B(\sigma) = \emptyset$$

$$G(\sigma) \neq \emptyset \implies \sigma \in P \quad (\text{i.e., } \sigma \notin P \implies G(\sigma) = \emptyset)$$

$$\sigma \notin P \implies ((I(\sigma) = \emptyset) \implies B(\sigma) \neq \emptyset \wedge G(\sigma) = \emptyset)$$

i.e., if $\sigma \notin P$, either

a computation **diverge**, or reach only **bad states**

Weaken the pre and strengthen the post!

$$(NC) \frac{P' \Rightarrow P \quad (P') c (Q') \quad Q \Rightarrow Q'}{(P) c (Q)}$$

$$\{P\}c\{Q\} \iff (\neg P)c(\neg Q)$$

$$[[c]]P \subseteq Q \iff [[c]]^{op}(\neg Q) \subseteq \neg P$$

$$(P) \ c \ (Q)$$

$$\text{validity: } P \supseteq \llbracket \overleftarrow{c} \rrbracket Q$$

$$\forall \delta \in Q. \forall \sigma \in \llbracket \overleftarrow{c} \rrbracket \delta. \sigma \in P$$

express necessary conditions for correctness
(any execution of c from outside P is incorrect)

Other:

Recalling that both $\{\text{false}\} \ c \ \{Q\}$ and $\{P\} \ c \ \{\text{true}\}$ are valid HL triples (for any P, Q and c)
can we claim something about the validity of NC triples such as

$$\{\text{false}\} \ c \ (Q) \iff \{\text{true}\} \ c \ \{\neg Q\} \quad \times$$

$$(P) \ c \ \{\text{true}\} \iff \{\neg P\} \ c \ \{\text{false}\} \quad \times$$

$$\{\text{true}\} \ c \ (Q) \iff \{\text{false}\} \ c \ \{\neg Q\} \quad \checkmark$$

$$(P) \ c \ \{\text{false}\} \iff \{\neg P\} \ c \ \{\text{true}\} \quad \checkmark$$

In base all'**isomorfismo tra Necessary Condition Logic (NC) e Hoare Logic (HL)**, la validità di una tripla NC $(P)c(Q)$ è equivalente alla validità della tripla di Hoare $\neg Pc\neg Q$.

Ecco l'analisi della validità per i casi da te elencati:

1. Triple sempre valide (per ogni c, P, Q)

Queste triple NC sono valide perché i loro equivalenti in HL sono assiomi fondamentali del sistema:

- $\{\text{true}\} \ c \ (Q) \iff \{\text{false}\} \ c \ \{\neg Q\}$: Questa tripla è **sempre valida**. In HL, una pre-condizione **false** rende la tripla vacuamente vera poiché non esiste alcuno

stato iniziale che la soddisfi.

- $(P) \ c \ (false) \iff \{\neg P\} \ c \ \{true\}$: Questa tripla è **sempre valida**. In HL, una post-condizione `true` è sempre soddisfatta da qualsiasi stato finale raggiunto dal programma.

2. Triple generalmente non valide

La loro validità dipende dal comportamento specifico del comando c e non può essere garantita a priori:

- $(false) \ c \ (Q) \iff \{true\} \ c \ \{\neg Q\}$: È valida solo se il programma c , partendo da qualsiasi stato, termina sempre in $\neg Q$ o diverge. Poiché non è vera per ogni c , la tripla NC non è universalmente valida.
- $(P) \ c \ (true) \iff \{\neg P\} \ c \ \{false\}$: È valida solo se il programma c diverge (va in loop infinito) per ogni stato iniziale in $\neg P$. In caso di terminazione, la post-condizione `false` non sarebbe soddisfatta, rendendo la tripla NC non valida.

In sintesi: Mentre in HL è "facile" essere corretti partendo da `false` o arrivando a `true`, in NC la validità universale si ottiene quando la pre-condizione è `true` (perché è sempre possibile trovare un'esecuzione corretta partendo da "ovunque" se la precondizione HL è falsa) o la post-condizione è `false`.

SIL

	Forward	Backward
Over	HL $\{P\} \ c \ \{Q\}$ $\llbracket c \rrbracket P \subseteq Q$ $\forall \sigma \in P. \forall \delta \in \llbracket c \rrbracket \sigma. \delta \in Q$	NC $(P) \ c \ (Q)$ $P \supseteq \llbracket \overleftarrow{c} \rrbracket Q$ $\forall \delta \in Q. \forall \sigma \in \llbracket \overleftarrow{c} \rrbracket \delta. \sigma \in P$
Under	IL $[P] \ c \ [Q]$ $\llbracket c \rrbracket P \supseteq Q$ $\forall \delta \in Q. \exists \sigma \in P. \delta \in \llbracket c \rrbracket \sigma$	SIL $\langle P \rangle \ c \ \langle Q \rangle$ $P \subseteq \llbracket \overleftarrow{c} \rrbracket Q$ $\forall \sigma \in P. \exists \delta \in Q. \delta \in \llbracket c \rrbracket \sigma$

$\langle P \rangle c \langle Q \rangle$ is valid when $\llbracket c \rrbracket^{op} Q \supseteq P$

$\forall \sigma \in P. \exists \delta \in Q. \delta \in \llbracket c \rrbracket \sigma$

A backward under-approximation logic to expose some initial states leading to errors

$$\langle P \rangle c \langle Q \rangle$$

$$\text{validity: } P \subseteq \llbracket c \rrbracket^{op} Q$$

$$\forall \sigma \in P. \exists \delta \in Q. \delta \in \llbracket c \rrbracket \sigma$$

express sufficient conditions for incorrectness
(any state in P can lead within *er* : Q)

Q is a manifest error $\iff \langle true \rangle c \langle Q \rangle$ is valid

$\frac{}{\llbracket \langle \bar{c} \rangle \rrbracket Q \rangle c \langle Q \rangle} \langle \text{atom} \rangle$	$\frac{P \subseteq P' \quad \langle P' \rangle r \langle Q' \rangle \quad Q' \subseteq Q}{\langle P \rangle r \langle Q \rangle} \langle \text{cons} \rangle$
$\frac{\langle P_1 \rangle r_1 \langle Q \rangle \quad \langle P_2 \rangle r_2 \langle Q \rangle}{\langle P_1 \cup P_2 \rangle r_1 + r_2 \langle Q \rangle} \langle \text{choice} \rangle$	$\frac{\langle P \rangle r_1 \langle R \rangle \quad \langle R \rangle r_2 \langle Q \rangle}{\langle P \rangle r_1; r_2 \langle Q \rangle} \langle \text{seq} \rangle$
$\frac{\forall n \geq 0. \langle Q_{n+1} \rangle r \langle Q_n \rangle}{\langle \bigcup_{n \geq 0} Q_n \rangle r^* \langle Q_0 \rangle} \langle \text{iter} \rangle$	
Additional rules	
$\frac{}{\langle \emptyset \rangle r \langle Q \rangle} \langle \text{empty} \rangle$	$\frac{\langle P_1 \rangle r \langle Q_1 \rangle \quad \langle P_2 \rangle r \langle Q_2 \rangle}{\langle P_1 \cup P_2 \rangle r \langle Q_1 \cup Q_2 \rangle} \langle \text{disj} \rangle$
$\frac{}{\langle Q \rangle r^* \langle Q \rangle} \langle \text{iter0} \rangle$	$\frac{\langle P \rangle r^*; r \langle Q \rangle}{\langle P \rangle r^* \langle Q \rangle} \langle \text{unroll} \rangle$
$\frac{\langle P \rangle r^*; r \langle Q_1 \rangle}{\langle P \cup Q_2 \rangle r^* \langle Q_1 \cup Q_2 \rangle} \langle \text{unroll-split} \rangle$	

Soundness:

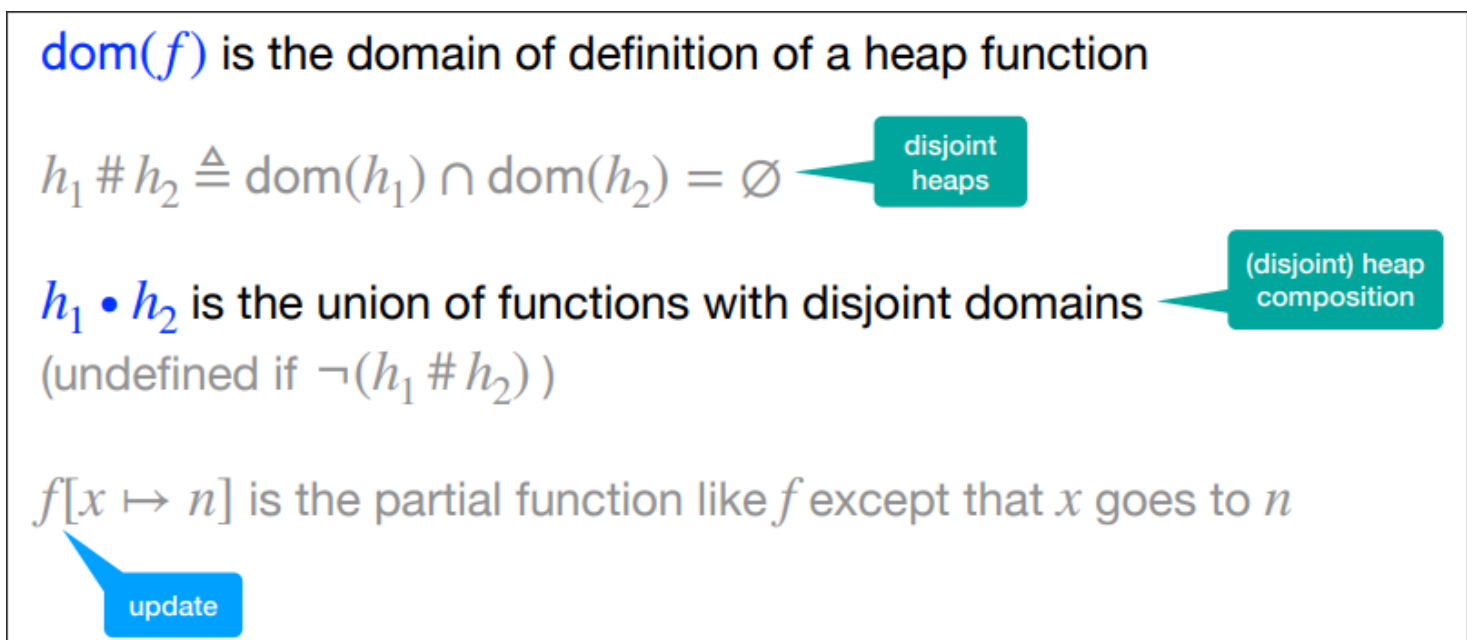
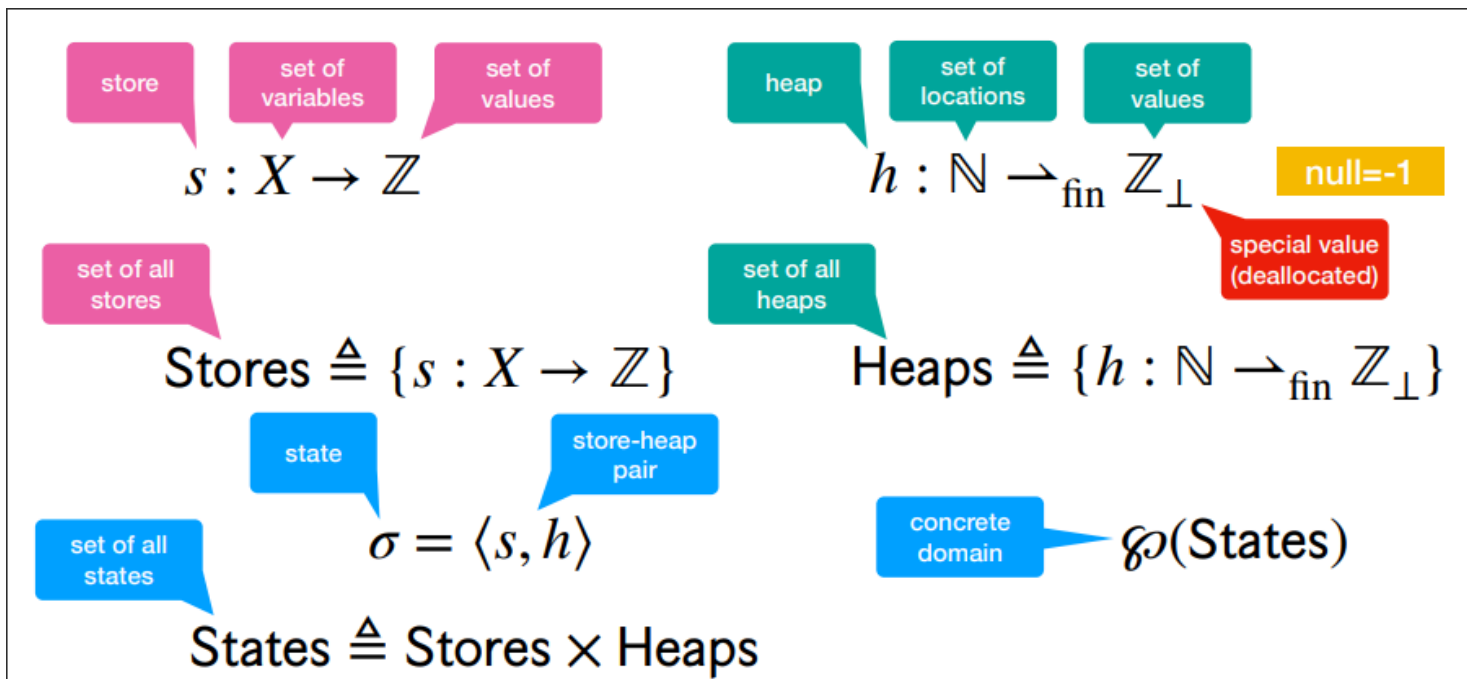
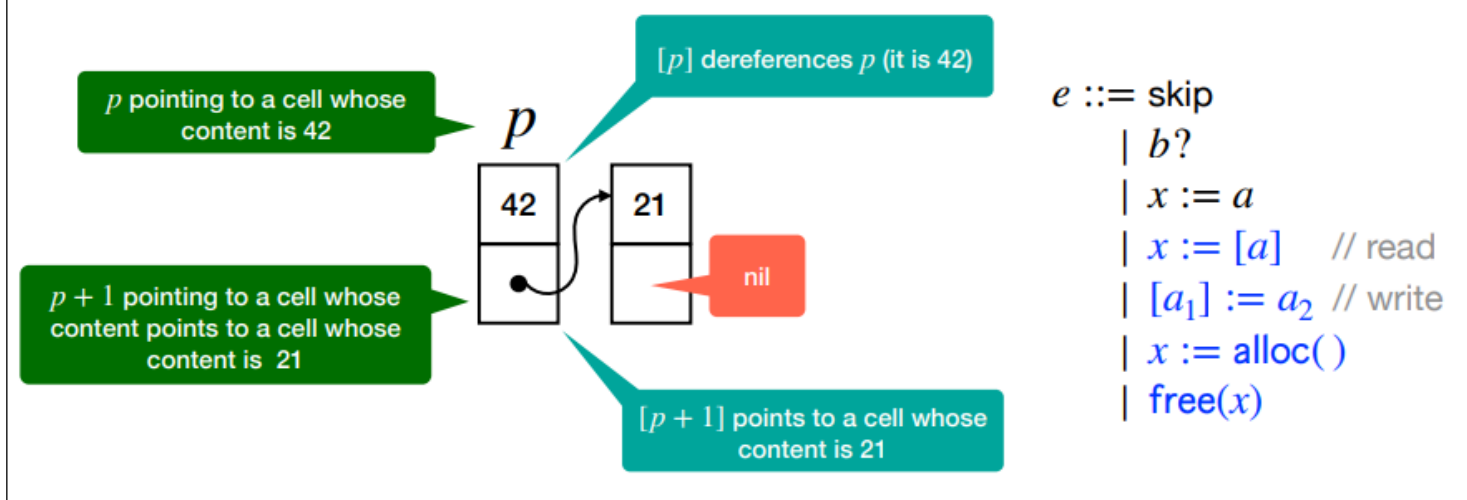
Th. All provable SIL triples are valid

Completeness:

Th. All valid triples are provable (using core rules)

SL

HL + pointers



$$\langle s, h \rangle \models P$$

the assertion P holds
for the given state $\langle s, h \rangle$

$$\langle s, h \rangle \models a_1 < a_2 \quad \text{iff} \quad s \models a_1 < a_2$$

$$\langle s, h \rangle \models P_1 \wedge P_2 \quad \text{iff} \quad \langle s, h \rangle \models P_1 \text{ and } \langle s, h \rangle \models P_2$$

$$\langle s, h \rangle \models \forall x. P \quad \text{iff} \quad \forall v \in \mathbb{Z}. \langle s[x \mapsto v], h \rangle \models P$$

$$\langle s, h \rangle \models P$$

the assertion P holds
for the given state $\langle s, h \rangle$

$$\langle s, h \rangle \models a_1 \mapsto a_2 \quad \text{iff} \quad \text{dom}(h) = \{\llbracket a_1 \rrbracket s\} \text{ and } h(\llbracket a_1 \rrbracket s) = \llbracket a_2 \rrbracket s$$

$$\langle s, h \rangle \models \text{emp} \quad \text{iff} \quad h \text{ is the empty map } []$$

$$\langle s, h \rangle \models P_1 * P_2 \quad \text{iff} \quad \exists h_1, h_2. \langle s, h_1 \rangle \models P_1 \text{ and } \langle s, h_2 \rangle \models P_2 \text{ and } h = h_1 \bullet h_2$$

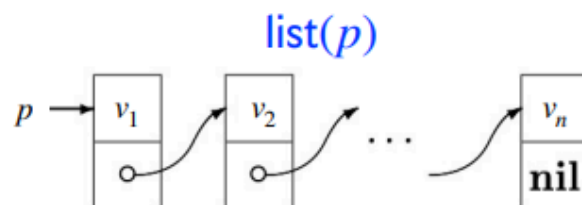
splits the heap
but not the store!

Let us define the following inductive predicate for **list segments**

$$\text{ls}(a_1, a_2, 0) \triangleq a_1 = a_2 \wedge \text{emp}$$

$$\text{ls}(a_1, a_2, n + 1) \triangleq a_1 \neq a_2 \wedge \exists \ell. a_1 \mapsto \ell * \text{ls}(\ell, a_2, n)$$

and let $\text{ls}(a_1, a_2) \triangleq \exists n. \text{ls}(a_1, a_2, n)$ and $\text{list}(a) \triangleq \text{ls}(a, \text{nil})$

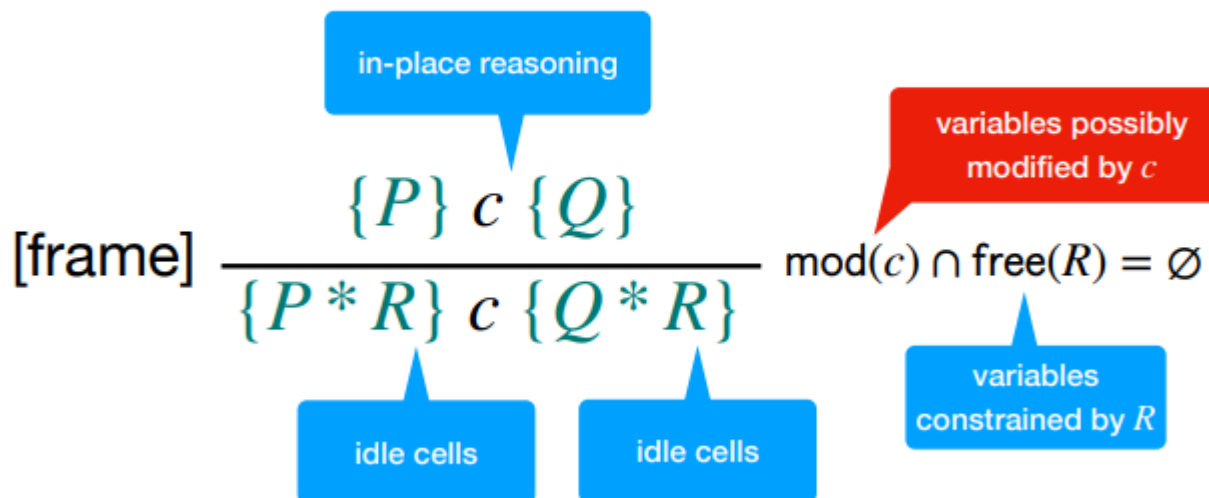


$\{P\}$ atomic command $\{Q\}$

minimal
requirement
for correct
execution

in-place reasoning

strongest
postcondition,
given the
minimal
precondition



$\text{mod}(\text{skip}) = \emptyset$

$\text{mod}(b?) = \emptyset$

$\text{mod}(\text{free}(x)) = \emptyset$

$\text{mod}([x] := y) = \emptyset$

$\text{mod}(c_1 + c_2) = \text{mod}(c_1) \cup \text{mod}(c_2)$

$\text{mod}(x := a) = \{x\}$

$\text{mod}(x := \text{alloc}()) = \{x\}$

$\text{mod}(x := [y]) = \{x\}$

$\text{mod}(c_1; c_2) = \text{mod}(c_1) \cup \text{mod}(c_2)$

$\text{mod}(c^*) = \text{mod}(c)$

Note that $\text{free}(x)$ and $[x] := y$ do not modify x :
this is because they only modify the value pointed by x , not the actual value of x

$$fv(true) = \emptyset$$

$$fv(false) = \emptyset$$

$$fv(a_1 = a_2) = fv(a_1) \cup fv(a_2)$$

$$fv(p \wedge q) = fv(p) \cup fv(q)$$

...

$$fv(\exists x. p) = fv(p) \setminus \{x\}$$

$$fv(emp) = \emptyset$$

$$fv(x \mapsto a) = \{x\} \cup fv(a)$$

$$fv(p * q) = fv(p) \cup fv(q)$$

structural
assertions

$$a \mapsto _ \triangleq \exists v. a \mapsto v \quad (\text{for } v \text{ fresh})$$

$$a_1 \doteq a_2 \triangleq (a_1 = a_2) \wedge emp$$

$$a \mapsto \langle a_0, \dots, a_k \rangle \triangleq (a \mapsto a_0) * \dots * (a + k \mapsto a_k)$$

$$[\text{write}] \frac{}{\{a_1 \mapsto _ \} [a_1] := a_2 \{a_1 \mapsto a_2 \}}$$

$$[\text{read}] \frac{}{\{a \mapsto v \wedge x = x'\} x := [a] \{x = v \wedge a[x'/x] \mapsto v \}}$$

not strictly necessary

$$[\text{alloc}] \frac{}{\{emp\} x := alloc() \{x \mapsto _ \}}$$

$$[\text{dispose}] \frac{}{\{a \mapsto _ \} free(a) \{emp \}}$$

$$\{x \doteq x'\} \quad x := \text{cons}(a_1, \dots, a_k) \quad \{x \mapsto \langle a_1[x'/x], \dots, a_k[x'/x] \rangle\}$$

$$a \mapsto \langle a_0, \dots, a_k \rangle \triangleq (a \mapsto a_0) * \dots * (a + k \mapsto a_k)$$

Relational semantics

$$\llbracket \text{skip} \rrbracket \triangleq \{(\sigma, \sigma)\}$$

$$\llbracket b? \rrbracket \triangleq \{(\sigma, \sigma) \mid \sigma = \langle s, h \rangle \wedge s \models b\}$$

$$\llbracket x := a \rrbracket \triangleq \{(\langle s, h \rangle, \langle s[x \mapsto \llbracket a \rrbracket s], h) \rangle\}$$

$$\llbracket x := [a] \rrbracket \triangleq \{(\langle s, h \rangle, \langle s[x \mapsto v], h \rangle) \mid v = h(\llbracket a \rrbracket s) \in \mathbb{Z}\}$$

$$\llbracket [a_1] := a_2 \rrbracket \triangleq \{(\langle s, h \rangle, \langle s, h[\llbracket a_1 \rrbracket s \mapsto \llbracket a_2 \rrbracket s] \rangle) \mid h(\llbracket a_1 \rrbracket s) \in \mathbb{Z}\}$$

$$\llbracket x := \text{alloc}() \rrbracket \triangleq \{(\langle s, h \rangle, \langle s[x \mapsto n], h[n \mapsto v] \rangle) \mid v \in \mathbb{Z} \wedge (n \notin \text{dom}(h) \vee h(n) = \perp)\}$$

$$\llbracket \text{free}(x) \rrbracket \triangleq \{(\langle s, h \bullet [s(x) \mapsto v] \rangle, \langle s, h \rangle) \mid s(x) \in \mathbb{N} \wedge v \in \mathbb{Z}\}$$

$$\llbracket x := \text{cons}(a_0, \dots, a_k) \rrbracket \triangleq \{(\langle s, h \rangle, \langle s[x \mapsto n], h[n \mapsto \llbracket a_0 \rrbracket s, \dots, (n+k) \mapsto \llbracket a_k \rrbracket s] \rangle) \mid (\forall i \in [n, n+k]. i \notin \text{dom}(h) \vee h(i) = \perp)\}$$

Correctness:

Th. if $\{P\}c\{Q\}$ then $\llbracket c \rrbracket P \subseteq Q$

Proof. By induction on the derivation

Incompleteness:

Th. There exist valid SL triples that are not provable

Proof. Footprint

$$a \doteq b \triangleq (a = b) \wedge \text{emp}$$

Footprint:

1. Derive the spec of c using local axioms
2. Apply rule frame to complete the specification

ISL

$$\text{ISL} = \text{IL} + \text{SL}$$

bug finding

local reasoning

IL

 $[P] \text{ c } [\epsilon : Q]$

SL

$$\frac{\{P\} \text{ c } \{Q\}}{\{P * R\} \text{ c } \{Q * R\}}$$

ISL

$$\frac{[P] \text{ c } [\epsilon : Q]}{[P * R] \text{ c } [\epsilon : Q * R]}$$

track deallocated locations

 $\langle s, h \rangle \models a_1 \mapsto a_2 \quad \text{iff } \text{dom}(h) = \{\llbracket a_1 \rrbracket s\} \text{ and } h(\llbracket a_1 \rrbracket s) = \llbracket a_2 \rrbracket s$
 $\langle s, h \rangle \models \text{emp} \quad \text{iff } h \text{ is the empty map } []$
 $\langle s, h \rangle \models P_1 * P_2 \quad \text{iff } \exists h_1, h_2. \langle s, h_1 \rangle \models P_1 \text{ and } \langle s, h_2 \rangle \models P_2 \text{ and } h = h_1 \bullet h_2$
 $\langle s, h \rangle \models x \nrightarrow \quad \text{iff } \text{dom}(h) = \{s(x)\} \text{ and } h(s(x)) = \perp$

Notably: $x \mapsto v * x \nrightarrow \equiv \text{false} \equiv x \nrightarrow * x \nrightarrow$

$y \mapsto v * x \nrightarrow \equiv y \mapsto v * x \nrightarrow \wedge x \neq y$

Actual ISL rules

SKIP $\vdash [\text{emp}] \text{ skip } [ok : \text{emp}]$	ASSIGN $\vdash [x=x'] \text{ x} := e \text{ } [ok : x=e[x'/x]]$	HAVOC $\vdash [x=x'] \text{ x} := * [ok : x=v]$	FRAME $\frac{\vdash [p] \text{ C } [\epsilon : q] \quad \text{mod}(\text{C}) \cap \text{fv}(r) = \emptyset}{\vdash [p * r] \text{ C } [\epsilon : q * r]}$	ALLOC1 $\vdash [x=x'] \text{ x} := \text{alloc}() \text{ } [ok : x \mapsto -]$
ASSUME $\vdash [\text{emp}] \text{ assume}(B) \text{ } [ok : B]$	ERROR $\vdash [\text{emp}] \text{ L: error } [er(\text{L}) : \text{emp}]$		FREE $\vdash [x \mapsto e] \text{ L: free}(x) \text{ } [ok : x \nrightarrow]$	ALLOC2 $\vdash [x=x' * y \nrightarrow] \text{ x} := \text{alloc}() \text{ } [ok : x=y * y \mapsto -]$
SEQ1 $\frac{\vdash [p] \text{ C}_1 [er(\text{L}) : q]}{\vdash [p] \text{ C}_1; \text{C}_2 [er(\text{L}) : q]}$	SEQ2 $\frac{\vdash [p] \text{ C}_1 [ok : r] \quad \vdash [r] \text{ C}_2 [\epsilon : q]}{\vdash [p] \text{ C}_1; \text{C}_2 [\epsilon : q]}$	LOOP1 $\vdash [p] \text{ C}^* [ok : p]$	FREEER $\vdash [x \nrightarrow] \text{ L: free}(x) \text{ } [er(\text{L}) : x \nrightarrow]$	FREE NULL $\vdash [x=\text{null}] \text{ L: free}(x) \text{ } [er(\text{L}) : x=\text{null}]$
CHOICE $\frac{\vdash [p] \text{ C}_i [\epsilon : q] \quad \text{for some } i \in \{1, 2\}}{\vdash [p] \text{ C}_1 + \text{C}_2 [\epsilon : q]}$	EXIST $\frac{\vdash [p] \text{ C } [\epsilon : q] \quad x \notin \text{fv}(\text{C})}{\vdash [\exists x. p] \text{ C } [\epsilon : \exists x. q]}$	LOOP2 $\frac{\vdash [p] \text{ C}^* \text{ C } [\epsilon : q]}{\vdash [p] \text{ C}^* [\epsilon : q]}$	LOAD $\vdash [x=x' * y \mapsto e] \text{ L: x} := [y] \text{ } [ok : x=e[x'/x] * y \mapsto e[x'/x]]$	STORE $\vdash [x \mapsto e] \text{ L: [x]} := y \text{ } [ok : x \mapsto y]$
CONS $\frac{p' \Rightarrow p \quad \vdash [p'] \text{ C } [\epsilon : q'] \quad q \Rightarrow q'}{\vdash [p] \text{ C } [\epsilon : q]}$	DISJ $\frac{\vdash [p_1] \text{ C } [\epsilon : q_1] \quad \vdash [p_2] \text{ C } [\epsilon : q_2]}{\vdash [p_1 \vee p_2] \text{ C } [\epsilon : q_1 \vee q_2]}$		LOADER $\vdash [y \nrightarrow] \text{ L: x} := [y] \text{ } [er(\text{L}) : y \nrightarrow]$	STOREER $\vdash [x \nrightarrow] \text{ L: [x]} := y \text{ } [er(\text{L}) : x \nrightarrow]$
SUBST $\frac{\vdash [p] \text{ C } [\epsilon : q] \quad y \notin \text{fv}(p, \text{C}, q)}{\vdash [p[y/x]] \text{ C}[y/x] [\epsilon : q[y/x]]}$	LOCAL $\frac{\vdash [p] \text{ C } [\epsilon : q]}{\vdash [\exists x. p] \text{ local } x \text{ in } \text{C} [\epsilon : \exists x. q]}$		LOAD NULL $\vdash [y=\text{null}] \text{ L: x} := [y] \text{ } [er(\text{L}) : y=\text{null}]$	STORE NULL $\vdash [x=\text{null}] \text{ L: [x]} := y \text{ } [er(\text{L}) : x=\text{null}]$

Correctness:

Th. if $[P]c[\epsilon:Q]$ then $Q \subseteq [[c]]\epsilon(P)$

Proof. By induction on the derivation

Footprint:

Th. Any valid ISL triple $[\sigma_P]c[\epsilon:\sigma_P]$ can be derived

**ISL = IL + SL
for bug
catching!**

ISL address compositional bug catching

targets memory safety bugs (use-after-free)

no-false-positives theorem: all bugs are true

SepSIL

SepSIL = SIL + IL

SIL

$\langle P \rangle c \langle Q \rangle$

SL

$$\frac{\{P\} c \{Q\}}{\{P * R\} c \{Q * R\}}$$

SepSIL

$$\frac{\langle P \rangle c \langle Q \rangle}{\langle P * R \rangle c \langle Q * R \rangle}$$

Actual rules of SepSIL

$$\frac{}{\langle\langle\mathbf{emp}\rangle\rangle \text{ skip } \langle\langle\mathbf{emp}\rangle\rangle} \langle\langle\text{skip}\rangle\rangle$$

$$\frac{}{\langle\langle q[a/x] \rangle\rangle x := a \langle\langle q \rangle\rangle} \langle\langle\text{assign}\rangle\rangle$$

$$\frac{}{\langle\langle\mathbf{emp}\rangle\rangle x := \text{alloc}() \langle\langle x \mapsto v \rangle\rangle} \langle\langle\text{alloc1}\rangle\rangle$$

$$\frac{}{\langle\langle q \wedge b \rangle\rangle b? \langle\langle q \rangle\rangle} \langle\langle\text{assume}\rangle\rangle$$

$$\frac{}{\langle\langle \beta \not\mapsto \rangle\rangle x := \text{alloc}() \langle\langle x = \beta \wedge x \mapsto v \rangle\rangle} \langle\langle\text{alloc2}\rangle\rangle$$

$$\frac{}{\langle\langle x \mapsto - \rangle\rangle \text{free}(x) \langle\langle x \not\mapsto \rangle\rangle} \langle\langle\text{free}\rangle\rangle$$

$$\frac{x \notin \text{fv}(a)}{\langle\langle y \mapsto a * q[a/x] \rangle\rangle x := [y] \langle\langle y \mapsto a * q \rangle\rangle} \langle\langle\text{load}\rangle\rangle$$

$$\frac{}{\langle\langle x \mapsto - \rangle\rangle [x] := y \langle\langle x \mapsto y \rangle\rangle} \langle\langle\text{store}\rangle\rangle$$

$$\frac{\langle\langle p \rangle\rangle r \langle\langle q \rangle\rangle \quad \text{fv}(t) \cap \text{mod}(r) = \emptyset}{\langle\langle p * t \rangle\rangle r \langle\langle q * t \rangle\rangle} \langle\langle\text{frame}\rangle\rangle$$

$$\frac{\langle\langle p \rangle\rangle r \langle\langle q \rangle\rangle \quad x \notin \text{fv}(r)}{\langle\langle \exists x. p \rangle\rangle r \langle\langle \exists x. q \rangle\rangle} \langle\langle\text{exists}\rangle\rangle$$

$$\llbracket \text{skip} \rrbracket \triangleq \{(\sigma, \sigma)\}$$

$$\llbracket b? \rrbracket \triangleq \{(\sigma, \sigma) \mid \sigma = \langle s, h \rangle \wedge s \models b\}$$

$$\llbracket x := a \rrbracket \triangleq \{(\langle s, h \rangle, \langle s[x \mapsto \llbracket a \rrbracket s], h) \}$$

$$\llbracket x := [y] \rrbracket \triangleq \{(\langle s, h \rangle, \langle s[x \mapsto v], h \rangle) \mid v = h(s(y)) \in \mathbb{Z}\}$$

$$\llbracket [x] := y \rrbracket \triangleq \{(\langle s, h \rangle, \langle s, h[s(x) \mapsto s(y)] \rangle) \mid h(s(x)) \in \mathbb{Z}\}$$

$$\llbracket x := \text{alloc}() \rrbracket \triangleq \{(\langle s, h \rangle, \langle s[x \mapsto n], h[n \mapsto v] \rangle) \mid v \in \mathbb{Z} \wedge (n \notin \text{dom}(h) \vee h(n) = \perp)\}$$

$$\llbracket \text{free}(x) \rrbracket \triangleq \{(\langle s, h \rangle, \langle s, h[s(x) \mapsto \perp] \rangle) \mid h(s(x)) \in \mathbb{Z}\}$$

Correctness:

Th. if $\langle P \rangle c \langle Q \rangle$ then $P \subseteq \llbracket c \rrbracket^{op} Q$

Proof. by induction on the derivation

Completeness:

Th. Any valid triple $\langle P \rangle c \langle Q \rangle$ can be derived

Intro-AI

Abstract interpretation = A powerful framework for designing correct static analyses: simple and eye-opening

The concretization $\gamma(a)$ of an abstract element a is the set of program states that satisfy it.

Best abstraction

An abstract element a is the best abstraction
of a set of points S iff

$$1) S \subseteq \gamma(a)$$

is an over-
approximation

2) $S \subseteq \gamma(b)$ implies b coarser than a

$$\gamma(a) \subseteq \gamma(b)$$

any other over-
approximation of S
is coarser

Abstractions:

- Non-Relational:
 - signs $[x \leq 0, y \geq 0]$
 - intervals $[1 \leq x, 1 \leq y] \rightarrow$ more expressive than signs
- Relational:
 - Convex Polyhedra $[c_1x + c_2y \leq c] \rightarrow$ more expressive than intervals, Computing the best abstraction can be expensive or sometimes not even possible

Compositionality principle:

compute a sound analysis of a program by computing sound abstract semantics of program's components

analysis function:

input: a program p , an abstract element a (pre-state)

output: an abstract element a' (post-state)

$analysis(p, a)$

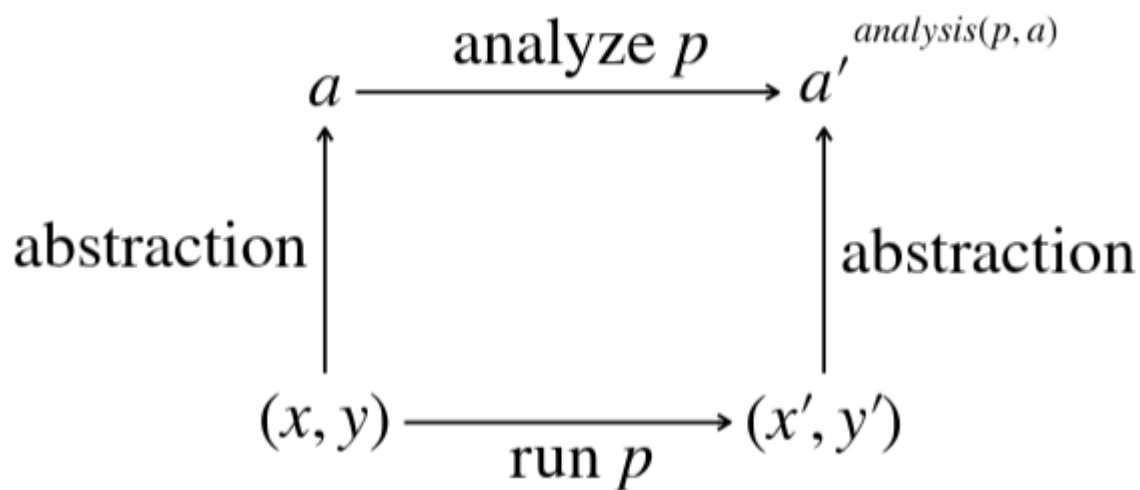
$\llbracket p \rrbracket^\# a$

a sound *analysis* function must capture the real executions:

if p moves a point (x, y) to (x', y')

then for any abstract element a s.t. $(x, y) \in \gamma(a)$

it must be the case that $(x', y') \in \gamma(analysis(p, a))$



Widening

To enforce termination of the analysis
 we need to enforce convergence of iterations
 (not necessary if finitely many abstract elements are traversed)

widening amounts to over-approximate
 the union of two abstract elements and
 further loose precision to gain convergence

Analysis, at a glance

```

analysis(init( $\mathfrak{R}$ ),  $a$ ) = best abstraction of the region  $\mathfrak{R}$ 
analysis(translation( $u, v$ ),  $a$ ) = { return an abstract state that contains
                                the translation of  $a$ 
analysis(rotation( $u, v, \theta$ ),  $a$ ) = { return an abstract state that contains
                                the rotation of  $a$ 
analysis( $\{p_0\}$  or  $\{p_1\}$ ,  $a$ ) = union(analysis( $p_1, a$ ), analysis( $p_0, a$ ))
analysis( $p_0; p_1$ ,  $a$ ) = analysis( $p_1$ , analysis( $p_0, a$ ))
analysis(iter{ $p$ },  $a$ ) = { R  $\leftarrow a$ ;
                        repeat
                            T  $\leftarrow R$ ;
                            R  $\leftarrow$  widen( $R$ , analysis( $p, R$ ));
                        until inclusion( $R, T$ )
                        return T;
  
```

Rice's Theorem: Every non-trivial semantic property of programs is undecidable

Static analysis $\rightarrow$ "not YES" means "Don't know"

Example: Halting problem

Let $H_{\sigma_0} \triangleq \{c \mid \llbracket c \rrbracket \sigma_0 \neq \emptyset\}$

i.e., it is the set of programs that **terminate** on the initial memory $\sigma_0 \triangleq \lambda x.0$

Halting problem (HP): Given c , does $c \in H_{\sigma_0}$?

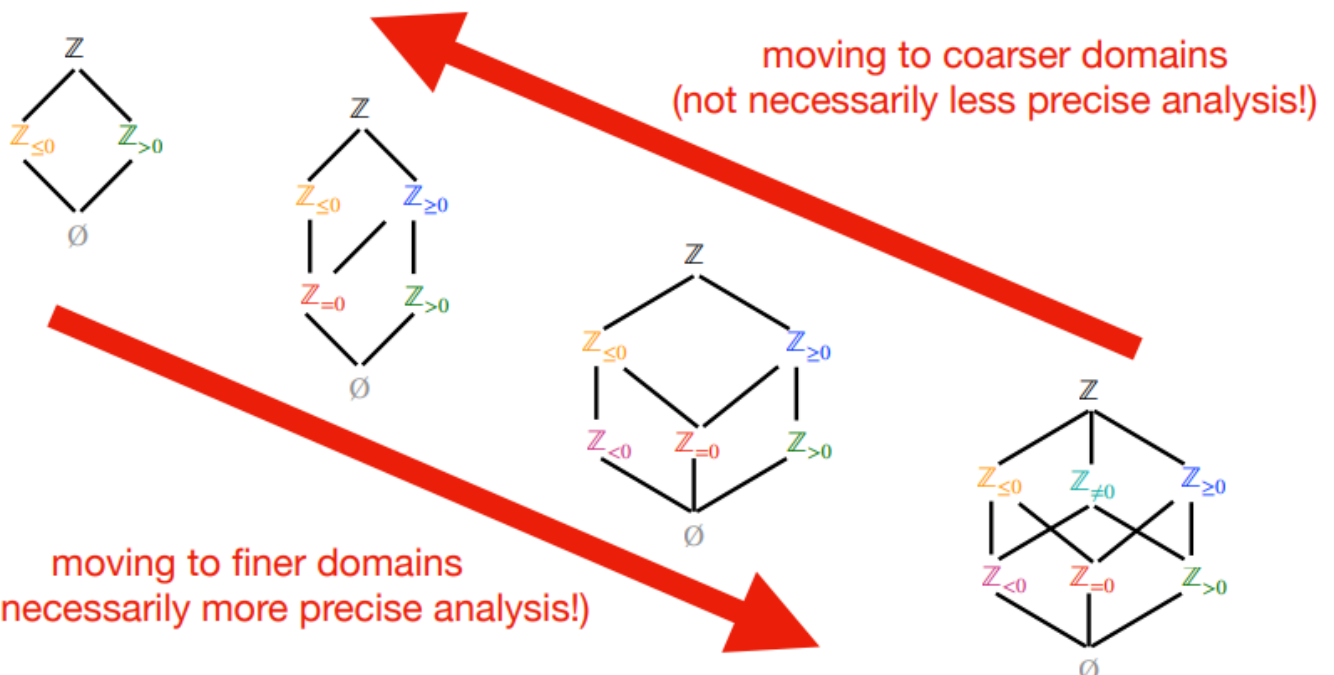
For any (Turing equivalent) programming language, HP is **undecidable**:
No analyser can always correctly answer in finite time if $c \in H_{\sigma_0}$ or not

it means "for any program c "

To design a **sound static analysis** we need to:

1. have a reference program semantics $\llbracket \cdot \rrbracket$
2. specify what and how the static analysis $\llbracket \cdot \rrbracket^\#$ computes
3. prove the correctness of $\llbracket \cdot \rrbracket^\#$ w.r.t. $\llbracket \cdot \rrbracket$
4. (efficiently) implement a static analyzer for $\llbracket \cdot \rrbracket^\#$

Which domain?

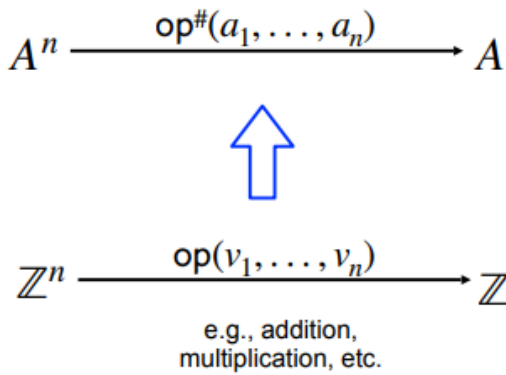


A **transfer function** defines how the abstract state of a program changes after the execution of an instruction

Soundness requirement:

if a_i is a sound abstraction of v_i for any i ,

then $a = \text{op}^\#(a_1, \dots, a_n)$ must be a **sound** abstraction of $v = \text{op}(v_1, \dots, v_n)$



Concrete values are mapped to abstract ones

Different concrete values can be mapped to the same abstract one

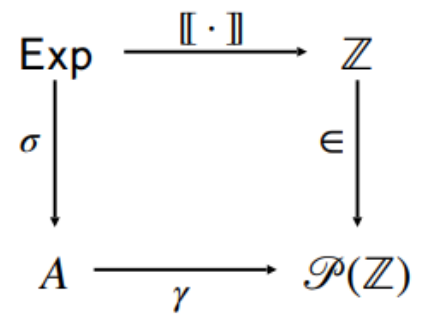
To some extent, an abstract value represents all its underlying concrete values

$\gamma : A \rightarrow \mathcal{P}(\mathbb{Z})$ called **concretization map**

$$\gamma(\mathbb{Z}_{<0}) = \{n \in \mathbb{Z} \mid n < 0\}$$

$$\gamma(\mathbb{Z}_{=0}) = \{0\}$$

$$\gamma(\mathbb{Z}_{>0}) = \{n \in \mathbb{Z} \mid n > 0\}$$



$$\gamma(\mathbb{Z}) = \{n \in \mathbb{Z}\} \text{ note that it contains each of } \gamma(\mathbb{Z}_{<0}), \gamma(\mathbb{Z}_{=0}) \text{ and } \gamma(\mathbb{Z}_{>0})$$

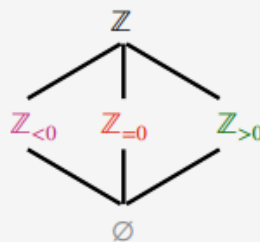
$$\gamma(\emptyset) = \emptyset \quad \text{note that it is contained in all the previous ones}$$

The **abstract domain** is a lattice whose partial order $\sqsubseteq$ defined the notion of approximation/precision

Reminder: lattice means that each subset of the abstract domain has lub and glb

The partial order must be coherent with the concretization map: $\gamma : A \rightarrow \mathcal{P}(\mathbb{Z})$

$$a_1 \sqsubseteq a_2 \implies \gamma(a_1) \subseteq \gamma(a_2)$$



The **abstraction map** $\alpha : \mathcal{P}(\mathbb{Z}) \rightarrow A$ is the counterpart of the concretization map γ

The function $\alpha : \mathcal{P}(\mathbb{Z}) \rightarrow A$ maps a set of concrete values S into the **most precise** abstract value that is a (sound) approximation of S

In our example:

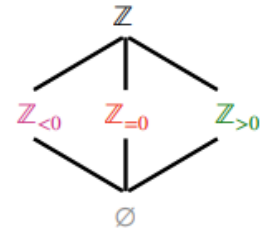
$$\alpha(\emptyset) = \emptyset$$

$$\alpha(\{0\}) = \mathbb{Z}_{=0}$$

$$\alpha(S) = \mathbb{Z}_{<0} \quad \text{if } \emptyset \subset S \subseteq \{n \in \mathbb{Z} \mid n < 0\}$$

$$\alpha(S) = \mathbb{Z}_{>0} \quad \text{if } \emptyset \subset S \subseteq \{n \in \mathbb{Z} \mid n > 0\}$$

$$\alpha(S) = \mathbb{Z} \quad \text{otherwise}$$



An **abstract interpretation** consists of:

- A concrete domain: (usually) a complete lattice C
- an abstract domain: (usually) a complete lattice A (smaller means "more precise")
- A monotone abstraction map $\alpha : C \rightarrow A$
- A monotone concretization map $\gamma : A \rightarrow C$
- The maps $\alpha : C \rightarrow A$ and $\gamma : A \rightarrow C$ form a **Galois insertion**
- **An abstract semantics $\llbracket \cdot \rrbracket^\#$ that is correct w.r.t. the concrete semantics $\llbracket \cdot \rrbracket$**

CFA for FUN

FUN: syntax

term in Term

$t ::= c$ // constant in **Const**

expression
(labelled term)
in Exp

| x // variable in **Var**

| $\text{fn } x \Rightarrow e_0$ // function definition (abstraction)

| $\text{fun } f \ x \Rightarrow e_0$ // recursive function definition

$e ::= t^\ell$

| $e_1 \ e_2$ // function application

| $\text{if } e_0 \text{ then } e_1 \text{ else } e_2$ // conditional

| $\text{let } x = e_1 \text{ in } e_2$ // local definition

| $e_1 \text{ op } e_2$ // binary operation (op in **Op**)

Label:
needed for the CFA
identify program
points (e.g., call sites)

Each label ℓ in **Lab** corresponds to a program point where CFA computes information

17

The purpose of Control Flow Analysis (CFA) is to determine which blocks may lead to which other blocks. But in modern languages, even building the CFG is not trivial.

intraprocedural analysis: $\rightarrow$ analyze a single function

interprocedural analysis: $\rightarrow$ analyze the whole program with function calls (tractable when the program has a simple structure $\rightarrow$ calls have statically known targets)

dynamic dispatch problem = variables can denote functions and which procedure/function gets invoked depends on runtime values

Solution: static analysis

$\rightarrow$ over-approximate the possible targets of each call. We cannot know the exact target of a call. Control Flow Analysis (CFA):

$\rightarrow$ For each function application (call site), compute a safe approximation

Associate for each variable and each expression, a safe over-approximation of the set of functions that they may evaluate to at runtime

$\emptyset$ -CFA is a Constraint Based Analysis

specification: describes when the analysis results are acceptable

computation: solving the constraints derived by the specification

We define our CFA in several steps:

- **Specification:** define which sets of functions are acceptable (acceptability relation)
- **Syntax-directed rules:** refine the acceptability relation to be syntax directed
- **Constraint generation:** turn the specification into a finite set of constraints
- **Constraint solving:** compute the least solution of the constraints (the analysis result)

$\hat{C}: l \mapsto$ set of function abstractions the sub-expr labelled l may evaluate to

$\hat{\rho}: l \mapsto$ set of function abstractions x may be bound to

for each $(t_1^{l_1} t_2^{l_2})^l$ determine the possible functions for t_1 from set of function abstractions t_2 may evaluate to

Θ -CFA means that no context information is considered, e.g., same function set at all call sites

The result of a CFA analysis is a pair $(\hat{C}, \hat{\rho})$:

$\hat{C} \in \widehat{\text{Cache}} : \text{Lab} \rightarrow \widehat{\text{Val}}$

$\hat{\rho} \in \widehat{\text{Env}} : \text{Var} \rightarrow \widehat{\text{Val}}$

$\hat{v} \in \widehat{\text{Val}} = \wp(\text{Term})$

Only functions,
no constants
(no DFA component):
pure CFA

$\text{fn } x \Rightarrow e_0$ abstracts the
corresponding function
in all contexts

- An **abstract value** abstracts a set of functions:, i.e., terms in the form $\text{fn } x \Rightarrow e_0$ or $\text{fun } f \ x \Rightarrow e_0$
- An **abstract cache** $\hat{C}$ associates abstract values with each labelled program point
- An **abstract environment** $\hat{\rho}$ associates abstract values with each variable

We assume that all variables and labels are distinct

Given a guess $(\hat{C}, \hat{\rho})$, we validate whether it is an **acceptable** CFA

A guess is acceptable if it safely over-approximates the runtime behavior

For instance, if at runtime

- t^ℓ evaluates to a function, then this must be duly recorded in $\hat{C}(\ell)$
- x is bound to a function at runtime, this must also be duly recorded in $\hat{\rho}(x)$

Validation

check clauses on judgements

It means that the guess $(\hat{C}, \hat{\rho})$ is
acceptable for expression e

$(\hat{C}, \hat{\rho}) \models e$

relation

$\models: \widehat{\text{Cache}} \times \widehat{\text{Env}} \times \text{Exp} \rightarrow \{\text{true}, \text{false}\}$

Rules:

- [con] always true
- [var] propagates info from vars to program points (*from $\hat{\rho}$ to $\hat{C}$*)
- [fn] functional term must be included in $\hat{C}(l)$ (no checks on body)
- [fun] functional term must be included in $\hat{C}(l)$ (no checks on body)
- [if] check on sub-exprs, propagates the values produced by sub-exprs $\hat{C}(l_i)$ to the overall expression $\hat{C}(l)$ (merging the possible values from both branches)
- [op] checks on sub-exprs (no track data values)
- [let] checks on sub-exprs, propagation of the values:
 - $\hat{C}(l_1) \subseteq \hat{\rho}(x)$ (binding)
 - $\hat{C}(l_2) \subseteq \hat{C}(l)$ (evaluation)
- [app] (simulate all possible calls):
 - checks on sub-exprs
 - for each function the can be applied:
 - check on body
 - bind argument (via $\hat{\rho}(x)$)
 - propagate results (results merged in $\hat{C}(l)$)

CFA

[con] $(\hat{C}, \hat{\rho}) \models c^\ell$ always

[var] $(\hat{C}, \hat{\rho}) \models x^\ell$ iff $\hat{\rho}(x) \subseteq \hat{C}(\ell)$

[fn] $(\hat{C}, \hat{\rho}) \models (\text{fn } x \Rightarrow e_0)^\ell$ iff $\{\text{fn } x \Rightarrow e_0\} \subseteq \hat{C}(\ell)$

[fun] $(\hat{C}, \hat{\rho}) \models (\text{fun } f \ x \Rightarrow e_0)^\ell$ iff $\{\text{fun } f \ x \Rightarrow e_0\} \subseteq \hat{C}(\ell)$

[app] $(\hat{C}, \hat{\rho}) \models (t_1^{\ell_1} \ t_2^{\ell_2})^\ell$
 iff $(\hat{C}, \hat{\rho}) \models t_1^{\ell_1} \wedge (\hat{C}, \hat{\rho}) \models t_2^{\ell_2} \wedge$
 $(\forall (\text{fn } x \Rightarrow t_0^{\ell_0}) \in \hat{C}(\ell_1) :$
 $\quad (\hat{C}, \hat{\rho}) \models t_0^{\ell_0} \wedge$
 $\quad \hat{C}(\ell_2) \subseteq \hat{\rho}(x) \wedge \hat{C}(\ell_0) \subseteq \hat{C}(\ell)) \wedge$
 $(\forall (\text{fun } f \ x \Rightarrow t_0^{\ell_0}) \in \hat{C}(\ell_1) :$
 $\quad (\hat{C}, \hat{\rho}) \models t_0^{\ell_0} \wedge$
 $\quad \hat{C}(\ell_2) \subseteq \hat{\rho}(x) \wedge \hat{C}(\ell_0) \subseteq \hat{C}(\ell) \wedge$
 $\quad \{\text{fun } f \ x \Rightarrow t_0^{\ell_0}\} \subseteq \hat{\rho}(f))$

[if] $(\hat{C}, \hat{\rho}) \models (\text{if } t_0^{\ell_0} \text{ then } t_1^{\ell_1} \text{ else } t_2^{\ell_2})^\ell$
 iff $(\hat{C}, \hat{\rho}) \models t_0^{\ell_0} \wedge$
 $(\hat{C}, \hat{\rho}) \models t_1^{\ell_1} \wedge (\hat{C}, \hat{\rho}) \models t_2^{\ell_2} \wedge$
 $\hat{C}(\ell_1) \subseteq \hat{C}(\ell) \wedge \hat{C}(\ell_2) \subseteq \hat{C}(\ell)$

[let] $(\hat{C}, \hat{\rho}) \models (\text{let } x = t_1^{\ell_1} \text{ in } t_2^{\ell_2})^\ell$
 iff $(\hat{C}, \hat{\rho}) \models t_1^{\ell_1} \wedge (\hat{C}, \hat{\rho}) \models t_2^{\ell_2} \wedge$
 $\hat{C}(\ell_1) \subseteq \hat{\rho}(x) \wedge \hat{C}(\ell_2) \subseteq \hat{C}(\ell)$

[op] $(\hat{C}, \hat{\rho}) \models (t_1^{\ell_1} \text{ op } t_2^{\ell_2})^\ell$ iff $(\hat{C}, \hat{\rho}) \models t_1^{\ell_1} \wedge (\hat{C}, \hat{\rho}) \models t_2^{\ell_2}$

In this way, we analyse the body of a function only if the function may be applied somewhere Unreachable functions are not analysed

Circular dependency $\Rightarrow$ coinduction: assume it holds and verify it is preserved

The **specification** ($\models$) is **coinductive**

loose: includes many solutions

- It defines **all acceptable solutions**
- A solution is accepted by default: it is **rejected only if it violates a constraint**

Semantically this is a **greatest fixed point**

Computation

Among all acceptable solutions, we are interested in the smallest

We compute **the least solution satisfying the specification**

Algorithmically, this is a **least fixed point**

most precise safe approximation

61

flow-insensitive: it computes a single abstract value for each variable (or expression) that over-approximates all values it may take during execution, without distinguishing between different program points or execution orders

```
let x = 0 in
  let x = 1 in
    x
```

Flow-sensitive $\rightarrow x = 1$

Flow-insensitive $\rightarrow x \in \{0, 1\}$

context-insensitive: the same function is analyzed in the same way at all call sites

```
let f = fn x => x in
  let y = f 1 in
    let z = f 2 in
      ...
```

0-CFA analyzes **f once**
merging all calls

Goal: show that acceptable CFA results are semantically correct

The analysis information is a safe description of what will happen during the evaluation of the program, i.e., a **sound over-approximation** of the runtime behaviour. For instance, if at runtime

- t^ℓ evaluates to a function, then this must be duly recorded in $\hat{C}(\ell)$
- x is bound to a function at runtime, this must also be duly recorded in $\hat{\rho}(x)$

To reason on correctness, we define a **Small Step Structural Operational Semantics** for FUN, where:

- We use **intermediate expressions** ie to handle the binding of local variables
- We choose an approach based on **explicit environments** (matching the CFA abstraction) rather than substitutions

to preserve the identity of functions (and hence abstract values) during evaluation

Subject reduction result

If the abstract ($\hat{\rho}$) and concrete environments (ρ) agree ($\rho R \hat{\rho}$):

$(\hat{C}, \hat{\rho}) \models ie$ and $\rho \vdash ie \rightarrow ie'$ then $(\hat{C}, \hat{\rho}) \models ie'$

$\rho R \hat{\rho}$ iff $dom(\rho) \subseteq dom(\hat{\rho}) \wedge \forall x \in dom(\rho) \forall t_x \forall \rho_x : (\rho(x) = \text{close } t_x \text{ in } \rho_x) \Rightarrow (t_x \in \hat{\rho}(x) \wedge \rho_x R \hat{\rho})$

A function definition will evaluate to a **closure** containing

- the syntax of the function definition, together with
- an environment mapping its free variables to their values

$v \in \mathbf{Val}$: values (evaluation results)

$\rho \in \mathbf{Env}$: environments (mapping free variables to their *values*)

$v ::= c \mid \text{close } t \text{ in } \rho$

$\rho ::= [] \mid \rho[x \mapsto v]$

Static binding:
functions evaluate to
closures capturing their
environment

Terms and expressions

$ie \in \mathbf{IExp}$: intermediate expressions

$it \in \mathbf{ITerm}$: intermediate terms

$ie ::= it^\ell$

$it ::= c \mid x \mid \text{fn } x \Rightarrow e_0 \mid \text{fun } f \ x \Rightarrow e_0$

$\mid ie_1 \ ie_2 \mid \text{if } ie_0 \text{ then } e_1 \text{ else } e_2 \mid \text{let } x = ie_1 \text{ in } e_2 \mid ie_1 \text{ op } ie_2$

$\mid \text{bind } \rho \text{ in } ie // \text{bind the environment (encoding of frames)}$

$\mid \text{close } t \text{ in } \rho // \text{closure}$

Transitions: $\rho \vdash ie \rightarrow ie'$ one step of computation of the expression ie

Rules:

- [var] The value of a variable is obtained from the environment
- [fn] This axiom constructs the appropriate closure, it restricts the environment ρ to the free variables of the abstraction (only free variables

are kept)

- [let] The expression e_2 is evaluated in the environment augmented with the new binding
- [app] This axiom describes the binding of the actual parameter to the formal parameter
- [bind] The bind construct is evaluated by repeatedly applying $[bind_1]$, The final evaluation result is get by using rule $[bind_2]$

Structural Operational Semantic I

$$\begin{array}{ll}
\text{[var]} & \rho \vdash x^\ell \rightarrow v^\ell \text{ if } x \in \text{dom}(\rho) \text{ and } v = \rho(x) \\
\text{[fn]} & \rho \vdash (\text{fn } x \Rightarrow e_0)^\ell \rightarrow (\text{close } (\text{fn } x \Rightarrow e_0) \text{ in } \rho_0)^\ell \\
& \text{where } \rho_0 = \rho \mid FV(\text{fn } x \Rightarrow e_0) \\
\text{[fun]} & \rho \vdash (\text{fun } f \ x \Rightarrow e_0)^\ell \rightarrow (\text{close } (\text{fun } f \ x \Rightarrow e_0) \text{ in } \rho_0)^\ell \\
& \text{where } \rho_0 = \rho \mid FV(\text{fun } f \ x \Rightarrow e_0) \\
\text{[app}_1\text{]} & \frac{\rho \vdash ie_1 \rightarrow ie'_1}{\rho \vdash (ie_1 \ ie_2)^\ell \rightarrow (ie'_1 \ ie_2)^\ell} \\
\text{[app}_2\text{]} & \frac{\rho \vdash ie_2 \rightarrow ie'_2}{\rho \vdash (v_1^{\ell_1} \ ie_2)^\ell \rightarrow (v_1^{\ell_1} \ ie'_2)^\ell} \\
\text{[app}_{fn}\text{]} & \rho \vdash ((\text{close } (\text{fn } x \Rightarrow e_1) \text{ in } \rho_1)^{\ell_1} \ v_2^{\ell_2})^\ell \rightarrow \\
& (\text{bind } \rho_1[x \mapsto v_2] \text{ in } e_1)^\ell \\
\text{[app}_{fun}\text{]} & \rho \vdash ((\text{close } (\text{fun } f \ x \Rightarrow e_1) \text{ in } \rho_1)^{\ell_1} \ v_2^{\ell_2})^\ell \rightarrow \\
& (\text{bind } \rho_2[x \mapsto v_2] \text{ in } e_1)^\ell \\
& \text{where } \rho_2 = \rho_1[f \mapsto \text{close } (\text{fun } f \ x \Rightarrow e_1) \text{ in } \rho_1] \\
\text{[bind}_1\text{]} & \frac{\rho_1 \vdash ie_1 \rightarrow ie'_1}{\rho \vdash (\text{bind } \rho_1 \text{ in } ie_1)^\ell \rightarrow (\text{bind } \rho_1 \text{ in } ie'_1)^\ell} \\
\text{[bind}_2\text{]} & \rho \vdash (\text{bind } \rho_1 \text{ in } v_1^{\ell_1})^\ell \rightarrow v_1^\ell
\end{array}$$

$$\begin{array}{ll}
\text{[if}_1\text{]} & \frac{\rho \vdash ie_0 \rightarrow ie'_0}{\rho \vdash (\text{if } ie_0 \text{ then } e_1 \text{ else } e_2)^\ell \rightarrow (\text{if } ie'_0 \text{ then } e_1 \text{ else } e_2)^\ell} \\
\text{[if}_2\text{]} & \rho \vdash (\text{if true}^{\ell_0} \text{ then } t_1^{\ell_1} \text{ else } t_2^{\ell_2})^\ell \rightarrow t_1^\ell \\
\text{[if}_3\text{]} & \rho \vdash (\text{if false}^{\ell_0} \text{ then } t_1^{\ell_1} \text{ else } t_2^{\ell_2})^\ell \rightarrow t_2^\ell \\
\text{[let}_1\text{]} & \frac{\rho \vdash ie_1 \rightarrow ie'_1}{\rho \vdash (\text{let } x = ie_1 \text{ in } e_2)^\ell \rightarrow (\text{let } x = ie'_1 \text{ in } e_2)^\ell} \\
\text{[let}_2\text{]} & \rho \vdash (\text{let } x = v_1^{\ell_1} \text{ in } e_2)^\ell \rightarrow (\text{bind } \rho_0[x \mapsto v] \text{ in } e_2)^\ell \\
& \text{where } \rho_0 = \rho \mid FV(e_2) \\
\text{[op}_1\text{]} & \frac{\rho \vdash ie_1 \rightarrow ie'_1}{\rho \vdash (ie_1 \ \text{op} \ ie_2)^\ell \rightarrow (ie'_1 \ \text{op} \ ie_2)^\ell} \\
\text{[op}_2\text{]} & \frac{\rho \vdash ie_2 \rightarrow ie'_2}{\rho \vdash (v_1^{\ell_1} \ \text{op} \ ie_2)^\ell \rightarrow (v_1^{\ell_1} \ \text{op} \ ie'_2)^\ell} \\
\text{[op}_3\text{]} & \rho \vdash (v_1^{\ell_1} \ \text{op} \ v_2^{\ell_2})^\ell \rightarrow v^\ell \quad \text{if } v = v_1 \ \text{op} \ v_2
\end{array}$$

$$[bind] (\hat{C}, \hat{\rho}) \models (\text{bind } \rho \text{ in } it_0^{\ell_0})^\ell \text{ iff } (\hat{C}, \hat{\rho}) \models it_0^{\ell_0} \wedge \hat{C}(\ell_0) \subseteq \hat{C}(\ell) \wedge \rho R \hat{\rho}$$

$$[close] (\hat{C}, \hat{\rho}) \models (\text{close } t_0 \text{ in } \rho)^\ell \text{ iff } \{t_0\} \subseteq \hat{C}(\ell) \wedge \rho R \hat{\rho}$$

$$\boxed{\rho R \hat{\rho} \text{ iff } \text{dom}(\rho) \subseteq \text{dom}(\hat{\rho}) \wedge \forall x \in \text{dom}(\rho) \forall t_x \forall \rho_x : (\rho(x) = \text{close } t_x \text{ in } \rho_x) \Rightarrow (t_x \in \hat{\rho}(x) \wedge \rho_x R \hat{\rho})}$$

For the other clauses just replace e with ie

- The concrete domain must be included in the abstract one (is an over-approximation)
- For all variables $x \in \text{dom}(\rho)$, if $\rho(x) = \text{close } t_x \text{ in } \rho_x$ then
 - $t_x \in \hat{\rho}(x)$: the function is predicted
 - $\rho_x R \hat{\rho}$: the captured environment ρ_x is also correct
 - This second condition is recursive: ρ_x may contain closures, whose environments must also be checked

The abstract environment over-approximates every concrete closure, including all environments they recursively capture

The definition of R can be split into two mutually recursive definitions

$$\boxed{v V (\hat{\rho}, \hat{v}) \text{ iff } \forall t \forall \rho : (v = \text{close } t \text{ in } \rho) \Rightarrow (t \in \hat{v} \wedge \rho R \hat{\rho})}$$

$$\boxed{\rho R \hat{\rho} \text{ iff } \text{dom}(\rho) \subseteq \text{dom}(\hat{\rho}) \wedge \forall x \in \text{dom}(\rho) : \rho(x) V (\hat{\rho}, \hat{\rho}(x))}$$

The two definitions of R are equivalent. We focus on $v V (\hat{\rho}, \hat{v})$

- The values must agree as well
- For all t , and for all ρ in $(v = \text{close } t \text{ in } \rho)$
 - $t_x \in \hat{v}$: the value is predicted
 - $\rho_x R \hat{\rho}$: the captured environment ρ_x is also correct
 - This check is recursive: environments may contain closures

A concrete value is a closure, so we must approximate both the function and all the environments it captures

Theorem

If $\rho R \hat{\rho}$ and $\rho \vdash ie \rightarrow ie'$ then $(\hat{C}, \hat{\rho}) \models ie$ implies $(\hat{C}, \hat{\rho}) \models ie'$

Idea: if there is a possible evaluation of the program s.t. the function at a call point evaluates to some abstraction, then this abstraction has to be in the set of possible abstractions computed by the analysis

All acceptable analysis results remain acceptable under evaluation

Proof idea

We assume that $\rho R \hat{\rho}$ and $\rho \vdash ie \rightarrow ie'$ and prove $(\hat{C}, \hat{\rho}) \models ie'$ by **induction** on the structure of the **inference tree** for $\rho \vdash ie \rightarrow ie'$

For each rule, we show CFA is preserved

Proof

We assume that $\rho R \hat{\rho}$ and $\rho \vdash ie \rightarrow ie'$ and prove $(\hat{C}, \hat{\rho}) \models ie'$ by **induction** on the structure of the **inference tree** for $\rho \vdash ie \rightarrow ie'$

The case **[var]**. Here $\rho \vdash ie \rightarrow ie'$ is $\rho \vdash x^\ell \rightarrow v^\ell$ because $x \in \text{dom}(\rho) \wedge v = \rho(x)$

[Case $v = c$]: nothing to prove.

[Case $v = \text{close } t_0 \text{ in } \rho_0$].

- From $\rho R \hat{\rho}$ we get $v \ V (\hat{\rho}, \hat{\rho}(x))$ and hence $t_0 \in \hat{\rho}(x)$ and $\rho_0 R \hat{\rho}$.
- From $(\hat{C}, \hat{\rho}) \models ie$ we get $\hat{\rho}(x) \subseteq \hat{C}(\ell)$, and hence $t_0 \in \hat{C}(\ell)$.

Since $t_0 \in \hat{C}(\ell)$ and $\rho_0 R \hat{\rho}$ we have established $(\hat{C}, \hat{\rho}) \models ie'$.

[var] $(\hat{C}, \hat{\rho}) \models x^\ell$ iff $\hat{\rho}(x) \subseteq \hat{C}(\ell)$

$v \ V (\hat{\rho}, \hat{v})$ iff $\forall t \ \forall \rho : (v = \text{close } t \text{ in } \rho) \Rightarrow (t \in \hat{v} \wedge \rho R \hat{\rho})$

the value is already predicted by the analysis

94

Proof

The case **[fn]**. Here $\rho \vdash ie \rightarrow ie'$ is $\rho \vdash (\text{fn } x \Rightarrow e_0)^\ell \rightarrow (\text{close } (\text{fn } x \Rightarrow e_0) \text{ in } \rho_0)^\ell$ where $\rho_0 = \rho \mid FV(\text{fn } x \Rightarrow e_0)$

- From $(\hat{C}, \hat{\rho}) \models ie$ we get $\{\text{fn } x \Rightarrow e_0\} \subseteq \hat{C}(\ell)$.
- From $\rho R \hat{\rho}$ we get we immediately get $\rho_0 R \hat{\rho}$ and then we can establish $(\hat{C}, \hat{\rho}) \models ie'$.

[fn] $(\hat{C}, \hat{\rho}) \models (\text{fn } x \Rightarrow e_0)^\ell$ iff $\{\text{fn } x \Rightarrow e_0\} \subseteq \hat{C}(\ell)$

[close] $(\hat{C}, \hat{\rho}) \models (\text{close } t_0 \text{ in } \rho)^\ell$ iff $\{t_0\} \subseteq \hat{C}(\ell) \wedge \rho R \hat{\rho}$

the abstraction is already recorded in $\hat{C}(\ell)$

Proof

The case **[app₁]**. Here $\rho \vdash ie \rightarrow ie'$ is $\rho \vdash (ie_1 ie_2)^\ell \rightarrow (ie'_1 ie_2)^\ell$ because $\rho \vdash ie_1 \rightarrow ie'_1$

- From the induction hypothesis applied to $(\hat{C}, \hat{\rho}) \models ie_1$, $\rho R \hat{\rho}$ and $\rho \vdash ie_1 \rightarrow ie'_1$, we get $(\hat{C}, \hat{\rho}) \models ie'$.

This suffices to establish $(\hat{C}, \hat{\rho}) \models (ie'_1 ie_2)^\ell$.

Proof

The case **[app_{fn}]**.

Here $\rho \vdash ie \rightarrow ie'$ is $\rho \vdash (\text{close } (\text{fn } x \Rightarrow t_0^{\ell_0}) \text{ in } \rho_1)^{\ell_1} v_2^{\ell_2} \rightarrow (\text{bind } \rho_1[x \mapsto v_2] \text{ in } t_0^{\ell_0})^\ell$

- From $(\hat{C}, \hat{\rho}) \models ie$ we have $(\hat{C}, \hat{\rho}) \models (\text{close } (\text{fn } x \Rightarrow t_0^{\ell_0}) \text{ in } \rho_1)^{\ell_1}$, which yields

$\{\text{fn } x \Rightarrow t_0^{\ell_0}\} \subseteq \hat{C}(\ell_1)$ and $\rho R \hat{\rho}$.

- Further, we have $(\hat{C}, \hat{\rho}) \models v_2^{\ell_2}$ $[\text{close}] (\hat{C}, \hat{\rho}) \models (\text{close } t_0 \text{ in } \rho)^\ell \text{ iff } \{t_0\} \subseteq \hat{C}(\ell) \wedge \rho R \hat{\rho}$
 $[\text{bind}] (\hat{C}, \hat{\rho}) \models (\text{bind } \rho \text{ in } t_0^{\ell_0})^\ell \text{ iff } (\hat{C}, \hat{\rho}) \models t_0^{\ell_0} \wedge \hat{C}(\ell_0) \subseteq \hat{C}(\ell) \wedge \rho R \hat{\rho}$

[case $v_2 = c$]: immediate.

[case $v_2 = \text{close } t_2 \text{ in } \rho_2$]: it follows immediately from $(\hat{C}, \hat{\rho}) \models v_2^{\ell_2}$.

Finally, the first universally quantified formula of the definition of $(\hat{C}, \hat{\rho}) \models ie$ gives

$(\hat{C}, \hat{\rho}) \models t_0^{\ell_0}$, $\hat{C}(\ell_2) \subseteq \hat{C}(\ell)$ and $(\rho_1[x \mapsto v_2]) R \hat{\rho}$ leading to establish $(\hat{C}, \hat{\rho}) \models ie'$.

we simulate all possible function calls

Proof

The case **[bind₂]**. Here $\rho \vdash ie \rightarrow ie'$ is $\rho \vdash (\text{bind } \rho_1 \text{ in } v_1^{\ell_1})^\ell \rightarrow v_1^\ell$

- From $(\hat{C}, \hat{\rho}) \models ie$ we have $(\hat{C}, \hat{\rho}) \models v_1^{\ell_1}$ and $\hat{C}(\ell_1) \subseteq \hat{C}(\ell)$

The desired $(\hat{C}, \hat{\rho}) \models v_1^\ell$ follows from **Fact**.

This completes the proof. ■

$[\text{bind}] (\hat{C}, \hat{\rho}) \models (\text{bind } \rho \text{ in } t_0^{\ell_0})^\ell \text{ iff } (\hat{C}, \hat{\rho}) \models t_0^{\ell_0} \wedge \hat{C}(\ell_0) \subseteq \hat{C}(\ell) \wedge \rho R \hat{\rho}$

CFA predictions are stable under evaluation

Fewer possible flows $\Rightarrow$ more precise analysis

Which **partial order** to compare different CFAs?

The set of proposed solutions can be partially ordered by setting

pointwise inclusion

$$(\hat{C}_1, \hat{\rho}_1) \sqsubseteq (\hat{C}_2, \hat{\rho}_2) \text{ iff } \begin{cases} \forall \ell \in \mathbf{Lab} : \hat{C}_1(\ell) \subseteq \hat{C}_2(\ell) \\ \forall x \in \mathbf{Var} : \hat{\rho}_1(x) \subseteq \hat{\rho}_2(x) \end{cases}$$

smaller = more precise

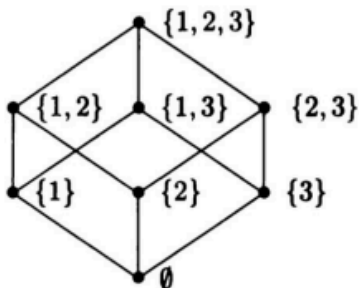
This suffices for making the set of proposed solutions into a **complete lattice**

Moore family

model intersection property

A **Moore family** is a subset Y of a complete lattice $L = (L, \sqsubseteq)$ that is closed under greatest lower bounds: $(\sqcap Y') \in Y$ for all $Y' \subseteq Y$

- It follows that a Moore family always contains a **least element** $\sqcap Y$, and a **greatest element** $\sqcap \emptyset$, which equals the greatest element $\top$ from L
- A Moore family is never empty



The complete lattice $(\wp(\{1,2,3\}), \subseteq)$

The subsets:

- $\{\{2\}, \{1,2\}, \{2,3\}, \{1,2,3\}\}$ and $\{\emptyset, \{1,2,3\}\}$ are both Moore families, while
- $\{\{1\}, \{2\}\}$ and $\{\emptyset, \{1\}, \{2\}, \{1,2\}\}$ are not

$\{1,2\} \cap \{2,3\} = \{2\} \in S \dots$
 $\sqcap \emptyset = \{1,2,3\} \in S$

$\emptyset \cap \{1,2,3\} = \emptyset \in S$ and
 $\sqcap \emptyset = \{1,2,3\} \in S$

$\{1\} \cap \{2\} = \emptyset \notin S$

$\sqcap \emptyset = \{1,2,3\} \notin S$

106

The set of acceptable analyses is a complete lattice closed under intersection and it forms a **Moore family**

In particular, the set of acceptable analyses has a **least element**

Theorem

For all $ie \in \mathbf{IExp}$ the set $\{(\hat{C}, \hat{\rho}) \mid (\hat{C}, \hat{\rho}) \models ie\}$ is a **Moore family**

Proof idea

By **coinduction** on ie : we assume that $\forall i \in I : (\hat{C}_i, \hat{\rho}_i) \models ie$ and that $\sqcap_i (\hat{C}_i, \hat{\rho}_i) \models ie$

The least CFA is obtained by intersection

107

Theorem

For all $ie \in \mathbf{IExp}$ the set $\{(\hat{C}, \hat{\rho}) \mid (\hat{C}, \hat{\rho}) \models ie\}$ is a Moore family

taking the intersection over an empty set yields a valid analysis

Corollaries:

- all intermediate expressions admit a CFA: let $Y' = \emptyset$, then $(\sqcap Y') \in \{(\hat{C}, \hat{\rho}) \mid (\hat{C}, \hat{\rho}) \models ie\}$. So there is at least an analysis of ie
- all intermediate expressions have a least CFA: let $Y' = \{(\hat{C}, \hat{\rho}) \mid (\hat{C}, \hat{\rho}) \models ie\}$, then $(\sqcap Y') \in \{(\hat{C}, \hat{\rho}) \mid (\hat{C}, \hat{\rho}) \models ie\}$. So it will also be an analysis of ie . Clearly $(\sqcap Y') \sqsubseteq (\hat{C}, \hat{\rho})$ for all other analyses $(\hat{C}, \hat{\rho})$ of ie so it is the least analysis result.

If acceptability were defined **inductively** with an acceptability relation $\models'$ then there exists an expression e_* such that acceptable analyses are not closed under intersection

Idea:

- If there are circular dependencies, it cannot resolve anything and may therefore return an empty solution
- Inductive definitions are too strict: they exclude valid over-approximations

If acceptability were defined **inductively** with an acceptability relation $\models'$ then there exists an expression e_* such that acceptable analyses are not closed under intersection

Proposition

There exists $e_* \in \mathbf{Exp}$ such that the set $\{(\hat{C}, \hat{\rho}) \mid (\hat{C}, \hat{\rho}) \models' e_*\}$ is **not** a Moore family

Proof idea: Use a very simple but “tricky” program and examine its CFA solutions:

- with the greatest fixed point $\rightarrow$ everything is fine
- with the least fixed point $\rightarrow$ we get a “bad” set ($\emptyset$)

We conclude it is not a Moore family

Least solutions (w.r.t. $\sqsubseteq$) exist, but how can we compute them?

Alternative:

1. derive a finite **set of constraints** from the program, such as the ones of the form $lhs \sqsubseteq rhs$,
2. take the **least solution** to this system of constraints (using fixpoint computation)

First, we will introduce a more computationally oriented specification: $\models_s$

Syntax directed specification

Syntax directed CFA

Look for a more computationally oriented specification

Goal: analyze each function body *at most once* rather than each time the function could be applied

Idea: one way to obtain this is to analyse each function body *exactly once*, at its definition

This is done in the so-called **syntax directed O-CFA analysis**:

- acceptability relation: $\models_s$
- judgments: $(\hat{C}, \hat{\rho}) \models_s e_*$

Since semantic correctness was dealt with for the previous CFA there is no longer any need to consider intermediate expressions

If we want to analyze a given expression e_* , we reformulate the specification of $(\hat{C}, \hat{\rho}) \models e_*$ into the more computationally oriented specification $(\hat{C}, \hat{\rho}) \models_s e_*$ such that:

$$(\hat{C}, \hat{\rho}) \models_s e_* \Rightarrow (\hat{C}, \hat{\rho}) \models e_* \text{ (if restricted to program terms)}$$

i.e., $\models_s$ is a safe approximation of $\models$: all solutions accepted by $\models_s$ are also valid for $\models$

We need to focus on solutions satisfying some additional properties (e.g., that only program fragments of e_* appear in the range of $\hat{C}$ and $\hat{\rho}$)

Rules:

- [fn] Function body e_0 is now analyzed at definition time, even if the function is never applied

- [fun] Function body e_0 is now analyzed at definition time, even if the function is never applied, checks on the recursive binding for [fun] is included: self-binding is enforced immediately
- [app] The body constraints have already been generated by [fn]
- All other rules are unchanged: just replace $\models$ with $\models_s$

Syntax directed CFA

$$\begin{aligned}
[\text{con}] \quad & (\widehat{C}, \widehat{\rho}) \models_s c^\ell \text{ always} \\
[\text{var}] \quad & (\widehat{C}, \widehat{\rho}) \models_s x^\ell \text{ iff } \widehat{\rho}(x) \subseteq \widehat{C}(\ell) \\
[\text{fn}] \quad & (\widehat{C}, \widehat{\rho}) \models_s (\text{fn } x \Rightarrow e_0)^\ell \\
& \text{iff } \{ \text{fn } x \Rightarrow e_0 \} \subseteq \widehat{C}(\ell) \wedge \\
& (\widehat{C}, \widehat{\rho}) \models_s e_0 \\
[\text{fun}] \quad & (\widehat{C}, \widehat{\rho}) \models_s (\text{fun } f \ x \Rightarrow e_0)^\ell \\
& \text{iff } \{ \text{fun } f \ x \Rightarrow e_0 \} \subseteq \widehat{C}(\ell) \wedge \\
& (\widehat{C}, \widehat{\rho}) \models_s e_0 \wedge \{ \text{fun } f \ x \Rightarrow e_0 \} \subseteq \widehat{\rho}(f) \\
[\text{app}] \quad & (\widehat{C}, \widehat{\rho}) \models_s (t_1^{\ell_1} t_2^{\ell_2})^\ell \\
& \text{iff } (\widehat{C}, \widehat{\rho}) \models_s t_1^{\ell_1} \wedge (\widehat{C}, \widehat{\rho}) \models_s t_2^{\ell_2} \wedge \\
& (\forall (\text{fn } x \Rightarrow t_0^{\ell_0}) \in \widehat{C}(\ell_1) : \\
& \quad \widehat{C}(\ell_2) \subseteq \widehat{\rho}(x) \wedge \widehat{C}(\ell_0) \subseteq \widehat{C}(\ell)) \wedge \\
& (\forall (\text{fun } f \ x \Rightarrow t_0^{\ell_0}) \in \widehat{C}(\ell_1) : \\
& \quad \widehat{C}(\ell_2) \subseteq \widehat{\rho}(x) \wedge \widehat{C}(\ell_0) \subseteq \widehat{C}(\ell)) \\
[\text{if}] \quad & (\widehat{C}, \widehat{\rho}) \models_s (\text{if } t_0^{\ell_0} \text{ then } t_1^{\ell_1} \text{ else } t_2^{\ell_2})^\ell \\
& \text{iff } (\widehat{C}, \widehat{\rho}) \models_s t_0^{\ell_0} \wedge \\
& (\widehat{C}, \widehat{\rho}) \models_s t_1^{\ell_1} \wedge (\widehat{C}, \widehat{\rho}) \models_s t_2^{\ell_2} \wedge \\
& \widehat{C}(\ell_1) \subseteq \widehat{C}(\ell) \wedge \widehat{C}(\ell_2) \subseteq \widehat{C}(\ell) \\
[\text{let}] \quad & (\widehat{C}, \widehat{\rho}) \models_s (\text{let } x = t_1^{\ell_1} \text{ in } t_2^{\ell_2})^\ell \\
& \text{iff } (\widehat{C}, \widehat{\rho}) \models_s t_1^{\ell_1} \wedge (\widehat{C}, \widehat{\rho}) \models_s t_2^{\ell_2} \wedge \\
& \widehat{C}(\ell_1) \subseteq \widehat{\rho}(x) \wedge \widehat{C}(\ell_2) \subseteq \widehat{C}(\ell) \\
[\text{op}] \quad & (\widehat{C}, \widehat{\rho}) \models_s (t_1^{\ell_1} \text{ op } t_2^{\ell_2})^\ell \text{ iff } (\widehat{C}, \widehat{\rho}) \models_s t_1^{\ell_1} \wedge (\widehat{C}, \widehat{\rho}) \models_s t_2^{\ell_2}
\end{aligned}$$

No circular dependency $\Rightarrow$ no need of coinduction

We can take $\models_s$ as the **largest relation satisfying the specification**

However, given the syntax directed nature of the specification (the relation is fully determined by the structure of expressions), there is in fact **only one** relation that satisfies the specification

unique

Hence it would be technically correct, but intuitively misleading, to claim that we take the **least relation that satisfies the specification**, because it suggests a choice among many relations

Before: many acceptable relations $\rightarrow$ take the least

Now: only one relation $\rightarrow$ no minimisation needed

Whether or not $\models_s$ is defined inductively or conductively, the same relation is defined. This is typical of syntax-directed definitions

The specification of the analysis uses **potentially infinite value spaces** although this is not really necessary

We can easily restrict ourselves to entities occurring in the original expression and this forms the basis for relating the results of the analysis of syntax directed CFA to those of the previous CFA

We obtain finite sets

Lab_{*} ⊆ **Lab**: finite set of labels occurring in e_* ,

Var_{*} ⊆ **Var**: finite set of variables occurring in e_*

Term_{*} ⊆ **Term**: finite set of subterms occurring in e_*

our universe

We introduce the solution $(\hat{C}^\top, \hat{\rho}^\top)$

the largest possible solution within the bounds of the expression

$$\hat{C}^\top(\ell) = \begin{cases} \emptyset & \text{if } \ell \notin \mathbf{Lab}_* \\ \mathbf{Term}_* & \text{if } \ell \in \mathbf{Lab}_* \end{cases} \quad \hat{\rho}^\top(x) = \begin{cases} \emptyset & \text{if } x \notin \mathbf{Var}_* \\ \mathbf{Term}_* & \text{if } x \in \mathbf{Var}_* \end{cases}$$

$(\hat{C}, \hat{\rho}) \sqsubseteq (\hat{C}^\top, \hat{\rho}^\top)$: intuitively, $(\hat{C}, \hat{\rho})$ is concerned **only with subterms occurring in the expression e_***

We can directly reformulate the CFA in terms of restricted sets

$$(\hat{C}, \hat{\rho}) \in \widehat{\mathbf{Cache}}_* \times \widehat{\mathbf{Env}}_*$$

$$\widehat{\mathbf{Cache}}_* : \mathbf{Lab}_* \rightarrow \widehat{\mathbf{Val}}_*$$

$$\widehat{\mathbf{Env}}_* : \mathbf{Var}_* \rightarrow \widehat{\mathbf{Val}}_*$$

$$\widehat{\mathbf{Val}}_* = \wp(\mathbf{Term}_*)$$

Two views of the analysis

check when needed

- Original CFA $\models$: checks correctness during the simulation of the evaluation, i.e., when analysing function application
- Syntax-directed CFA $\models_s$: analyzes all subterms upfront, so when we reach a call site, everything has already been analysed

As a consequence:

check in advance: nothing missing at call time

$$(\hat{C}, \hat{\rho}) \models_s e_* \Rightarrow (\hat{C}, \hat{\rho}) \models e_* \text{ (if restricted to program terms)}$$

- We can prove that all the solutions to $\models_s e_*$ that are "less than" $(\hat{C}^T, \hat{\rho}^T)$ are also solutions to $\models e_*$
- Syntax-directed CFA is a **sound approximation** of the original CFA when restricted to program terms

boundedness condition

Proposition

If $(\hat{C}, \hat{\rho}) \models_s e_*$ and $(\hat{C}, \hat{\rho}) \sqsubseteq (\hat{C}^T, \hat{\rho}^T)$ then $(\hat{C}, \hat{\rho}) \models e_*$

Idea: syntax-directed CFA already enforces what the original CFA requires

Proof idea:

- Restrict to program terms and use coinduction
- Show that every syntax-directed clause implies the original one
- Therefore, the solution also holds for the original CFA

Proof sketch:

1. Restrict to program terms $\mathbf{Exp}_*$, the set of all sub-expressions occurring in e_*
2. By syntax-directedness: $(\hat{C}, \hat{\rho}) \models_s e_* \Rightarrow \forall e \in \mathbf{Exp}_* : (\hat{C}, \hat{\rho}) \models_s e$
3. By coinduction, show that every syntax-directed clause (its *rhs*) implies the original one. Intuitively:
 - [con], [var], [if], [let], [op]: immediate, since identical.
 - [fn], [fun]: syntax-directed rules are stronger than the original ones.
 - [app]: $\models_s$ already enforces all constraints required by $\models$ (function bodies are already analyzed by (2)).

So, $\models_s$ captures all semantically required flows on reachable terms

EXISTENCE

Theorem

$\{(\hat{C}, \hat{\rho}) \in (\widehat{\mathbf{Cache}}_* \times \widehat{\mathbf{Env}}_*) \mid (\hat{C}, \hat{\rho}) \models_s e\}$ is a **Moore family**

Corollaries:

- Each expression e_* admits a CFA that is "less than" $(\hat{C}^T, \hat{\rho}^T)$
- Each expression e_* has a "least" CFA that is "less than" $(\hat{C}^T, \hat{\rho}^T)$

This means that the properties obtained for the original analysis holds for the syntax-directed one

Proof:

$(\hat{C}_*^T, \hat{\rho}_*^T)$ is the greatest element of $\widehat{\mathbf{Cache}_*} \times \widehat{\mathbf{Env}_*}$. It is immediate to show that:

(a) $(\hat{C}_*^T, \hat{\rho}_*^T) \models_s e$

(b) $(\hat{C}_1, \hat{\rho}_1) \models_s e$ and $(\hat{C}_2, \hat{\rho}_2) \models_s e$ then $((\hat{C}_1, \hat{\rho}_1) \cap (\hat{C}_2, \hat{\rho}_2)) \models_s e$

for all **subexpressions** e of e_* , by means of *structural induction* on e . Hence:

(a) and (b) also for $e = e_*$.

Next consider some

$$Y \subseteq \{(\hat{C}, \hat{\rho}) \in (\widehat{\mathbf{Cache}_*} \times \widehat{\mathbf{Env}_*}) \mid (\hat{C}, \hat{\rho}) \models_s e\}$$

and note that one can write

$Y = \{(\hat{C}_i, \hat{\rho}_i) \mid i \in \{1, \dots, n\}\}$ for some $n \geq 0$ since $(\widehat{\mathbf{Cache}_*} \times \widehat{\mathbf{Env}_*})$ is finite. So

$$\sqcap Y \in \{(\hat{C}_i, \hat{\rho}_i) \mid i \in \{1, \dots, n\}\}$$

follows from (a) and (b), because

$$\sqcap Y = (\hat{C}_*^T, \hat{\rho}_*^T) \sqcap (\hat{C}_1, \hat{\rho}_1) \sqcap \dots \sqcap (\hat{C}_n, \hat{\rho}_n)$$

[app] guard operand analysis

- The syntax directed analysis analyzes each subexpression of e_* “exactly once”, rather than “at most once” as really called for
- This may introduce constraints from code that is never executed
- How to amend this?

Goal:

- only expressions that are reachable will actually be analysed, and
- the same expression is analysed “at most once”

Idea:

1. modify the analysis to track the reachability of subexpressions explicitly
2. modify it such that subexpressions are analysed “at most once”

Reachability CFA

How?

By introducing a new abstract reachability component

$$\hat{R} \in \mathbf{Reach} : \mathbf{Lab} \rightarrow \wp(\{\mathbf{on}\})$$

and modifying the syntax directed analysis

$$(\hat{C}, \hat{\rho}, \hat{R}) \models'_s e$$

$\{\mathbf{on}\} \subseteq \hat{R}(\ell)$ means: the expression at label ℓ is reachable

- $\hat{R}(\ell) = \emptyset$: the program point ℓ is not reachable (t_ℓ must be analysed)
- $\{\mathbf{on}\} \subseteq \hat{R}(\ell_0)$: the program point ℓ is reachable
- $(\text{fn } x \Rightarrow t_0^{\ell_0})^\ell$ has $\{\mathbf{on}\} \subseteq \hat{R}(\ell_0)$ iff the function is indeed applied somewhere
- A function body is analyzed only if it is reachable: the "recursive call" $(\hat{C}, \hat{\rho}, \hat{R}) \models'_s t_0^{\ell_0}$ is performed if and only if $\{\mathbf{on}\} \subseteq \hat{R}(\ell_0)$
- Hence, function bodies are analyzed "at most once"

Rules:

- [fn] Function body e_0 is analysed at definition time, but only if the function is applied somewhere
- [var] x is analyzed only if reachable
- [app] If the function possibly occurs in an application, then this means it is reachable
- All other rules are unchanged: just replace $\models_s$ with $\models'_s$

Reachability

Proposition

If $(\hat{C}, \hat{\rho}, \hat{R}) \models'_s t_*^{\ell_*}$ and $\{\mathbf{on}\} \subseteq \hat{R}(\ell_*)$ and $(\hat{C}, \hat{\rho}) \sqsubseteq (\hat{C}_*, \hat{\rho}_*)$ then $(\hat{C}, \hat{\rho}) \models t_*^{\ell_*}$

Proof sketch:

1. Restrict to program terms, $\mathbf{Exp}_*$, the set of all reachable subexpressions occurring in e_*
2. By syntax-directedness: $(\hat{C}, \hat{\rho}, \hat{R}) \models'_s t_*^{\ell_*} \Rightarrow \forall e \in \mathbf{Exp}_* : (\{\mathbf{on}\} \subseteq \hat{R}(\ell_*) \Rightarrow (\hat{C}, \hat{\rho}) \models_s e)$
3. By coinduction, show that every syntax-directed clause (its *rhs*) implies the original one.

Intuitively:

- [con], [var], [if], [let], [op]: immediate, since identical.
- [fn], [fun]: syntax-directed rules with the additional constraint on $\hat{R}$ are equivalent to the original ones, on reachable terms.
- [app]: $\models_s$ enforces all constraints required by $\models$ on reachable terms, and propagates reachability information $\hat{R}$.

So, $\models'_s$ captures all semantically required flows on reachable terms, provided that $\hat{R}$ correctly over-approximates reachability

Does the converse hold?

1. $(\hat{C}, \hat{\rho}) \models e_*$ then $(\hat{C}, \hat{\rho}, \hat{R}) \models'_s e_*$ for some $\hat{R}$

2. $(\hat{C}, \hat{\rho}) \models e_*$ and $(\hat{C}, \hat{\rho}) \sqsubseteq (\hat{C}_*^T, \hat{\rho}_*^T)$ then $(\hat{C}, \hat{\rho}, \hat{R}) \models'_s e_*$ for some $\hat{R}$

1. does not hold:

- the original CFA can introduce and treat consistently “spurious” abstract values, without losing acceptability, since it provides an over-approximation
- the reachability CFA, instead, even forcing to **on** the “spurious” value, cannot analyze the abstraction (and its body), since it does not occur in e_*

2. $(\hat{C}, \hat{\rho}) \models e_*$ and $(\hat{C}, \hat{\rho}) \sqsubseteq (\hat{C}_*^T, \hat{\rho}_*^T)$ then $(\hat{C}, \hat{\rho}, \hat{R}) \models'_s e_*$ for some $\hat{R}$

Does the converse hold?

1. $(\hat{C}, \hat{\rho}) \models e_*$ then $(\hat{C}, \hat{\rho}, \hat{R}) \models'_s e_*$ for some $\hat{R}$

2. $(\hat{C}, \hat{\rho}) \models e_*$ and $(\hat{C}, \hat{\rho}) \sqsubseteq (\hat{C}_*^T, \hat{\rho}_*^T)$ then $(\hat{C}, \hat{\rho}, \hat{R}) \models'_s e_*$ for some $\hat{R}$

2. Since $(\hat{C}, \hat{\rho}) \sqsubseteq (\hat{C}_*^T, \hat{\rho}_*^T)$ then no “spurious” value can be added. In this case the bodies analyzed in the two analyses are the same, provided a suitable $\hat{R}$ is provided

Constraint Based 0-CFA

- We are now ready to compute the least solution $(\hat{C}, \hat{\rho})$ such that
- We have learnt that constraint generation is a syntax driven approach
- We turn semantic rules into constraints
- To this aim, we first construct a finite set $C_*[[e_*]]$ of *constraints* and *conditional constraints* ls in the form:

$$lhs \subseteq rhs$$
$$\{t\} \subseteq rhs' \Rightarrow lhs \subseteq rhs$$

if t may flow to rhs' , then
lhs must flow to rhs

where:

- $lhs ::= \hat{C}(\ell) \mid \hat{\rho}(x), \{t\}$ where t is a function abstraction
- $rhs ::= \hat{C}(\ell) \mid \hat{\rho}(x)$

- Expand each clause defining $(\hat{C}, \hat{r}) \models_s e_*$ into a **finite set of constraints** and
- Collect all constraints into $C_*[[e_*]]$

logical formula

set of syntactic
entities

Idea:

Occurrences of $\hat{C}$ are changed into C and
occurrences of $\hat{r}$ are changed into r

C and r are the **constraint variables** representing the unknown solution

The function C_* given an expression e_* returns a set of constraints $C_*[[e_*]]$
We generate constraints from the syntax-directed rules

Rules:

- [con] $\emptyset$ constants generate no constraints
- [var] Variables generate a constraint reflecting a flow from $r(x)$ to $C(\ell)$
- [fn] generate a flow constraint, and, recursively, constraints from the body
- [fun] generate a flow constraint, and, recursively, constraints from the body (recursive binding: function flows to f)
- [app] Application generates recursively constraints from its terms plus the body conditional constraints

Constraint based CFA

$$[con] \quad C_*[c^\ell] = \emptyset$$

$$[var] \quad C_*[x^\ell] = \{r(x) \subseteq C(\ell)\}$$

$$[fn] \quad C_*[(\text{fn } x \Rightarrow e_0)^\ell] = \{\{\text{fn } x \Rightarrow e_0\} \subseteq C(\ell)\} \cup C_*[e_0]$$

$$[fun] \quad C_*[(\text{fun } f \text{ } x \Rightarrow e_0)^\ell] = \{\{\text{fun } f \text{ } x \Rightarrow e_0\} \subseteq C(\ell)\} \cup C_*[e_0] \cup \{\{\text{fun } f \text{ } x \Rightarrow e_0\} \subseteq r(f)\}$$

$$[app] \quad C_*[(t_1^{\ell_1} \ t_2^{\ell_2})^\ell] = C_*[t_1^{\ell_1}] \cup C_*[t_2^{\ell_2}] \cup \{\{t\} \subseteq C(\ell_1) \Rightarrow C(\ell_2) \subseteq r(x) \mid t = (\text{fn } x \Rightarrow t_0^{\ell_0}) \in \text{Term}_*\} \cup \{\{t\} \subseteq C(\ell_1) \Rightarrow C(\ell_0) \subseteq C(\ell) \mid t = (\text{fn } x \Rightarrow t_0^{\ell_0}) \in \text{Term}_*\} \cup \{\{t\} \subseteq C(\ell_1) \Rightarrow C(\ell_2) \subseteq r(x) \mid t = (\text{fun } f \text{ } x \Rightarrow t_0^{\ell_0}) \in \text{Term}_*\} \cup \{\{t\} \subseteq C(\ell_1) \Rightarrow C(\ell_0) \subseteq C(\ell) \mid t = (\text{fun } f \text{ } x \Rightarrow t_0^{\ell_0}) \in \text{Term}_*\}$$

$$[if] \quad C_*[(\text{if } t_0^{\ell_0} \text{ then } t_1^{\ell_1} \text{ else } t_2^{\ell_2})^\ell] = C_*[t_0^{\ell_0}] \cup C_*[t_1^{\ell_1}] \cup C_*[t_2^{\ell_2}] \cup \{C(\ell_1) \subseteq C(\ell)\} \cup \{C(\ell_2) \subseteq C(\ell)\}$$

$$[let] \quad C_*[(\text{let } x = t_1^{\ell_1} \text{ in } t_2^{\ell_2})^\ell] = C_*[t_1^{\ell_1}] \cup C_*[t_2^{\ell_2}] \cup \{C(\ell_1) \subseteq r(x)\} \cup \{C(\ell_2) \subseteq C(\ell)\}$$

$$[op] \quad C_*[(t_1^{\ell_1} \text{ op } t_2^{\ell_2})^\ell] = C_*[t_1^{\ell_1}] \cup C_*[t_2^{\ell_2}]$$

The function C^* uses the set of terms $\mathbf{Term}_*$. Let n be the size of the expression e_*

Naive estimate:

- $O(n^2)$ constraints
- $O(n^4)$ conditional constraints

the number of sub-terms

Refined estimate:

Key observation: Every construct, except applications, gives us only one constraint. Applications may give us $O(n)$ constraints:

- $O(n)$ constraints
- $O(n^2)$ conditional constraints

Each of the $O(n)$ sub-terms only generate $O(1)$ constraints and $O(n)$ conditional constraints

The analysis remains **polynomial** despite higher-order functions

To give meaning to the syntax, we can translate syntactic entities to sets of terms:

$$(\hat{C}, \hat{\rho}) \llbracket C(\ell) \rrbracket = \hat{C}(\ell)$$

$$(\hat{C}, \hat{\rho}) \llbracket r(x) \rrbracket = \hat{\rho}(x)$$

$$(\hat{C}, \hat{\rho}) \llbracket \{t\} \rrbracket = \{t\}$$

$$(\hat{C}, \hat{\rho}) \llbracket \{t\} \subseteq rhs' \Rightarrow lhs \rrbracket = \begin{cases} (\hat{C}, \hat{\rho}) \llbracket lhs \rrbracket & \text{if } \{t\} \in (\hat{C}, \hat{\rho}) \llbracket rhs' \rrbracket \\ \emptyset & \text{otherwise} \end{cases}$$

Satisfaction relation for constraints:

$$(\hat{C}, \hat{\rho}) \llbracket C(\ell) \rrbracket \models_c (ls \subseteq rhs) \text{ iff } (\hat{C}, \hat{\rho}) \llbracket ls \rrbracket \subseteq (\hat{C}, \hat{\rho}) \llbracket rhs \rrbracket$$

$$(\hat{C}, \hat{\rho}) \llbracket C(\ell) \rrbracket \models_c C \text{ iff } \forall (ls \subseteq rhs) \in C : (\hat{C}, \hat{\rho}) \models_c (ls \subseteq rhs)$$

set of constraints

165

- All solutions to the set $C_* \llbracket e_* \rrbracket$ of constraints also satisfy the syntax directed specification of the CFA and vice versa (when restricted to program terms)
- Thus computing the least solution of the constraints $C_* \llbracket e_* \rrbracket$ gives exactly the least CFA solution $(\hat{C}, \hat{\rho})$ to $(\hat{C}, \hat{\rho}) \models_s e_*$

Proposition

If $(\hat{C}, \hat{\rho}) \subseteq (\hat{C}^\top, \hat{\rho}^\top)$

then $(\hat{C}, \hat{\rho}) \models_s e_*$ iff $(\hat{C}, \hat{\rho}) \models_c C_* \llbracket e_* \rrbracket$

Proof sketch

A simple structural induction on e shows that $(\hat{C}, \hat{\rho}) \models_s e$ iff $(\hat{C}, \hat{\rho}) \models_c C_* \llbracket e \rrbracket$ for all sub-terms e of e_* .

In the case of function application $(t_1^{\ell_1} t_2^{\ell_2})^\ell$ the assumption $(\hat{C}, \hat{\rho}) \subseteq (\hat{C}^\top, \hat{\rho}^\top)$ is used to ensure that $\hat{C}(\ell_1) \subseteq \mathbf{Terms}_*$

Finding the **least solution** of the constraints amounts to computing the **least fixed point** of a function

1. Define a suitable monotone function F_* over analyses that propagates constraints over C and r

$$F_* : \widehat{\text{Cache}}_* \times \widehat{\text{Env}}_* \rightarrow \widehat{\text{Cache}}_* \times \widehat{\text{Env}}_*$$

show that it has a least fixed point $\text{lfp}(F_*)$ that is indeed the **least solution**

2. Compute the $\text{lfp}(F_*)$

Straightforward techniques allow us to compute the least fixed point in $O(n^5)$ time when n is the size of the expression e_*

But we can do better, using **graph formulation of the constraints**: we obtain $O(n^3)$

Constraint graph:

- Nodes
 - a node $C(l)$ for each label $l \in \text{Lab}_*$
 - a node $r(x)$ for each variable $x \in \text{Var}_*$
- Edges
 - Each constraint $p_1 \subseteq p_2$ adds an initial edge (p_1, p_2) to the graph
 - each constraint $\{t\} \subseteq p \implies p_1 \subseteq p_2$ adds
 - a candidate edge (p_1, p_2) , with "trigger" $\{t\} \subseteq p$, and
 - an edge (p, p_2)

Associated with each node p we have a data field $D[p]$ representing the current knowledge. Initially $D[p]$ is given by

$$D[p] = \{t \mid (\{t\} \subseteq p) \in C_*[\![e_*]\!]\}$$

All edges are traversed in order to propagate information from one data field to another

Each **node** p of the constraint graph is associated with a set (Data array) $D[p]$ of values
Goal: find the value set of every node

Initially, we have

$$D[p] = \{t \mid (\{t\} \subseteq p) \in C_*[\![e_*]\!]\}$$

The idea of the algorithm is to iteratively propagate the information from one data field to another according to the constraints

- If we have an edge (p_1, p_2) in our graph, then the value set of p_1 must flow into the value set of p_2



- If a trigger is true then also in the corresponding edge the value set flows

Constraint solving = graph reachability + propagation

Input: a set of constraint $C_*[[e_*]]$

Output: the least solution $(\hat{C}, \hat{p})$ to the constraints (an assignment to variables $C(\ell)$ and $r(x)$)

Data structures:

- a **worklist** W containing the nodes whose data array should be propagated (a list of nodes whose outgoing edges should be traversed)
- a **data array** D storing for each node the current abstract value in $\mathbf{Val}_*$
- an **edge array** E storing for each node a list of the constraints that the node may affect (i.e., a list of constraints from which a list of the successor nodes can be computed)

The **nodes** consist $C(\ell)$ for all labels ℓ and $r(x)$ for all labels x

The solver computes a least fixed point by propagating values until saturation

D = current knowledge

E = how knowledge propagates

W = what still needs to be processed

The algo:

1. Initialize all the data structures D , E and W to $[]$

Build the graph:

Nodes

- a node $C(\ell)$ for each label $\ell \in \mathbf{Lab}_*$
- a node $r(x)$ for each variable $x \in \mathbf{Var}_*$

$$E[p_1] := \text{cons}(p_1 \subseteq p_2, E[p_1])$$

Edges $E[p_1] := \text{cons}(\{t\} \subseteq p \Rightarrow p_1 \subseteq p_2, E[p_1])$

$$E[p] := \text{cons}(\{t\} \subseteq p \Rightarrow p_1 \subseteq p_2, E[p])$$

- Each constraint $p_1 \subseteq p_2$ adds an initial edge (p_1, p_2) to the graph
- Each constraint $\{t\} \subseteq p \Rightarrow p_1 \subseteq p_2$ adds a candidate edge (p_1, p_2) , with “trigger” $\{t\} \subseteq p$, and an edge (p, p_2)

Initial assignments to D : if $(\{t\} \subseteq p) \in C_*[e_*]$ and $\{t\} \not\subseteq D[p]$: $\text{add}(p, \{t\})$

- incorporates $\{t\}$ into $D[p]$ and
- adds p to the worklist W

$$D[p] = \{t \mid (\{t\} \subseteq p) \in C_*[e_*]\}$$

177

2.

Continue propagating the information from one data field to another, along edges, as long as W is non-empty

$E[q]$: list of the constraints that q may affect

We extract the head element q of W and for all constraints cc in $E[q]$

- $cc = p_1 \subseteq p_2$: If we have an edge (p_1, p_2) in our graph, then the value set of p_1 must flow into the value set of p_2 ($\text{add}(p_2, D[p_1])$)
- $cc = \{t\} \subseteq p \Rightarrow p_1 \subseteq p_2$: If the trigger is true ($t \in D[p]$), then also in the corresponding edge the value set flows ($\text{add}(p_2, D[p_1])$)

3.

$$4. \forall l \in \mathbf{Lab}_* : \hat{C}(l) := D[C(l)]$$

$$5. \forall x \in \mathbf{Var}_* : \hat{r}(x) := D[r(x)]$$

Complexity = $O(n^3)$

Correctness of the algorithm

Proposition

Given the input $C_*[[e_*]]$, the algorithm terminates and the result $(\hat{C}, \hat{\rho})$ satisfies

$$(\hat{C}, \hat{\rho}) = \sqcap \{(\hat{C}', \hat{\rho}') \mid (\hat{C}', \hat{\rho}') \models_c C_*[[e_*]]\}$$

and hence it is the least solution to $C_*[[e_*]]$

is the least
solution of the
constraints

As a consequence $(\hat{C}, \hat{\rho}) \models_c C_*[[e_*]]$, i.e., the computed solution satisfies all constraints

Corollary

Assume that $(\hat{C}, \hat{\rho})$ is the solution to the constraints $C_*[[e_*]]$ computed by the algorithm, then $(\hat{C}, \hat{\rho}) \models e_*$

$(\hat{C}, \hat{\rho})$ is a valid
CFA of e_*

Solving constraints yields a sound analysis

Proof sketch

Termination:

(The steps 1,2,4 terminate. We have just to look at step 3).

The algorithm uses a finite set of terms (**Term**_{*}) and monotone updates.

Soundness:

If $(\hat{C}', \hat{\rho}') \models_c C_*[[e_*]]$: the following invariant is maintained:

$$D[C(\ell)] \subseteq \hat{C}'(\ell) \text{ and } D[r(x)] \subseteq \hat{\rho}'(x).$$

Hence $(\hat{C}, \hat{\rho}) \subseteq (\hat{C}', \hat{\rho}')$.

We must prove that $(\hat{C}, \hat{\rho}) \models_c C_*[[e_*]]$. 202

Proof sketch (cont.)

Soundness (cont.):

Assume by contradiction that some constraint $cc \in C_*[[e_*]]$ is not satisfied by $(\hat{C}, \hat{\rho})$. According to the algorithm this could not be true.

$\Rightarrow$ The computed solution satisfies all $cc \in C_*[[e_*]]$, i.e., $(\hat{C}, \hat{\rho}) \models_c cc$.

Minimality:

The result is included in every solution $\Rightarrow$ it is the least solution, computed as

$$\sqcap \{(\hat{C}', \hat{\rho}') \mid (\hat{C}', \hat{\rho}') \models_c C_*[[e_*]]\}$$

Goal: Compute a solution vs check that a solution is acceptable

There are applications where the ability to check $(\hat{C}, \hat{\rho}) \models e_*$ is more important than computing the solution

Scenario

Open system e_0 that uses resources of an **unspecified library** e_*

Analyses and optimizations of e_0 therefore rely on an abstraction $(\hat{C}, \hat{\rho})$ expressing the relevant properties about the library e_* , meaning that

$$(\hat{C}, \hat{\rho}) \models e_*$$

Idea:

Use $(\hat{C}, \hat{\rho})$ as a **specification** of the library e_*

Guarantee

Any other library e'_* such that $(\hat{C}, \hat{\rho}) \models e'_*$ can **safely** replace e_* , since the specification $(\hat{C}, \hat{\rho})$ still guarantees all required properties about the library e'_*

CFA + DFA

We extend Control Flow Analysis with data-flow information

Abstract values = functions + **data-flow properties**

$$\hat{v} \in \widehat{\mathbf{Val}}_d = \wp(\mathbf{Term} \cup \mathbf{Data})$$

Abstract values

Now contain

- possible functions (as before) and
- abstract data-flow properties, (i.e., properties of booleans and integers, such as sign, constantness, or ranges of values)

The data-flow component can be:

- a powerset (e.g., sets of possible values)
- a complete lattice (e.g., intervals, signs, constants)

Abstract domains

$$\hat{v} \in \widehat{\mathbf{Val}}_d = \wp(\mathbf{Term} \cup \mathbf{Data})$$

Constants

Each constant $c \in \mathbf{Const}$ is mapped to an element $d_c \in \mathbf{Data}$ specifying the abstract property of c :

d_c can be thought as $\alpha(c)$ for a suitable abstraction α

Operators are lifted from concrete data to abstract values, i.e., they operate on abstract properties instead of concrete values

To each operator $op \in \mathbf{Op}$, corresponds a total function

$\hat{op} : \widehat{\mathbf{Val}}_d \times \widehat{\mathbf{Val}}_d \rightarrow \widehat{\mathbf{Val}}_d$ telling how op operates on abstract properties

Abstract operators over-approximate concrete operations

Abstract operators

$$\hat{op} : \widehat{\mathbf{Val}}_d \times \widehat{\mathbf{Val}}_d \rightarrow \widehat{\mathbf{Val}}_d$$

We apply the operator to all combinations of abstract data values

Typically, $\hat{op}$ has a definition of the form

$$\hat{v}_1 \hat{op} \hat{v}_2 = \bigcup \{d_{op}(d_1, d_2) \mid d_1 \in \hat{v}_1 \cap \mathbf{Data}, d_2 \in \hat{v}_2 \cap \mathbf{Data}\}$$

for some function $op : \mathbf{Data} \times \mathbf{Data} \rightarrow \wp(\mathbf{Data})$ specifying how the operator computes with the abstract properties of integers and booleans

This over-approximates all possible concrete computations

The acceptability relation of the combined analysis has the form

$$(\hat{C}, \hat{\rho}) \models_d e$$

Each rule over-approximates both control-flow and data-flow

The rules are as before, but operate on enriched abstract values

Key clauses:

- [con]: maps constants to abstract data values
- [if]: uses abstract boolean information to select possible branches
- [op]: uses abstract operators on data

Data-flow information is propagated through the evaluation of expressions

Rules:

- [con] We now track data values: the CFA records that d_c is a possible value of c (Constants introduce abstract data into the analysis)
- [if] Increased precision: we analyze only those branches consistent with the abstract value of the condition, which now is trackable (control-flow is now filtered by data-flow)
- [op] we do track data values, This rule applies abstract operators to propagate data-flow information Data-flow is computed locally using abstract operators

CFA with abstract values as powersets

$$\begin{aligned}
 [\text{con}] \quad & (\hat{C}, \hat{\rho}) \models_d c^\ell \text{ iff } \{d_c\} \subseteq \hat{C}(\ell) \\
 [\text{var}] \quad & (\hat{C}, \hat{\rho}) \models_d x^\ell \text{ iff } \hat{\rho}(x) \subseteq \hat{C}(\ell) \\
 [\text{fn}] \quad & (\hat{C}, \hat{\rho}) \models_d (\text{fn } x \Rightarrow e_0)^\ell \text{ iff } \{\text{fn } x \Rightarrow e_0\} \subseteq \hat{C}(\ell) \\
 [\text{fun}] \quad & (\hat{C}, \hat{\rho}) \models_d (\text{fun } f \ x \Rightarrow e_0)^\ell \text{ iff } \{\text{fun } f \ x \Rightarrow e_0\} \subseteq \hat{C}(\ell) \\
 [\text{app}] \quad & (\hat{C}, \hat{\rho}) \models_d (t_1^{t_1'} t_2^{t_2'})^\ell \\
 & \text{ iff } (\hat{C}, \hat{\rho}) \models_d t_1^{t_1'} \wedge (\hat{C}, \hat{\rho}) \models_d t_2^{t_2'} \wedge \\
 & \quad (\forall (\text{fn } x \Rightarrow t_0^{t_0'}) \in \hat{C}(\ell_1) : \\
 & \quad \quad (\hat{C}, \hat{\rho}) \models_d t_0^{t_0'} \wedge \\
 & \quad \quad \hat{C}(\ell_2) \subseteq \hat{\rho}(x) \wedge \hat{C}(\ell_0) \subseteq \hat{C}(\ell)) \wedge \\
 & \quad (\forall (\text{fun } f \ x \Rightarrow t_0^{t_0'}) \in \hat{C}(\ell_1) : \\
 & \quad \quad (\hat{C}, \hat{\rho}) \models_d t_0^{t_0'} \wedge \\
 & \quad \quad \hat{C}(\ell_2) \subseteq \hat{\rho}(x) \wedge \hat{C}(\ell_0) \subseteq \hat{C}(\ell) \wedge \\
 & \quad \quad \{\text{fun } f \ x \Rightarrow t_0^{t_0'}\} \subseteq \hat{\rho}(f)) \\
 [\text{if}] \quad & (\hat{C}, \hat{\rho}) \models_d (\text{if } t_0^{t_0'} \text{ then } t_1^{t_1'} \text{ else } t_2^{t_2'})^\ell \\
 & \text{ iff } (\hat{C}, \hat{\rho}) \models_d t_0^{t_0'} \wedge \\
 & \quad (d_{\text{true}} \in \hat{C}(\ell_0) \Rightarrow ((\hat{C}, \hat{\rho}) \models_d t_1^{t_1'} \wedge \hat{C}(\ell_1) \subseteq \hat{C}(\ell))) \wedge \\
 & \quad (d_{\text{false}} \in \hat{C}(\ell_0) \Rightarrow ((\hat{C}, \hat{\rho}) \models_d t_2^{t_2'} \wedge \hat{C}(\ell_2) \subseteq \hat{C}(\ell))) \\
 [\text{let}] \quad & (\hat{C}, \hat{\rho}) \models_d (\text{let } x = t_1^{t_1'} \text{ in } t_2^{t_2'})^\ell \\
 & \text{ iff } (\hat{C}, \hat{\rho}) \models_d t_1^{t_1'} \wedge (\hat{C}, \hat{\rho}) \models_d t_2^{t_2'} \wedge \\
 & \quad \hat{C}(\ell_1) \subseteq \hat{\rho}(x) \wedge \hat{C}(\ell_2) \subseteq \hat{C}(\ell) \\
 [\text{op}] \quad & (\hat{C}, \hat{\rho}) \models_d (t_1^{t_1'} \text{ op } t_2^{t_2'})^\ell \\
 & \text{ iff } (\hat{C}, \hat{\rho}) \models_d t_1^{t_1'} \wedge (\hat{C}, \hat{\rho}) \models_d t_2^{t_2'} \wedge \\
 & \quad \hat{C}(\ell_1) \text{ op } \hat{C}(\ell_2) \subseteq \hat{C}(\ell)
 \end{aligned}$$

Observation

The combined analysis does not directly yield a valid 0-CFA solution

Reason

data-flow information restricts the control flow

Intuition

If the condition is known to be false (or true), some branches are not analyzed at all

Even if we restrict to terms:

- $\hat{C}'(\ell) = \hat{C}(\ell) \cap \mathbf{Term}$ and

- $\hat{\rho}'(x) = \hat{\rho}(x) \cap \mathbf{Term}$

this does not guarantee a valid CFA solution, i.e.,

$(\hat{C}, \hat{\rho}) \models_d e$ does not guarantee that $(\hat{C}', \hat{\rho}') \models e$

Abstract Values as Complete Lattices

- Instead of using sets of concrete data, we can use structured abstract domains
- We separate:
 - **Term** information
 - **Data** information

Two components

$$\hat{v} \in \widehat{\mathbf{Val}}_d = \wp(\mathbf{Term} \cup \mathbf{Data}) \equiv \wp(\mathbf{Term}) \times \wp(\mathbf{Data})$$

- replace $\wp(\mathbf{Data})$ by a complete lattice L
- and perform a development closely related to that of the (forward) Monotone Frameworks

A **monotone structure** consists of:

- a complete lattice L , and
- a set F of monotone functions of $L \times L \rightarrow L$

An **instance** of a monotone structure then consists of the structure (L, F)

- a mapping ι from the constants $c \in \mathbf{Const}$ to values in L , and
- a mapping f from the binary operators $op \in \mathbf{Op}$ to functions of F

No need of flow component: it is up to the Control Flow Analysis to determine the flow

We combine CFA and DFA information, by tracking both functions and data simultaneously

Abstract domains

$$\widehat{\mathbf{Val}}_d = \wp(\mathbf{Term}) \times \wp(\mathbf{Data})$$

decoupling

CFA part

DFA part

$$\begin{aligned} \hat{v} &\in \widehat{\mathbf{Val}} = \wp(\mathbf{Term}) \\ \hat{\rho} &\in \widehat{\mathbf{Env}} : \mathbf{Var} \rightarrow \widehat{\mathbf{Val}} \\ \hat{C} &\in \widehat{\mathbf{Cache}} : \mathbf{Lab} \rightarrow \widehat{\mathbf{Val}} \end{aligned}$$

$$\begin{aligned} \hat{d} &\in \widehat{\mathbf{Data}} = L \\ \hat{\delta} &\in \widehat{\mathbf{DEnv}} : \mathbf{Var} \rightarrow \widehat{\mathbf{Data}} \\ \hat{D} &\in \widehat{\mathbf{DCache}} : \mathbf{Lab} \rightarrow \widehat{\mathbf{Data}} \end{aligned}$$

Control-flow and data-flow are computed together

Acceptability relation

$$(\hat{C}, \hat{D}, \hat{\rho}, \hat{\delta}) \models_D e$$

Rules:

- [con] At label l , the abstract data value of must be recorded l tells us which abstract value a constant contributes (Constants inject data-flow information into the analysis)
- [op] Now, we use the component f (Compute the abstract result of the operation and propagate it to the result label)
- [if] analyze condition, use abstract boolean value and propagate only analysed branches

$$\begin{aligned}
[\text{con}] \quad & (\bar{C}, \bar{D}, \hat{\rho}, \hat{\delta}) \models_D e^f \text{ iff } i_e \subseteq \bar{D}(\ell) \\
[\text{var}] \quad & (\bar{C}, \bar{D}, \hat{\rho}, \hat{\delta}) \models_D x^f \text{ iff } \hat{\rho}(x) \subseteq \bar{C}(\ell) \wedge \hat{\delta}(x) \subseteq \bar{D}(\ell) \\
[\text{fn}] \quad & (\bar{C}, \bar{D}, \hat{\rho}, \hat{\delta}) \models_D (\text{fn } x \Rightarrow e_0)^f \text{ iff } \{\text{fn } x \Rightarrow e_0\} \subseteq \bar{C}(\ell) \\
[\text{fun}] \quad & (\bar{C}, \bar{D}, \hat{\rho}, \hat{\delta}) \models_D (\text{fun } f \Rightarrow e_0)^f \text{ iff } \{\text{fun } f \Rightarrow e_0\} \subseteq \bar{C}(\ell) \\
[\text{app}] \quad & (\bar{C}, \bar{D}, \hat{\rho}, \hat{\delta}) \models_D (t_1^{f_1} t_2^{f_2})^f \\
& \text{ iff } (\bar{C}, \bar{D}, \hat{\rho}, \hat{\delta}) \models_D t_1^{f_1} \wedge (\bar{C}, \bar{D}, \hat{\rho}, \hat{\delta}) \models_D t_2^{f_2} \wedge \\
& \quad (\forall (\text{fn } x \Rightarrow e_0^f) \in \bar{C}(\ell_1) : \\
& \quad \quad (\bar{C}, \bar{D}, \hat{\rho}, \hat{\delta}) \models_D e_0^f \wedge \\
& \quad \quad \bar{C}(\ell_2) \subseteq \hat{\rho}(x) \wedge \bar{D}(\ell_2) \subseteq \hat{\delta}(x) \wedge \\
& \quad \quad \bar{C}(\ell_0) \subseteq \bar{C}(\ell) \wedge \bar{D}(\ell_0) \subseteq \bar{D}(\ell) \wedge \\
& \quad (\forall (\text{fun } f \Rightarrow e_0^f) \in \bar{C}(\ell_1) : \\
& \quad \quad (\bar{C}, \bar{D}, \hat{\rho}, \hat{\delta}) \models_D e_0^f \wedge \\
& \quad \quad \bar{C}(\ell_2) \subseteq \hat{\rho}(x) \wedge \bar{D}(\ell_2) \subseteq \hat{\delta}(x) \wedge \\
& \quad \quad \bar{C}(\ell_0) \subseteq \bar{C}(\ell) \wedge \bar{D}(\ell_0) \subseteq \bar{D}(\ell) \wedge \\
& \quad \quad \{\text{fun } f \Rightarrow e_0^f\} \subseteq \hat{\rho}(f)) \\
[\text{if}] \quad & (\bar{C}, \bar{D}, \hat{\rho}, \hat{\delta}) \models_D (\text{if } t_0^{f_0} \text{ then } t_1^{f_1} \text{ else } t_2^{f_2})^f \\
& \text{ iff } (\bar{C}, \bar{D}, \hat{\rho}, \hat{\delta}) \models_D t_0^{f_0} \wedge \\
& \quad (t_{\text{true}} \subseteq \bar{D}(\ell_0) \Rightarrow (\bar{C}, \bar{D}, \hat{\rho}, \hat{\delta}) \models_D t_1^{f_1} \wedge \\
& \quad \quad \bar{C}(\ell_1) \subseteq \bar{C}(\ell) \wedge \\
& \quad \quad \bar{D}(\ell_1) \subseteq \bar{D}(\ell) \wedge \\
& \quad (t_{\text{false}} \subseteq \bar{D}(\ell_0) \Rightarrow (\bar{C}, \bar{D}, \hat{\rho}, \hat{\delta}) \models_D t_2^{f_2} \wedge \\
& \quad \quad \bar{C}(\ell_2) \subseteq \bar{C}(\ell) \wedge \\
& \quad \quad \bar{D}(\ell_2) \subseteq \bar{D}(\ell)) \\
[\text{let}] \quad & (\bar{C}, \bar{D}, \hat{\rho}, \hat{\delta}) \models_D (\text{let } x = t_1^{f_1} \text{ in } t_2^{f_2})^f \\
& \text{ iff } (\bar{C}, \bar{D}, \hat{\rho}, \hat{\delta}) \models_D t_1^{f_1} \wedge (\bar{C}, \bar{D}, \hat{\rho}, \hat{\delta}) \models_D t_2^{f_2} \wedge \\
& \quad \bar{C}(\ell_1) \subseteq \hat{\rho}(x) \wedge \bar{D}(\ell_1) \subseteq \hat{\delta}(x) \wedge \\
& \quad \bar{C}(\ell_2) \subseteq \bar{C}(\ell) \wedge \bar{D}(\ell_2) \subseteq \bar{D}(\ell) \\
[\text{op}] \quad & (\bar{C}, \bar{D}, \hat{\rho}, \hat{\delta}) \models_D (t_1^{f_1} \text{ op } t_2^{f_2})^f \\
& \text{ iff } (\bar{C}, \bar{D}, \hat{\rho}, \hat{\delta}) \models_D t_1^{f_1} \wedge (\bar{C}, \bar{D}, \hat{\rho}, \hat{\delta}) \models_D t_2^{f_2} \wedge \\
& \quad f_{\text{op}}(\bar{D}(\ell_1), \bar{D}(\ell_2)) \subseteq \bar{D}(\ell)
\end{aligned}$$

The combined CFA + DFA is sound w.r.t. the operational semantics

- Same proof techniques as before apply
- The algorithm terminates if L satisfies the **Ascending Chain Condition (ACC)**, as in the case for Monotone Frameworks

no infinite strictly increasing sequences
 $\Rightarrow$ fixpoint computation terminates

- If we change the [if] clause in such a way that the data flow component cannot influence the control flow component, with a modified relation $\models'_D$, then $(\hat{C}, \hat{D}, \hat{\rho}, \hat{\delta}) \models'_D e$ guarantees that $(\hat{C}, \hat{\rho}) \models e$

We can solve:

1. control-flow ($\hat{C}$ and $\hat{\rho}$) first
2. data-flow ($\hat{D}$ and $\hat{\delta}$) afterwards

- The analysis still yields the **least solution** of the combined set of constraints

Uniform k-CFA

- 0-CFA is a **context-insensitive** (or **mono-variant**) analysis
- It does not distinguish various instances of program variables and program points from each other, in particular does not distinguish different calling contexts
- **Context-sensitive** (or **poly-variant**) analysis does distinguish
 - Examples of such analyses include **k-CFA** analyses, uniform **k-CFA** analyses, polynomial **k-CFA** analyses and the Cartesian Product Algorithm
- We focus on **uniform k-CFA** (“uniform” because environment and cache have the same precision)

Goal: distinguish different function calls

Idea: extend $\widehat{\mathbf{Val}}$ to include context information

- **Contexts** $\hat{\delta}$ record the **k** last dynamic call sites, e.g., for **k = 1**, the context amounts just to the label of the expression that called the current function
- In general, contexts are sequences of labels of length at most $\leq k$

$$\hat{\delta} \in \Delta = \mathbf{Lab}^{\leq k}$$

We track information separately for different call strings*

- Same function, different contexts $\Rightarrow$ different analyses

*A **call string** is a list of labels $\ell_1, \dots, \ell_n$ denoting that the last n calls were of the form $(e_{1n} \ e_{2n})^{\ell_n} \dots (e_{11} \ e_{21})^{\ell_1}$ with $(e_{11} \ e_{21})^{\ell_1}$ being the most recent call; we write ϵ for the empty call string

Abstract domains: context environment

- A **context environment** records in which context each variable was defined
- So we associate a context to each variable occurrence

Definition points of free variables of terms

$$ce \in \mathbf{Cenv} = \mathbf{Var} \rightarrow \Delta$$

- The same variable may have different values in different contexts
- The context environment will play a role similar to the environment of the semantics

In our set of values, instead of just the function abstractions, we need to track **closures**, i.e., a pair of a function label and a context environment

- A value is paired with the context environment where it was created
- We shall extend the abstract values as

$$\hat{v} \in \widehat{\mathbf{Val}} = \wp(\mathbf{Val} \times \mathbf{Cenv})$$

- Abstract environment $\hat{\rho}$ and abstract cache $\hat{C}$ will now map a variable (a label, resp.) and a context to an abstract value

$$\begin{aligned}\hat{\rho} \in \widehat{\mathbf{Cenv}} &= (\mathbf{Var} \times \Delta) \rightarrow \widehat{\mathbf{Val}} \\ \hat{C} \in \widehat{\mathbf{Cache}} &= (\mathbf{Lab} \times \Delta) \rightarrow \widehat{\mathbf{Val}}\end{aligned}$$

- Values are now indexed by context
- This treatment prevents merging values from different calls (as in the previous example)

- In **uniform k -CFA** we have a cache for each possible context

$$\hat{C} \in \widehat{\mathbf{Cache}} = (\mathbf{Lab} \times \Delta) \rightarrow \widehat{\mathbf{Val}}$$

where Δ : set of call strings (length $\leq k$)

- In the **classical k -CFA**, we have instead a cache for each possible full environment

more expensive

$$\hat{C} \in \widehat{\mathbf{Cache}} = (\mathbf{Lab} \times \mathbf{CEnv}) \rightarrow \widehat{\mathbf{Val}}$$

- In the **polynomial k -CFA**, instead of $\widehat{\mathbf{Val}} = \wp(\mathbf{Val} \times \mathbf{Cenv})$ we have

$$\hat{v} \in \widehat{\mathbf{Val}} = \wp(\mathbf{Val} \times \Delta)$$

$$(\hat{C}, \hat{\rho}) \models_{\delta}^{ce} e$$

- ce is the current context environment:
 - it tracks the context in which each variable of e is bound
 - it changes when variables are bound
- $\hat{\delta}$ is the current calling context
 - it changes when functions are applied

The formula expresses that $(\hat{C}, \hat{\rho})$ is an acceptable analysis of e in the context specified by ce and $\hat{\delta}$

Acceptability is now **context-dependent**: we analyse e in a specific calling context $\hat{\delta}$

Rules:

- [var] We look up in the context where it was bound and propagate it to the current context
 - $ce(x)$ context where x was defined
 - δ current context (where we use x)
 - A variable carries the context where it was created, but is used in the current context
- [fn]

The functional term $fn\ x \Rightarrow e_0$ must be included in $\hat{C}(\ell, \delta)$ together with the current context environment ce , restricted to the free variables of the function (denoted ce_0)

Functions are represented as **closures**: a function is paired with the context in which it is defined, restricted to its free variables

- [let] check sub-exprs,
 - the definition context of the bound variable of the let-construct is the current context (as no application point is passed)

Double flow: propagation of the values

 - from $\hat{C}(\ell_1, \delta)$ to $\hat{\rho}(x, \delta)$ **[binding]**
 - from $\hat{C}(\ell_2, \delta)$ to $\hat{C}(\ell, \delta)$ **[evaluation]**
- [app] both are bound in the new calling context δ_0 when analyzing the body

Uniform k -CFA

[con] $(\bar{C}, \bar{\rho}) \models_{\delta}^{ce} c^{\ell}$ always

[var] $(\bar{C}, \bar{\rho}) \models_{\delta}^{ce} x^{\ell}$ iff $\bar{\rho}(x, ce(x)) \subseteq \bar{C}(\ell, \delta)$

[fn] $(\bar{C}, \bar{\rho}) \models_{\delta}^{ce} (\mathbf{fn} \ x \Rightarrow e_0)^{\ell}$ iff $\{(\mathbf{fn} \ x \Rightarrow e_0, ce_0)\} \subseteq \bar{C}(\ell, \delta)$
where $ce_0 = ce \mid FV(\mathbf{fn} \ x \Rightarrow e_0)$

[fun] $(\bar{C}, \bar{\rho}) \models_{\delta}^{ce} (\mathbf{fun} \ f \ x \Rightarrow e_0)^{\ell}$ iff $\{(\mathbf{fun} \ f \ x \Rightarrow e_0, ce_0)\} \subseteq \bar{C}(\ell, \delta)$
where $ce_0 = ce \mid FV(\mathbf{fun} \ f \ x \Rightarrow e_0)$

[app] $(\bar{C}, \bar{\rho}) \models_{\delta}^{ce} (t_1^{\ell_1} \ t_2^{\ell_2})^{\ell}$
iff $(\bar{C}, \bar{\rho}) \models_{\delta}^{ce} t_1^{\ell_1} \wedge (\bar{C}, \bar{\rho}) \models_{\delta}^{ce} t_2^{\ell_2} \wedge$
 $(\forall (\mathbf{fn} \ x \Rightarrow t_0^{\ell_0}, ce_0) \in \bar{C}(\ell_1, \delta) :$
 $(\bar{C}, \bar{\rho}) \models_{\delta_0}^{ce_0} t_0^{\ell_0} \wedge$
 $\bar{C}(\ell_2, \delta) \subseteq \bar{\rho}(x, \delta_0) \wedge \bar{C}(\ell_0, \delta_0) \subseteq \bar{C}(\ell, \delta)$
where $\delta_0 = \lceil \delta, \ell \rceil_k$
and $ce_0' = ce_0[x \mapsto \delta_0] \wedge$
 $(\forall (\mathbf{fun} \ f \ x \Rightarrow t_0^{\ell_0}, ce_0) \in \bar{C}(\ell_1, \delta) :$
 $(\bar{C}, \bar{\rho}) \models_{\delta_0}^{ce_0} t_0^{\ell_0} \wedge$
 $\bar{C}(\ell_2, \delta) \subseteq \bar{\rho}(x, \delta_0) \wedge \bar{C}(\ell_0, \delta_0) \subseteq \bar{C}(\ell, \delta) \wedge$
 $\{(\mathbf{fun} \ f \ x \Rightarrow t_0^{\ell_0}, ce_0)\} \subseteq \bar{\rho}(f, \delta_0)$
where $\delta_0 = \lceil \delta, \ell \rceil_k$
and $ce_0' = ce_0[f \mapsto \delta_0, x \mapsto \delta_0])$

[if] $(\bar{C}, \bar{\rho}) \models_{\delta}^{ce} (\mathbf{if} \ t_0^{\ell_0} \mathbf{then} \ t_1^{\ell_1} \mathbf{else} \ t_2^{\ell_2})^{\ell}$
iff $(\bar{C}, \bar{\rho}) \models_{\delta}^{ce} t_0^{\ell_0} \wedge (\bar{C}, \bar{\rho}) \models_{\delta}^{ce} t_1^{\ell_1} \wedge (\bar{C}, \bar{\rho}) \models_{\delta}^{ce} t_2^{\ell_2} \wedge$
 $\bar{C}(\ell_1, \delta) \subseteq \bar{C}(\ell, \delta) \wedge \bar{C}(\ell_2, \delta) \subseteq \bar{C}(\ell, \delta)$

[let] $(\bar{C}, \bar{\rho}) \models_{\delta}^{ce} (\mathbf{let} \ x = t_1^{\ell_1} \mathbf{in} \ t_2^{\ell_2})^{\ell}$
iff $(\bar{C}, \bar{\rho}) \models_{\delta}^{ce} t_1^{\ell_1} \wedge (\bar{C}, \bar{\rho}) \models_{\delta}^{ce'} t_2^{\ell_2} \wedge$
 $\bar{C}(\ell_1, \delta) \subseteq \bar{\rho}(x, \delta) \wedge \bar{C}(\ell_2, \delta) \subseteq \bar{C}(\ell, \delta)$
where $ce' = ce[x \mapsto \delta]$

[op] $(\bar{C}, \bar{\rho}) \models_{\delta}^{ce} (t_1^{\ell_1} \ \mathbf{op} \ t_2^{\ell_2})^{\ell}$ iff $(\bar{C}, \bar{\rho}) \models_{\delta}^{ce} t_1^{\ell_1} \wedge (\bar{C}, \bar{\rho}) \models_{\delta}^{ce} t_2^{\ell_2}$

Soundness

The analysis is sound w.r.t. the operational semantics

Implementation

The analysis can be implemented via constraint solving / fixpoint iteration

Trade-off

- Higher $k \Rightarrow$ more precision
- But exponential worst-case complexity

- even for $k = 1$: **exponential worst case complexity**

Why? (intuition)

- Program size: n , number of variables: p

exponential blow up comes from the number of abstract closure environments:
each variable can be associated with one term

Contexts:

- Δ has $O(n)$ elements $\Rightarrow O(n \cdot p)$ different pairs (x, δ) and $O(n \cdot n) = O(n^2)$ different pairs (ℓ, δ)

Abstract states:

- $(\hat{C}, \hat{\rho})$ can be seen as an $O(n^2)$ tuple of values from $\widehat{\mathbf{Val}}$

closures

Values:

- $\widehat{\mathbf{Val}}$ itself is a powerset of pairs of the form $(t, ce) \Rightarrow O(n \cdot n^p)$ such pairs,
- Hence $\widehat{\mathbf{Val}}$ has height $O(n \cdot n^p)$
- Since $p = O(n)$, we have the **exponential worst case complexity**

- Here for $k = 0$

Why? (intuition)

- Program size: n , number of variables: p

No contexts:

- Δ is a singleton ($O(1)$) $\Rightarrow O(1 \cdot p)$ different pairs (x, δ) and $O(1 \cdot n)$ different pairs (ℓ, δ)

Abstract states:

- $(\hat{C}, \hat{\rho})$ can be seen as an $O(p + n)$ tuple of values from $\widehat{\mathbf{Val}}$

Values:

- $\widehat{\mathbf{Val}}$ (powerset of pairs of the form $(t, ce) \Rightarrow$ but here only $(t, ce_0) \Rightarrow O(n)$
- $\widehat{\mathbf{Val}}$ has height $O(n)$ and we have the **polynomial complexity** we already saw for non contextual analyses

Problem:

- In k -CFA, $\widehat{\mathbf{Val}} = \wp(\mathbf{Val} \times \mathbf{Cenv})$
- Context environments cause exponential blow-up

Idea 1: reduce lattice height

- Using abstraction to limit $\widehat{\mathbf{Val}}$
- Standard abstract interpretation techniques

Idea 2: simplify contexts $\widehat{\mathbf{Val}} = \wp(\mathbf{Val} \times \mathbf{Cenv})$

Replace context environments with contexts only

$$\widehat{\mathbf{Val}} = \wp(\mathbf{Term} \times \Delta)$$

closures = function +
full environment

closures = function +
call string

- No full environment
- Only call strings δ

Effect:

- Lattice of polynomial height
- No combinations of variables inside ce
- Number of abstract values drastically reduced
- Less precision, but much cheaper

Cartesian Product Algo (CPA)

term

$t ::= \dots$

$| \text{fn } x_1, \dots, x_m \Rightarrow e_b \text{ // function definition}$
(with m arguments)

$e ::= t^\ell \quad | \quad e_0(e_1, \dots, e_m) \text{ // function application}$
(with m arguments)

$| \dots$

- **well-formedness condition:** all function abstractions are closed, i.e.,
 $\text{FV}(\text{fn } x_1, \dots, x_m \Rightarrow e_0) = \emptyset$
- we do not treat recursive functions

We extend the analysis to take **context** into account

In this variant, **contexts** are **tuples of argument values**

$$\delta \in \Delta = \mathbf{Term}^m$$

$$\hat{v} \in \widehat{\mathbf{Val}} = \wp(\mathbf{Term})$$

$$\hat{\rho} \in \widehat{\mathbf{Cenv}} = (\mathbf{Var} \times \Delta) \rightarrow \widehat{\mathbf{Val}}$$

$$\hat{C} \in \widehat{\mathbf{Cache}} = (\mathbf{Lab} \times \Delta) \rightarrow \widehat{\mathbf{Val}}$$

The resulting analysis is a refinement of 0-CFA that partitions the flow of values to expressions and variables according to the actual argument types in their dynamic calling context

Rules:

- [app] each argument flows to its corresponding parameter

- The function body is analysed separately for each possible tuple of arguments
- No merging across different tuples
- This creates redundancy: it leads to many re-analyses of the same body: we need **memoisation** so that the bodies are only analysed once
- We memoise analyses of each $(\hat{C}, \hat{\rho}) \models_{CPA}^{\delta} t_b^{\ell_b}$ as *templates* (with entries like $t_b^{\ell_b}$ associated with $x \mapsto f, y \mapsto a$) and maintain them in a **global pool**
- when a template is created it is only added to the pool if it is not already present (template reuse)

- The analysis can be implemented in a straightforward manner because the product grows monotonically:
- $\hat{C}(\ell_1, \delta) \times \dots \times \hat{C}(\ell_m, \delta)$ will increase each time one of the $\hat{C}(\ell_i, \delta)$ increases (assuming that none is empty)

The evolution of CFA for FUN

0-CFA

Context-insensitive and flow-insensitive flow analysis (merges all calls)

Polynomial-time higher-order analysis

Syntax-directed CFA

Analyze each body once at definition site, but analyzes unreachable code

Constraint-based formulation

Context-sensitive CFA (k-CFA style)

Track calling contexts and closure environments

Distinguishes different calls

Cartesian Product Algorithm

Analyze bodies for each argument tuple

Larger abstract state space

For all CFAs, semantics are preserved, and a (least) solution exists.

Moreover, an implementation can be provided

Control-flow analysis for higher-order languages is a tradeoff between:

precision vs computational cost

More context $\Rightarrow$ fewer spurious flows $\Rightarrow$ higher complexity

The key challenge of CFA is balancing:

- soundness
- precision
- scalability

0-CFA π -calculus

Calculus of Communicating Systems (CCS)

π -calculus: syntax

Prefixes

$\pi ::= \bar{u}\langle v \rangle$ // send v on u
| $u(x)$ // receive on u and store in x
| τ // internal action

Processes

$P ::= \mathbf{0}$ // inactive process
| $\pi . P$ // action prefix
| $[x = y]P$ // matching (like, if $x = y$ then P)
| $(\nu n)P$ // restriction (like n only known in P)
| $\sum_{i \in I} \pi_i . P_i$ // non deterministic, guarded choice
| $P_1 | P_2$ // parallel composition
| $!P$ // replication (like $P | P | P | P | \dots$)

often omitted

sometimes written $P \setminus n$

nullary sum: $\mathbf{0}$
binary sum: $\pi_1 . P_1 + \pi_2 . P_2$

- Communication primitives: simple and powerful
- Scoping Rules: explicitly control the access to channels and to data

send z on x

Communication (on channel x)

$$\bar{x}\langle z \rangle . P \mid x(y) . Q \rightarrow P \mid Q[z/y]$$

receive y from x

y has been substituted for z in the second process

Extrusion of z

$$(\nu z) (\bar{x}\langle z \rangle . P) \mid x(y) . Q \rightarrow (\nu z) (P \mid Q[z/y])$$

- y has been substituted for z in the second process
- the scope of z has been extended to include both processes

7

A central component of the π -calculus is that of names
The only binders are input prefix and name restriction

$\pi ::=$

$\mid \bar{u}\langle v \rangle$ free names
 $\mid u(v)$ bound variable
 $\mid \tau$
 $\mid \dots$

disjoint union not present in the original π -calculus

$P ::= \dots$ bound constant
 $\mid (\nu n)$
 $\mid \dots$

Name = **Const** $\uplus$ **Var**

The notion of free and bound constants and variables are defined as expected:
 $fc(P)$ and $fv(P)$

- The notion of free and bound names (constants and variables) are defined as usual: we focus on $fc(P)$ and $fv(P)$
- The notion of **capture avoiding substitution** is defined as usual: $P[y/x]$
- Processes are taken up-to **alpha-renaming of bound names**:
 - $u(y) . P = u(z) . (P[z/y])$ if $z \notin fv(P)$
 - $(\nu n) . P = (\nu n') . (P[n'/n])$ if $n' \notin fc(P)$
- We focus on **closed processes**, i.e., process P such that $fc(P) = fv(P) = \emptyset$

$P \rightarrow Q$ = reduction relation (process form state P to state Q after interaction)

Abelian monoid laws for parallel

$$(P \mid Q) \mid R \equiv P \mid (Q \mid R)$$

$$P \mid Q \equiv Q \mid P$$

$$P \mid \mathbf{0} \equiv \mathbf{0}$$

Scope laws

$$(\nu n)(\nu m)P \equiv (\nu m)(\nu n)P$$

$$(\nu n)\mathbf{0} \equiv \mathbf{0}$$

$$(\nu n)(P \mid Q) \equiv (\nu m)P \mid Q \text{ if } n \notin \text{fc}(Q)$$

Matching

$$[x = x]P \equiv P$$

Summands can be permuted in

$$\sum_{i \in I} \pi_i . P_i$$

Unfolding of replication

$$!P \equiv P \mid !P$$

α -renaming

$$P \equiv_{\alpha} Q \Rightarrow P \equiv Q$$

Reduction semantics

$$[Com] (\bar{n}\langle m \rangle . P + P') \mid (n(x) . Q + Q') \rightarrow P \mid Q[m/x]$$

$$[Tau] (\tau . P + Q) \rightarrow P$$

$$[Res] \frac{P \rightarrow P'}{(\nu n)P \rightarrow (\nu n)P'}$$

$$[Par] \frac{P \rightarrow P'}{P \mid Q \rightarrow P' \mid Q}$$

$$[Eq] \frac{P \equiv Q \quad Q \rightarrow Q' \quad Q' \equiv P'}{P \rightarrow P'}$$

How is **abstract behaviour** described?

As a pair of abstract domains given as functions (ρ, κ)

- **Abstract values** abstracts **sets of constants**
- An **abstract environment** ρ associates a variable to the set of constants values it might be bound to
- An **abstract channel environment** κ maps a (channel) constant to the set of constants that may be communicated over it
- ρ and κ contain global information that captures information about the behaviour of the process P_* under analysis

$$\begin{aligned}\mathbf{Val} &= \wp(\mathbf{Const}) \\ \rho : \mathbf{Var} &\rightarrow \wp(\mathbf{Const}) \\ \kappa : \mathbf{Var} &\rightarrow \wp(\mathbf{Const})\end{aligned}$$

These domains can be turned into complete lattices by extending the subset ordering $\subseteq$ on $\wp(\mathbf{Const})$ in a pointwise manner

We extend the abstract env ρ to constants by setting $\rho(n) = \{n\}$

In particular the analysis predicts both possible executions

A guess (ρ, κ) is acceptable if it safely over-approximates the runtime behavior

Validation:

check clauses on logical judgements

- for process $(\hat{\rho}, \hat{\kappa}) \models_P e$
- for actions $(\hat{\rho}, \hat{\kappa}) \models_A \pi$

CFA Clauses for π -calculus

$$[res] (\rho, \kappa) \models_P (\nu n)P \text{ iff } (\rho, \kappa) \models_P P$$

$$[par] (\rho, \kappa) \models_P P_1 \mid P_2 \text{ iff } (\rho, \kappa) \models_P P_1 \wedge (\rho, \kappa) \models_P P_2$$

$$[sum] (\rho, \kappa) \models_P \sum_{i \in I} \pi_i . P_i \text{ iff } \forall i \in I : ((\rho, \kappa) \models_A \pi_i \wedge (\rho, \kappa) \models_P P_i)$$

$$[rep] (\rho, \kappa) \models_P !P \text{ iff } (\rho, \kappa) \models_P P$$

$$[match] (\rho, \kappa) \models_P [x = y]P \text{ iff } (\rho, \kappa) \models_P P$$

$$[nil] (\rho, \kappa) \models_P \mathbf{0} \text{ iff true}$$

$$[out] (\rho, \kappa) \models_A \bar{u}\langle v \rangle \text{ iff } \forall n \in \rho(u) : \rho(v) \subseteq \kappa(n)$$

$$[in] (\rho, \kappa) \models_A u(x) \text{ iff } \forall n \in \rho(u) : \kappa(n) \subseteq \rho(x)$$

$$[tau] (\rho, \kappa) \models_A \tau \text{ iff true}$$

The analysis information is a safe description of what will happen during the evaluation of the program, i.e., a sound over-approximation of the runtime behaviour

This Control Flow Analysis is **Context-Insensitive** and is called 0-CFA

$$P_1 \triangleq a(x) \mid \bar{a}\langle b \rangle \quad P_2 \triangleq a(x) + \bar{a}\langle b \rangle \quad P_3 \triangleq a(x) . \bar{a}\langle b \rangle$$

- Note that the variable y in P_1 can be bound to b , while in P_2 and in P_3 it cannot
- Instead, in the CFA estimate, we have that $\rho(x) \supseteq \{b\}$ in all the three cases

$[n]$ = is the equivalence class corresponding to n (canonical name)

we use a disciplined α -renaming $\Rightarrow$ names can only be substituted with names in the same equivalence class

We start from the process of interest P_* and we lift $\lfloor \cdot \rfloor$ to obtain $\lfloor P_* \rfloor$

Lemma

If $P \equiv Q$ then $(\rho, \kappa) \models_P \lfloor P \rfloor$ iff $(\rho, \kappa) \models_P \lfloor Q \rfloor$

Substitution lemma

$(\rho, \kappa) \models_P \lfloor P \rfloor$ then $(\rho, \kappa) \models_P \lfloor P[m/x] \rfloor$, provided that $\lfloor m \rfloor \subseteq \rho(x)$

Subject reduction result

$(\rho, \kappa) \models_P \lfloor P \rfloor$ and $P \rightarrow Q$ then $(\rho, \kappa) \models_P \lfloor Q \rfloor$

The set of proposed solutions can be partially ordered:

$(\rho, \kappa) \subseteq (\rho', \kappa')$ iff ordered pointwise

Moore family

The set $\{(\rho, \kappa) \models_P P\}$ is a Moore family for all processes P

Therefore there **always** exists a **least (best) solution** (ρ, κ)

obtained as $\sqcap \{(\rho, \kappa) \models_P P\}$

Polyadic π -calculus: syntax

$\pi ::= \bar{u} \langle \vec{v} \rangle$ // send v on u
| $u(\vec{x})$ // receive on u and store in v
| τ // internal action

Polyadic send and receive

$P ::= \mathbf{0}$ // inactive process
| $\pi . P$ // action prefix
| $[x = y]P$ // matching (like, if $x = y$ then P)
| $(\nu n)P$ // restriction (like n only known in P)
| $\sum_{i \in I} \pi_i . P_i$ // non deterministic choice
| $P_1 \mid P_2$ // parallel composition
| $!P$ // replication (like $P \mid P \mid P \mid P \mid \dots$)

Reduction semantics

$$[Com] (\bar{n}\langle\vec{m}\rangle . P + P') \mid (n(\vec{x}) . Q + Q') \rightarrow P \mid Q[\vec{m}/\vec{x}] \text{ if } |\vec{m}| = |\vec{x}|$$

now arity matters

$$[Tau] (\tau . P + Q) \rightarrow P$$

$$[Res] \frac{P \rightarrow P'}{(\nu n)P \rightarrow (\nu n)P'}$$

$$[Par] \frac{P \rightarrow P'}{P \mid Q \rightarrow P' \mid Q}$$

$$[Eq] \frac{P \equiv Q \quad Q \rightarrow Q' \quad Q' \equiv P'}{P \rightarrow P'}$$

- An **abstract environment** ρ associates a variable to the set of constants values it might be bound to
- An **abstract channel environment** κ maps a (channel) constant to the set of sequences of constants that may be communicated over it
- An **error component** ψ records the set of (channel) constants where there may be an arity mismatch in a communication
- ρ and κ contain global information that captures information about the behaviour of the process P_* under analysis, while ψ captures local information

$$\begin{aligned} \mathbf{Val} &= \wp(\mathbf{Const}) \\ \rho : \mathbf{Var} &\rightarrow \wp(\mathbf{Const}) \\ \kappa : \mathbf{Var} &\rightarrow \wp(\mathbf{Const}) \\ \psi &: \wp(\mathbf{Const}) \end{aligned}$$

Given a guess (ρ, κ, Ψ) , we validate whether it is an acceptable CFA. A guess is acceptable if it safely over-approximates the runtime behavior

Validation:

check clauses on logical judgements

- for processes $(\hat{\rho}, \hat{\kappa}) \models_P e : \Psi$
- for actions $(\hat{\rho}, \hat{\kappa}) \models_P \pi : \Psi$

A **context** C is formally given by

$$C ::= (\nu n)C \mid C \mid P \mid [.]$$

A context essentially is a **process with a hole** $[.]$ where another process can be inserted

- we write $C[P]$ for the process obtained by inserting P in the hole of C

Well-behavedness guarantees that communicating actions have **matching arities**

- The process P_* is **dynamically well-behaved** if whenever P evolves into a process that is structurally congruent to one of the form

$$C[(\bar{n}\langle \vec{m} \rangle . R + R') \mid (n(\vec{x}) . Q + Q')] \text{ for some context } C \text{ then } |\vec{m}| = |\vec{x}|$$

context

- The process P_* is **statically well-behaved** if there exist ρ and κ such that

$$(\rho, \kappa) \models_P P_* : \emptyset$$

meaning there is a valid analysis result for P_* where the error component ψ is empty

If the process P_* is statically well-behaved then it is also dynamically wellbehaved

Idea: Channels are not all interchangeable: **each channel carries values of a specific shape**

In π -calculus, any name can be: sent, received, used as a channel, or communicated over any other channel

Extremely expressive, but also too permissive, e.g., we could allow both

$$a\langle b \rangle \text{ and } a\langle b, c \rangle$$

on the same channel a

To discipline communication we can use **sorts**

A **sort** specifies

- the arity of the communicated tuples, and
- the sorts of their components (i.e., what kind of tuples can be communicated on a channel)

We have a set of **sorts** and require that each constant n has a sort

$$\Sigma(n) \in \text{Sort}$$

- A sorting is defined as a function

$$\Sigma : \text{Sort} \rightarrow \text{Sort}^*$$

that for each sort specifies a sequence of sorts describing not only the arity of the constants with that sort but also the sorts of the constants that may be communicated over it

- The sorts are preserved by disciplined α -renaming so if $[n] = [m]$ also $\sigma(n) = \sigma(m)$
- We extend σ in a component-wise manner to work on tuples as well as sets of tuples (sorting can then be checked componentwise)

- The process P_* is **dynamically well-sorted** if whenever P evolves into a process that is structurally congruent to one of the form $C[(\bar{n}\langle \vec{m} \rangle . R + R') \mid (n(\vec{x}) . Q + Q')]$ for some context C then $|\vec{m}| = |\vec{x}|$ as well as $\Sigma(\sigma(m)) = \sigma(\vec{m})$

- The process P_* is **statically well-sorted** if there exist ρ and κ such that

$$(\rho, \kappa) \models_P P_* : \emptyset$$

and furthermore, for all constants n ,

$$\sigma(\kappa(n)) \subseteq \{\Sigma(\sigma(\vec{n}))\}$$

abstract values are
tuples of names

everything CFA predicts for channel n
has the correct shape prescribed by Σ

If the process P_* is statically well-sorted then it is also dynamically wellsorted

We classify names as **secret** and **public** and use the analysis to impose a confidentiality policy on the processes, ensuring that they do not send **secret** constants on **public** channels

We introduce a mapping $\Sigma : \mathbf{Const} \rightarrow \{\text{secret}, \text{public}\}$

We assume that if $[n] = [m]$ then $\Sigma(n) = \Sigma(m)$

We extend Σ in a component-wise manner to work on tuples

Property: a secret channel cannot be sent on a public channel

Dynamic property

The process P_* is **dynamically secrecy-preserving** if whenever P_* evolves into a process that is structurally congruent to one of the form $C[(\bar{n}\langle\vec{m}\rangle).P + P' \mid (n(\vec{x}).Q + Q')]$, then $|\vec{m}| = |\vec{x}|$ as well as $\Sigma(n) = \text{public} \Rightarrow \Sigma(\vec{m}) = \text{public}$

Static property

The process P_* is **statically secrecy-preserving** if

- there exists ρ and κ such that $(\rho, \kappa) \models_P P_*$ and
- κ is such that

$$\Sigma(n) = \text{public} \Rightarrow \forall \vec{m} \in \kappa(n) : \Sigma(\vec{m}) = \text{public}$$

If the process P_* is statically secrecy-preserving then it is also dynamically secrecy-preserving

We assign security levels to the channels and use the analysis to impose a confidentiality policy on the processes, ensuring that they do not send constants with a high security level on channels with a lower security level For simplicity we only impose two levels, **low** and **high** (**low** $\sqsubseteq$ **high**), and we want to ensure that only low information is communicated on low channels

We introduce a mapping $\Lambda : \mathbf{Const} \rightarrow \{\text{low}, \text{high}\}$ that to each constant associates a *security level*

security lattice

We extend Λ in a component-wise manner to work on tuples

We assume that if $[n] = [m]$ then $\Lambda(n) = \Lambda(m)$

Property: the level of the msg cannot be higher than the level of the channel

Dynamic property

The process P_* is **dynamically non-leaking** if

whenever P_* evolves into a process that is structurally congruent to one of the form $C[(\bar{n}\langle\vec{m}\rangle).P + P' \mid (n(\vec{x}).Q + Q')]$ then $|\vec{m}| = |\vec{x}|$ as well as

$$\Lambda(\vec{m}) \sqsubseteq \Lambda(n)^{|\vec{m}|}$$

Static property

The process P_* is **statically non-leaking** if

- there exists ρ and κ such that $(\rho, \kappa) \models_P P_*$ and
- κ is such that $\forall \vec{m} \in \kappa(n) : \Lambda(\vec{m}) \sqsubseteq \Lambda(n)^{|\vec{m}|}$ for all constants n

Every value predicted by the CFA for channel n must be allowed by the security policy
So the CFA cache becomes an approximation of possible information flows

the abstract cache κ becomes a lightweight information-flow analysis

If the process P_* is statically non-leaking then it is also dynamically non-leaking

We introduce a mapping $\Theta : \mathbf{Const} \rightarrow \{\text{tainted}, \text{untainted}\}$

We assume that if $[n] = [m]$ then $\Theta(n) = \Theta(m)$

We extend Θ in a component-wise manner to work on tuples

A **sink** is a program point or channel where tainted data must not arrive, e.g., a channel where confidential or untrusted data must not flow

Property: a tainted value should never reach a sensitive sink

Taint safety

Dynamic property

The process P_* is **dynamically taint-safe** if

whenever P_* evolves into a process that is structurally congruent to one of the form $C[(\bar{n}\langle\vec{m}\rangle).P + P' \mid (n(\vec{x}).Q + Q')]$ and n is a sensitive sink channel then $|\vec{m}| = |\vec{x}|$ as well as

$$\Theta(\vec{m}) = \{\text{untainted}\}$$

Static property

The process P_* is **statically taint-safe** if

- there exists ρ and κ such that $(\rho, \kappa) \models_P P_*$ and for every sensitive sink channel n
- κ is such that

$$\forall \vec{m} \in \kappa(n) : \Theta(\vec{m}) = \{\text{untainted}\}$$

for all constants n

If the process P_* is statically taint-safe then it is also dynamically taint-safe

CFA vs Type Systems

- Type Systems are **prescriptive**, i.e. they infer types and impose the well-formedness conditions at the same time
- Control Flow Analysis is mainly **descriptive**, i.e. it merely infers the information and then leaves it to a separate step to actually impose demands on when programs are well-formed
- For each property, it is often the case that: a new ad hoc **type system** is necessary, while only a new test on the same **CFA analysis** is needed